Universidad Escuela Colombiana de Ingeniería Julio Garavito

Desarrollo Orientado por Objetos

DOPO

Presentado por:

Wilson Steven Mendivelso Sanabria

David Santiago Garzon Rodriguez

Docente:

María Irma Diaz Rozo

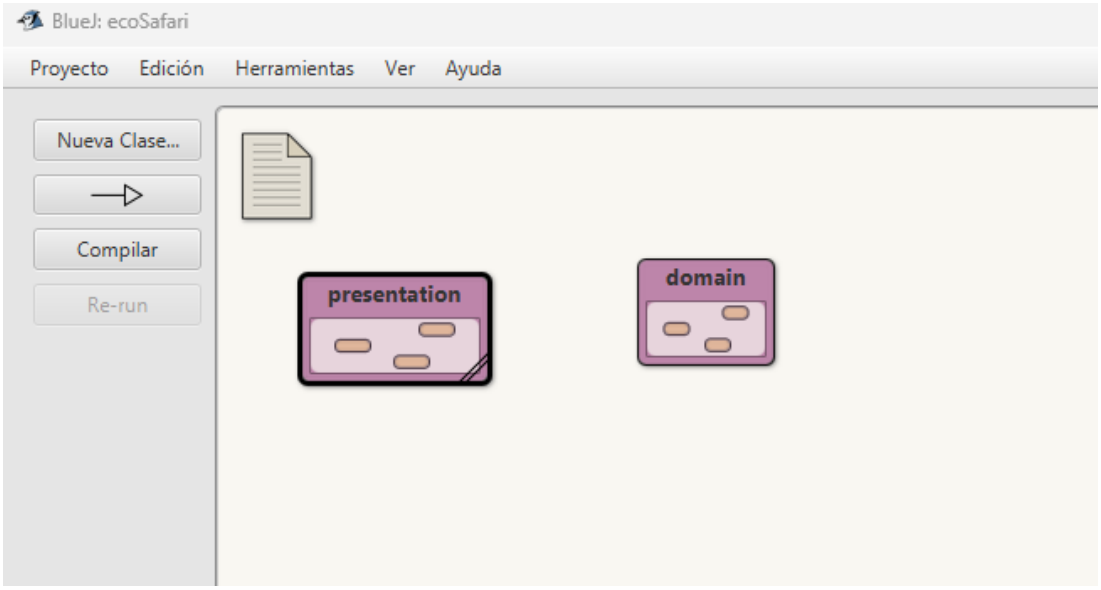
PARTE I. Conociendo el proyecto

A. Revisando línea base:

1. En el directorio descarguen los archivos contenidos de safari.zip. Revisen el código:

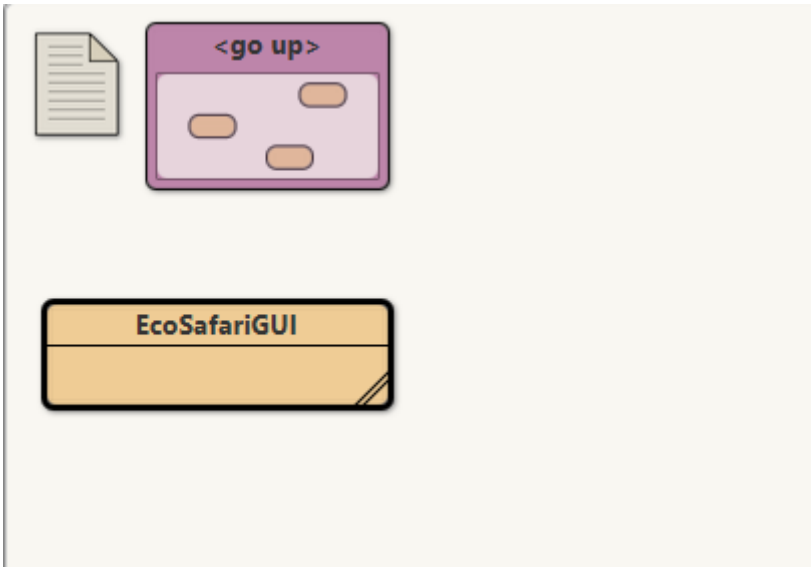
   a) ¿Cuántos paquetes tiene?

   Tiene 2 paquetes



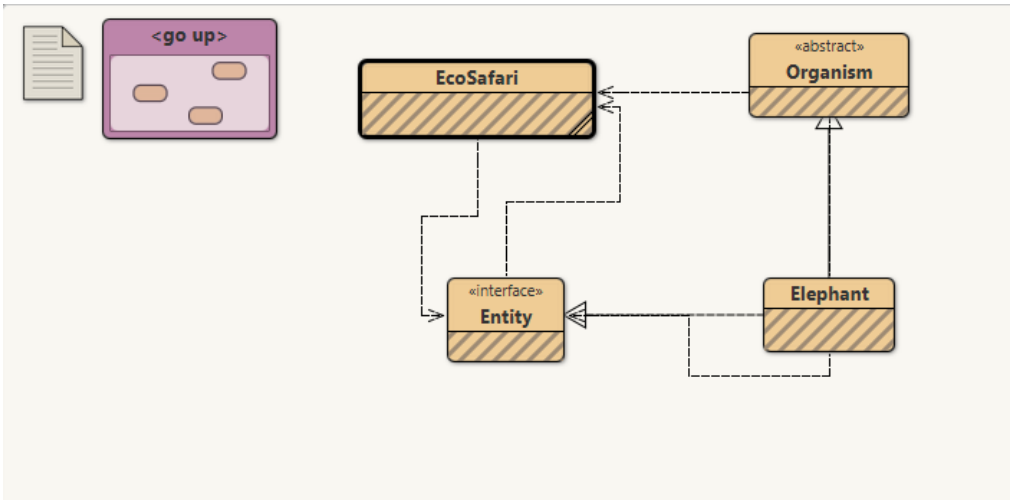
   b) ¿Cuál es el propósito del paquete de presentación?

   Su propósito es mostrar y ejecutar la mini-aplicación de manera que se pueda ver (o sea GUI).



c) ¿Cuál es el propósito del paquete de dominio?

El propósito es que en este paquete esta toda la lógica de la miniaplicación (código fuerte)



2. Revisen el paquete de dominio:

a) ¿Cuáles son los diferentes tipos de componentes de este paquete?

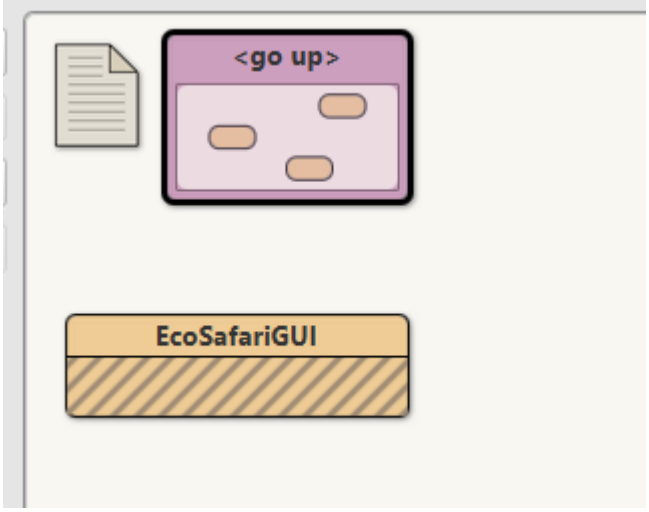
Los diferentes componentes son interfaz, concreta y abstracta

b) ¿Qué implica cada uno de estos tipos de componentes?

La interfaz sirve para que otros métodos implementen cosas de ella, sin embargo, no permite instanciarse. Las clases abstractas, tampoco se instancian, pero permite que a la hora de tener hijos estos tengan todo tipo de atributos y todo tipo de métodos. Por último, las clases concretas solo permiten métodos concretos, permiten atributos de todo tipo y si se pueden instanciar.

3. Revisen el paquete de presentación:

a) ¿Cuántos componentes tiene?



Solo tiene EcoSafariGUI

b) ¿Cuántos métodos públicos propios (no heredados) ofrece?

-Ofrece 4 metodos publicos:

-public EcoSafari gettheEcoSafari()

-public static void maini(String[] args)

-public PhotoEcoSafari (EcoSafariGUI gui)

-public void paintComponent(Graphics g)

EcoSafariGUI

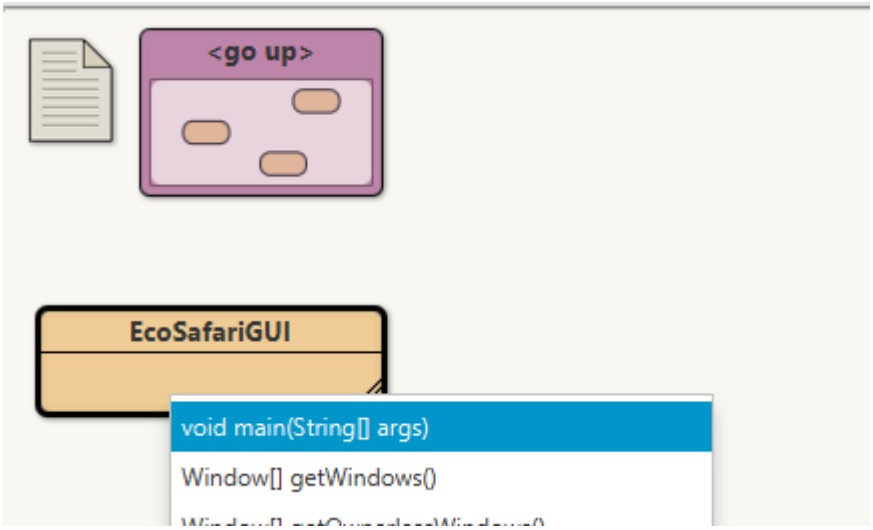
Compile | Undo | Cut | Copy | Paste | Find... | Close

```
46      }
47
48      public EcoSafari gettheEcoSafari(){
49          return theEcoSafari;
50      }
51
52      public static void main(String[] args) {
53          EcoSafariGUI cg=new EcoSafariGUI();
54          cg.setVisible(true);
55      }
56
57
58      class PhotoEcoSafari extends JPanel{
59
60          private EcoSafariGUI gui;
61
62          public PhotoEcoSafari(EcoSafariGUI gui) {
63              this.gui=gui;
64              setBackground(Color.white);
65              setPreferredSize(new Dimension(gui.SIDE*gui.SIZE+10, gui.SIDE*gui.SIZE+10));
66          }
67
68
69          public void paintComponent(Graphics g){
70              EcoSafari theEcoSafari=gui.gettheEcoSafari();
71              super.paintComponent(g);
72
73              for (int c=0;c<=theEcoSafari.getSize();c++){
74                  g.drawLine(c*gui.SIDE,0,c*gui.SIDE,theEcoSafari.getSize()*gui.SIDE);
75              }
76              for (int f=0;f<=theEcoSafari.getSize();f++){
77                  g.drawLine(0,f*gui.SIDE,theEcoSafari.getSize()*gui.SIDE,f*gui.SIDE);
78              }
79              for (int f=0;f<theEcoSafari.getSize();f++){
80                  for(int c=0;c<theEcoSafari.getSize();c++){
81                      if (theEcoSafari.get(f,c)!=null){
82                          g.setColor(theEcoSafari.get(f,c).getColor());
```

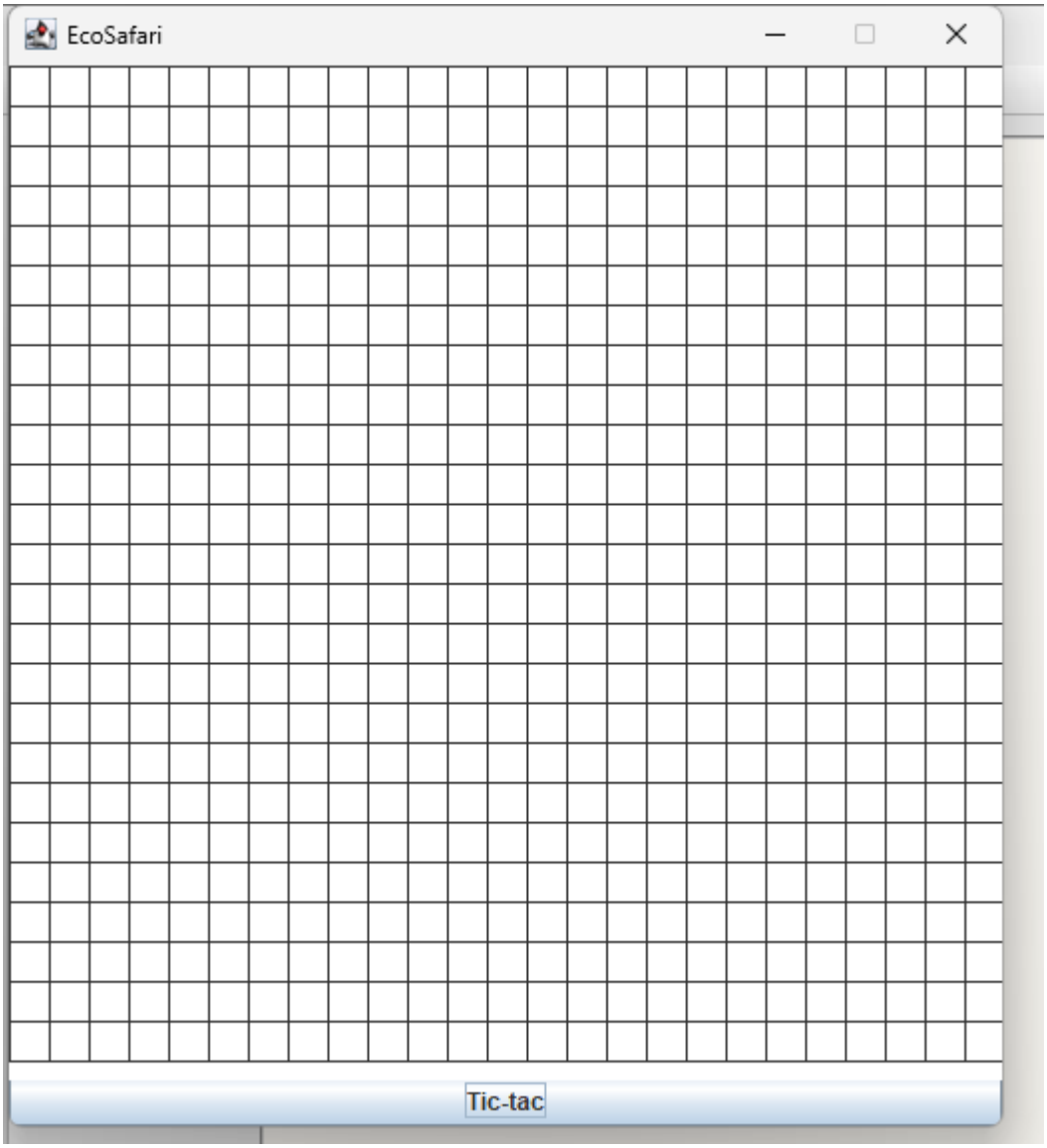
4. Para ejecutar un programa en java,

   a) ¿Qué método se debe ejecutar?

      Para ejecutar un programa, en el paquete presentation se debe llamar al metodo estatico main:

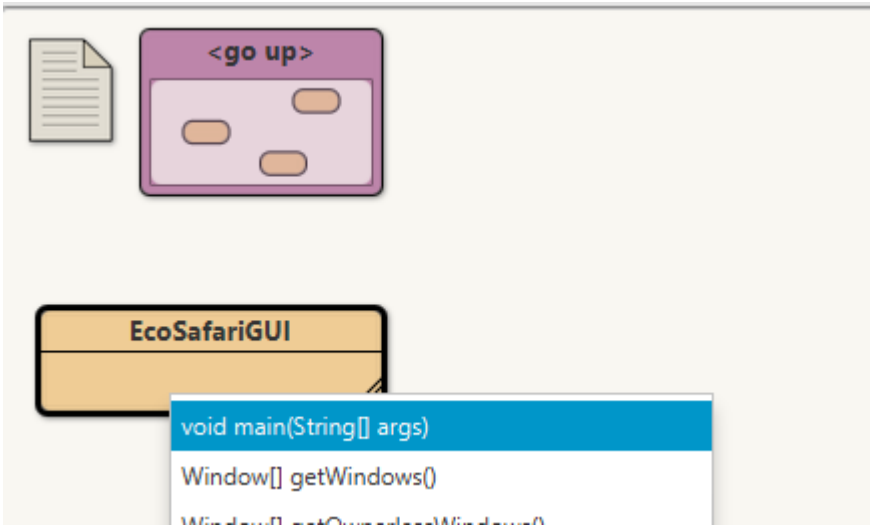






b) ¿En qué clase se encuentra?

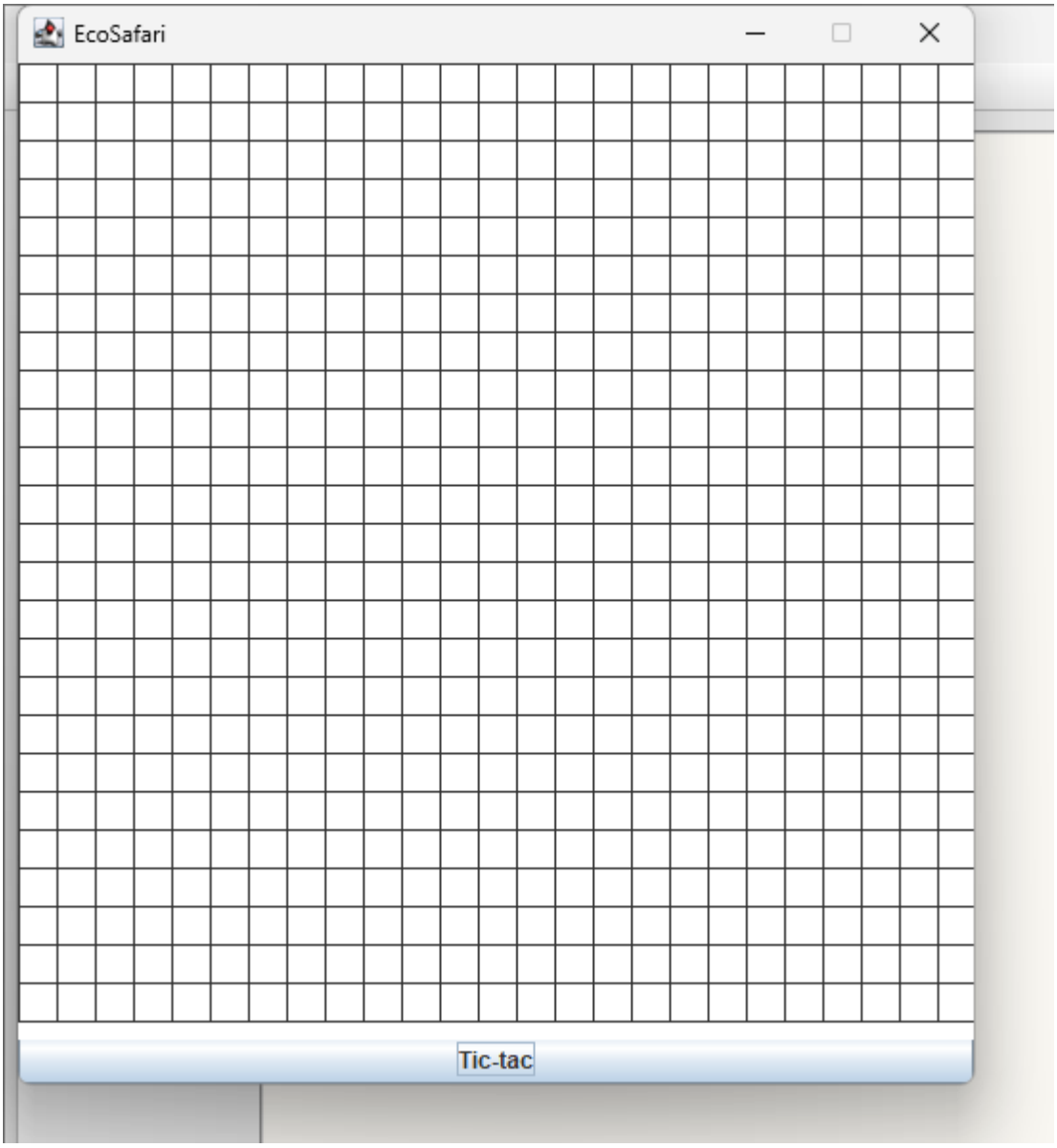
Se encuentra en la clase EcoSafariGUI



5. Ejecuten el programa.

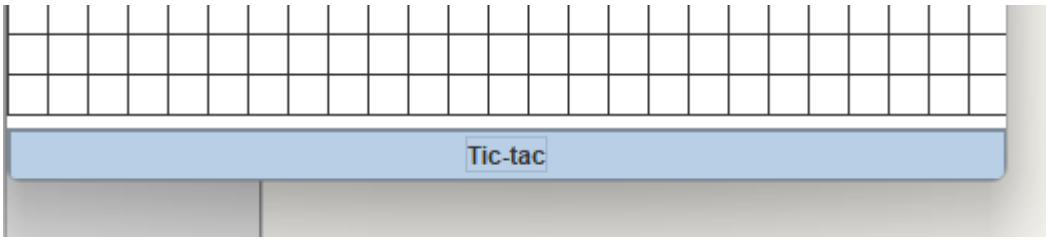
   a) ¿Qué funcionalidades ofrece?

      Ofrece el botón (funcionalidad) de Tic-tac



b) ¿Qué hace actualmente?

No hace nada



c) ¿Por qué?

Cada vez que uno oprime al botón Tic-tac en la ventana, uno llama al metodo ticTacButtonAction() en EcoSafariGUI

```java
private void prepareActions(){
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    ticTacButton.addActionListener(e-> ticTacButtonAction());
}

private void ticTacButtonAction() {
    theEcoSafari.ticTac();
    photo.repaint();
}
```

En ese método, se llama al método ticTac() de la clase EcoSafari, pero dicho método, aún no está implementado, por lo tanto, no hace nada.

```java
/**
 * Advances the simulation by one time step
 */
//First, all entities execute their tic() action
//Then, all entities execute their tac() actions
public void ticTac(){
}
```

B. Realizando ingeniería inversa. Arquitectura General

1. Consulten el significado de las palabras package e import de java.

   Package:

   Import:

   (a) ¿Qué es un paquete?

   -Contenedor que agrupa clases, interfaces y otros paquetes relacionados entre sí.

   (b) ¿Para qué sirve?

   -Sirve para agrupar clases relacionadas y organizadas en un proyecto.

   (c) ¿Para qué se importa?

   -Permite reutilizar herramientas ya programadas por Java o por uno mismo

   (d) Expliquen su uso en este programa.

```
package presentation;
import domain.*;

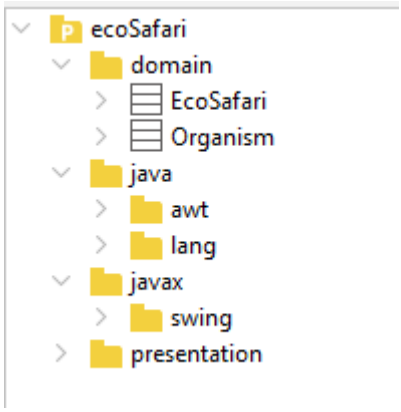
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
```

En este programa, se está importando el paquete domain que ya está creado, esta importación sirve para que la clase EcoSafariGUI utilice todas las clases de este paquete, luego también se usa para importar herramientas ya creadas

2. Revisen el contenido del directorio de trabajo y sus subdirectorios. Describan su contenido:

a) ¿Qué coincide hay entre paquetes y directorios?

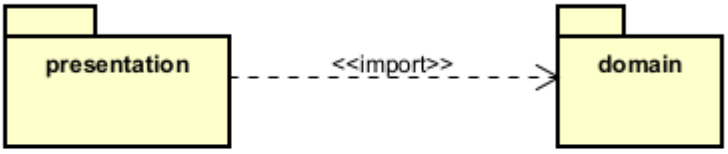
-Lo que coincide es que, un paquete es un directorio.




```
package presentation;
import domain.*;

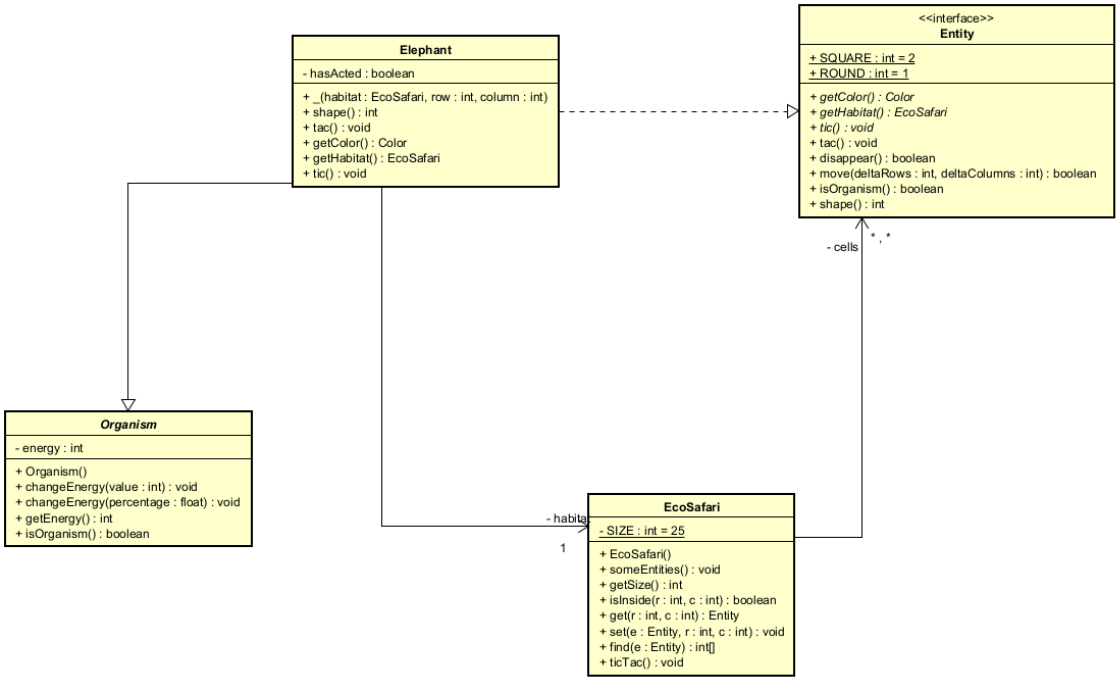
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
```

3. Adicionen al diseño la arquitectura general con un diagrama de paquetes en el que se presente los paquetes y las relaciones entre ellos. Consulte la referencia en Moodle. En astah, crear un diagrama de clases (cambiar el nombre por Package Diagram0)



C. Relizando ingeniería inversa. Arquitectura detallada

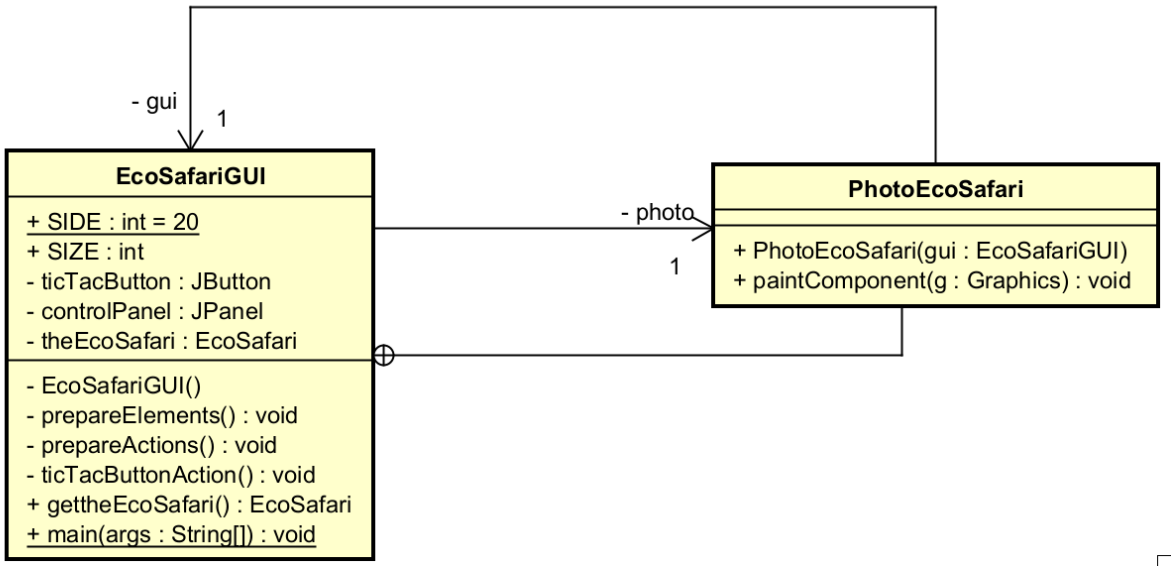
1. Para preparar el proyecto para MDD. Realicen el diseño detallado del paquete de dominio.



a) ¿Qué componentes hacían falta?

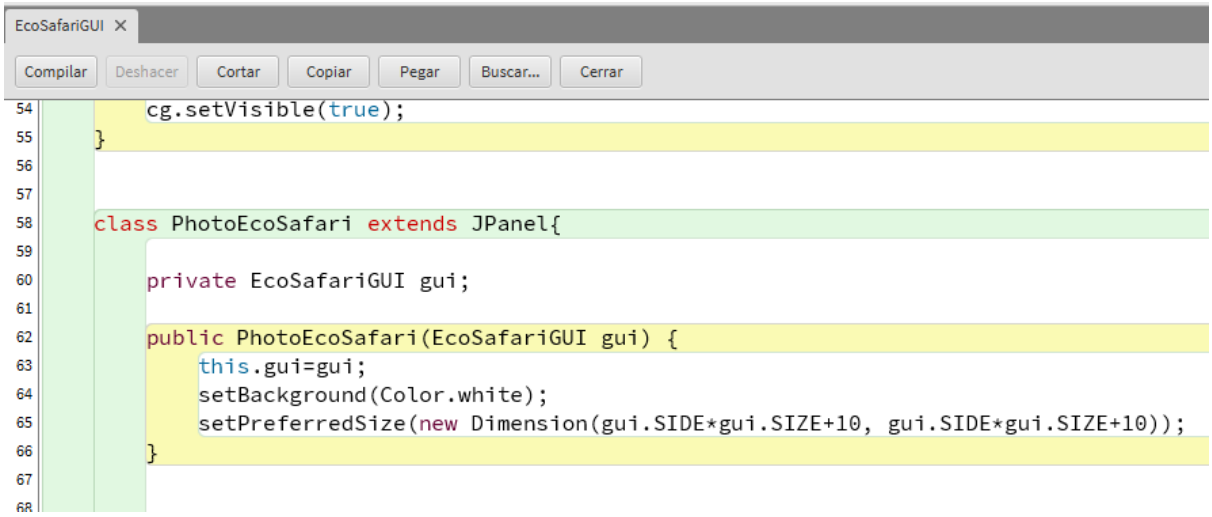
Hacían falta los componentes de Elephant y Entity.

2. Realicen el diseño detallado del paquete presentación.



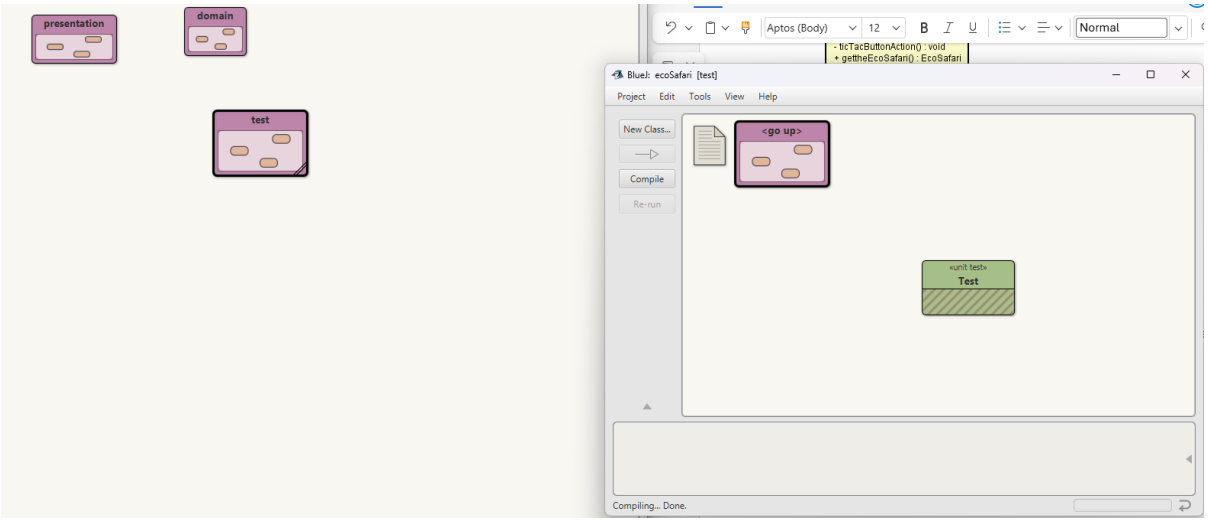
a) ¿Por qué hay dos clases y un archivo .java?

Porque dentro de EcoSafariGUI es donde se encuentra la otra clase, la de PhotoEcoSafari, por tanto no van a haber dos archivos .java.



Como se ve en la foto, estamos dentro de EcoSafariGUI y dentro de él, está la clase de PhotoEcoSafari

3. Adicione la clase de pruebas necesaria para BDD en un paquete independiente de test (No lo adicione al diagrama de clases)



a) ¿Qué clase debe importar?

Debe importar el EcoSafari.

```
package test;
import domain.EcoSafari;

import org.junit.Test;
import org.junit.After;
import org.junit.Before;
import static org.junit.Assert.*;
```

b) ¿Por qué?

Porque en EcoSafari está toda la lógica que queremos probar con los tests.

PARTE II. Desarrollando el proyecto

Para desarrollar esta aplicación vamos a consideras algunos ciclos.

A. Ciclo 1. Iniciando con elefantes

1. Estudie la clase EcoSafari.
   a) ¿Qué tipo de colección usa para albergar cosas?

      Usa una matriz compuesta por Entity's.

   b) ¿Puede tener elefantes?

      Si.

c) ¿Por qué?

Porque los elefantes implementan Entity

2. Estudie el código asociado a la clase Elephant:

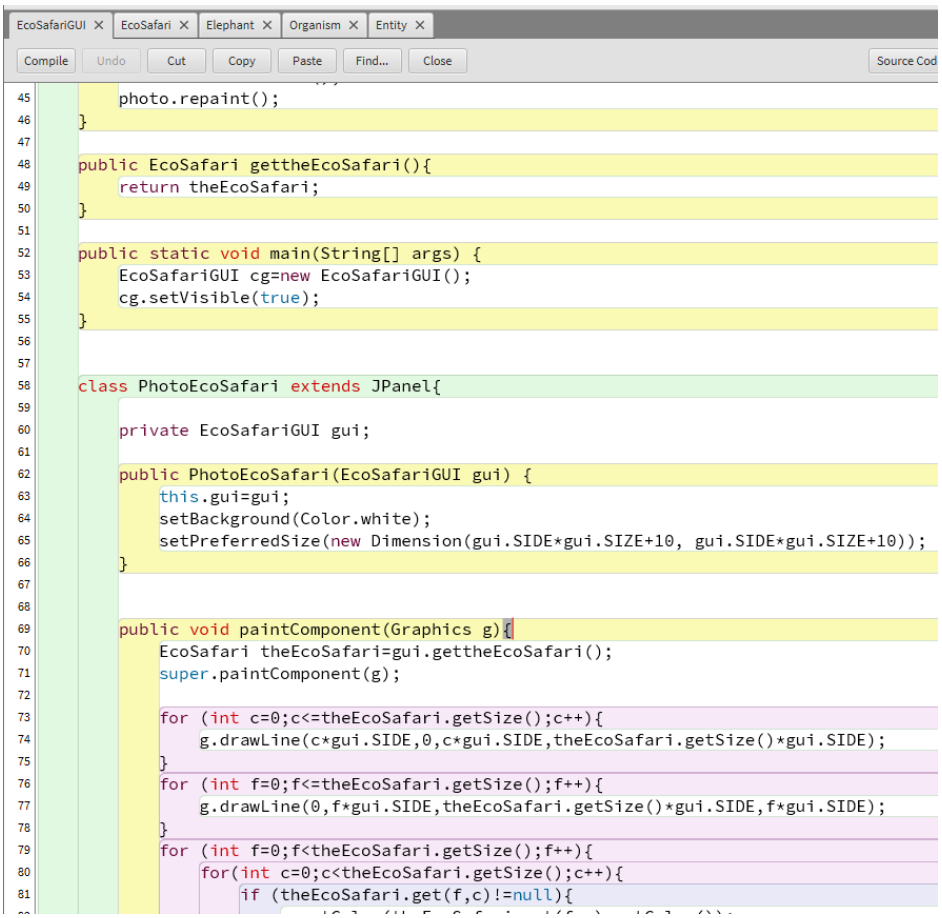
a) ¿De qué color es?

Depende de la energía que tenga el elefante, porque puede ser DARK_GRAY o puede ser LIGHT_GRAY

```
public final Color getColor(){
    return(getEnergy()>=80? Color.DARK_GRAY: Color.LIGHT_GRAY);
}
```

b) ¿Qué forma usa para pintarse?

Se pintan a través de PhotoEcoSafari usando el método de paintComponent().




```
45      photo.repaint();
46  }
47
48  public EcoSafari gettheEcoSafari(){
49      return theEcoSafari;
50  }
51
52  public static void main(String[] args) {
53      EcoSafariGUI cg=new EcoSafariGUI();
54      cg.setVisible(true);
55  }
56
57
58  class PhotoEcoSafari extends JPanel{
59
60      private EcoSafariGUI gui;
61
62      public PhotoEcoSafari(EcoSafariGUI gui) {
63          this.gui=gui;
64          setBackground(Color.white);
65          setPreferredSize(new Dimension(gui.SIDE*gui.SIZE+10, gui.SIDE*gui.SIZE+10));
66      }
67
68
69      public void paintComponent(Graphics g){
70          EcoSafari theEcoSafari=gui.gettheEcoSafari();
71          super.paintComponent(g);
72
73          for (int c=0;c<=theEcoSafari.getSize();c++){
74              g.drawLine(c*gui.SIDE,0,c*gui.SIDE,theEcoSafari.getSize()*gui.SIDE);
75          }
76          for (int f=0;f<=theEcoSafari.getSize();f++){
77              g.drawLine(0,f*gui.SIDE,theEcoSafari.getSize()*gui.SIDE,f*gui.SIDE);
78          }
79          for (int f=0;f<theEcoSafari.getSize();f++){
80              for(int c=0;c<theEcoSafari.getSize();c++){
81                  if (theEcoSafari.get(f,c)!=null){
```

c) ¿Cómo se mueve?

Se mueve por el método que está en Entity

```
public default  boolean move(int deltaRows, int deltaColumns){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.isInside(r+deltaRows,c+deltaColumns)){
            habitat.set(null,r,c);
            habitat.set(this,r+deltaRows,c+deltaColumns);
            ok=true;
        }
    }
    return ok;
}
```

d) ¿Qué componentes definen la clase Elephant? Justifique sus respuestas

Está compuesto por el Organism que es una clase abstracta y está heredando de ella, además de que está compuesto por Entity que es una interfaz. Esto se ve un poco más claro cuando lees el código y te das cuenta que extiende una e implementa otra:

```
6 public class Elephant extends Organism implements Entity{
```

3. Elephant por ser un Organism.

a) ¿Qué atributos tiene?

Tiene energy.

```
public abstract class Organism{

    private int energy; //Inv: 0<=energy<=100
```

b) ¿Qué puede hacer (métodos)?

Puede construirse a si mismo con energía de 100:

```
/**Create a new Organism
 */
public Organism(){
    energy=100;
}
```

Puede cambiar su energía estableciendo un valor específico:

```
/**Changes the organism's energy level by a given value.
  * @param value
  */
 public final void changeEnergy(int value){
     energy+=value;
     energy= (energy<0? 0 : (energy>100? 100 : energy));
 }
```

Puede cambiar su energía estableciendo un porcentaje especifico.

```
/**Changes the organism's energy level by a given percentage.
 * @param percentage
 * @return
 */
public final void changeEnergy(float percentage){
    changeEnergy((int)Math.ceil(energy*percentage));
}
```

Puedes obtener su energía:

```
 /**Returns the energy
 *@return
 */
public final int getEnergy(){
    return energy;
}
```

Puedes verificar si es un organismo:

```
/**Returns that it is an organism
 * @return
 */
public final boolean isOrganism(){
    return true;
}
```

c) ¿Qué decide hacer distinto (sobre-escritura)?

Elephant hace diferente el constructor. Puesto que en Organism está:

```java
/**Create a new Organism
 */
public Organism(){
    energy=100;
}
```

Mientras que en Elephant está:

```java
public Elephant(EcoSafari habitat,int row, int column){
    this.habitat=habitat;
    habitat.set((Entity)this, row, column);
    hasActed=false;
}
```

Entonces hace lo de organism de dejar su energía en 100, pero también hace otras cosas.

d) ¿Qué no puede hacer distinto (métodos finales)?

No puede hacer distinto, los métodos de changeEnergy:

```java
/**Changes the organism's energy level by a given value.
  * @param value
  */
 public final void changeEnergy(int value){
     energy+=value;
     energy= (energy<0? 0 : (energy>100? 100 : energy));
 }

 /**Changes the organism's energy level by a given percentage.
  * @param percentage
  * @return
  */
 public final void changeEnergy(float percentage){
     changeEnergy((int)Math.ceil(energy*percentage));
 }
```

Ni getEnergy ni isOrganism:

```java
    /**Returns the energy
     *@return
     */
    public final int getEnergy(){
        return energy;
    }

    /**Returns that it is an organism
     * @return
     */
    public final boolean isOrganism(){
        return true;
    }
}
```

e) ¿Qué debe aprender a hacer (métodos abstractos)? Justifique sus respuestas.

En este caso los métodos que la clase Organism tiene, no hay ninguno abstracto, pese a que la clase sea abstracta, no necesariamente tiene que tener algunos abstracto, además, buena parte de esos métodos son finales, y el compilador no deja que sean abstractos y finales al tiempo, porque tiene mucho sentido, no puede implementar algo que ya está en su estado final. Por tanto, elefante no tiene que aprender alguno.

4. Por comportarse como un Entity,

a) ¿Qué debe hacer?

Debe poder hacer:

Tic()

Tac()

Shape()

getColor()

isOrganism()

getHabitat()

disappear()

move()

```
public void tic();

public default void tac(){
}

public default int shape(){
    return SQUARE;
}

public abstract Color getColor();

 public default boolean isOrganism(){
   return false;
 }
```

```
public abstract EcoSafari getHabitat();

public default boolean disappear(){
  boolean ok=false;
  int [] position=this.getHabitat().find(this);
     if (position!=null){
         getHabitat().set(null,position[0],position[1]);
         ok=true;
     }
     return ok;
 }

 public default  boolean move(int deltaRows, int deltaColumns){
     int [] position=getHabitat().find(this);
     EcoSafari habitat=getHabitat();
     boolean ok=false;
     if (position!=null){
         int r = position[0];
         int c = position[1];
         if (habitat.isInside(r+deltaRows,c+deltaColumns)){
             habitat.set(null,r,c);
             habitat.set(this,r+deltaRows,c+deltaColumns);
             ok=true;
         }
     }
     return ok;
 }
```

b) ¿Qué decide hacer distinto?

Los override que hace elephant son:

getHabitat()

getColor()

shape()

tic()

tac()

```java
@Override
public EcoSafari getHabitat(){
    return habitat;
}

@Override
public final Color getColor(){
    return(getEnergy()>=80? Color.DARK_GRAY: Color.LIGHT_GRAY);
}

@Override
public final int shape(){
    return Entity.ROUND;
}
```

```java
@Override
public void tic(){
    if ((! hasActed) && (move(1, 1))) {
        changeEnergy(-10);
        if (getEnergy()==0){
            disappear();
        }
    }
    hasActed=true;
}

@Override
public void tac(){
    hasActed=false;
}
```

c) ¿Qué no puede hacer distinto?

No tiene ningún método final que no pueda hacer distinto.

d) ¿Qué debe aprender a hacer? Justifique sus respuestas

Los métodos abstractos que dene aprender a hacer son tic(), getColor(), getHabitat():

```
public void tic();

public abstract Color getColor();

public abstract EcoSafari getHabitat();
```

5. De acuerdo a lo anterior un Elephant

   a) ¿Cómo actúa?

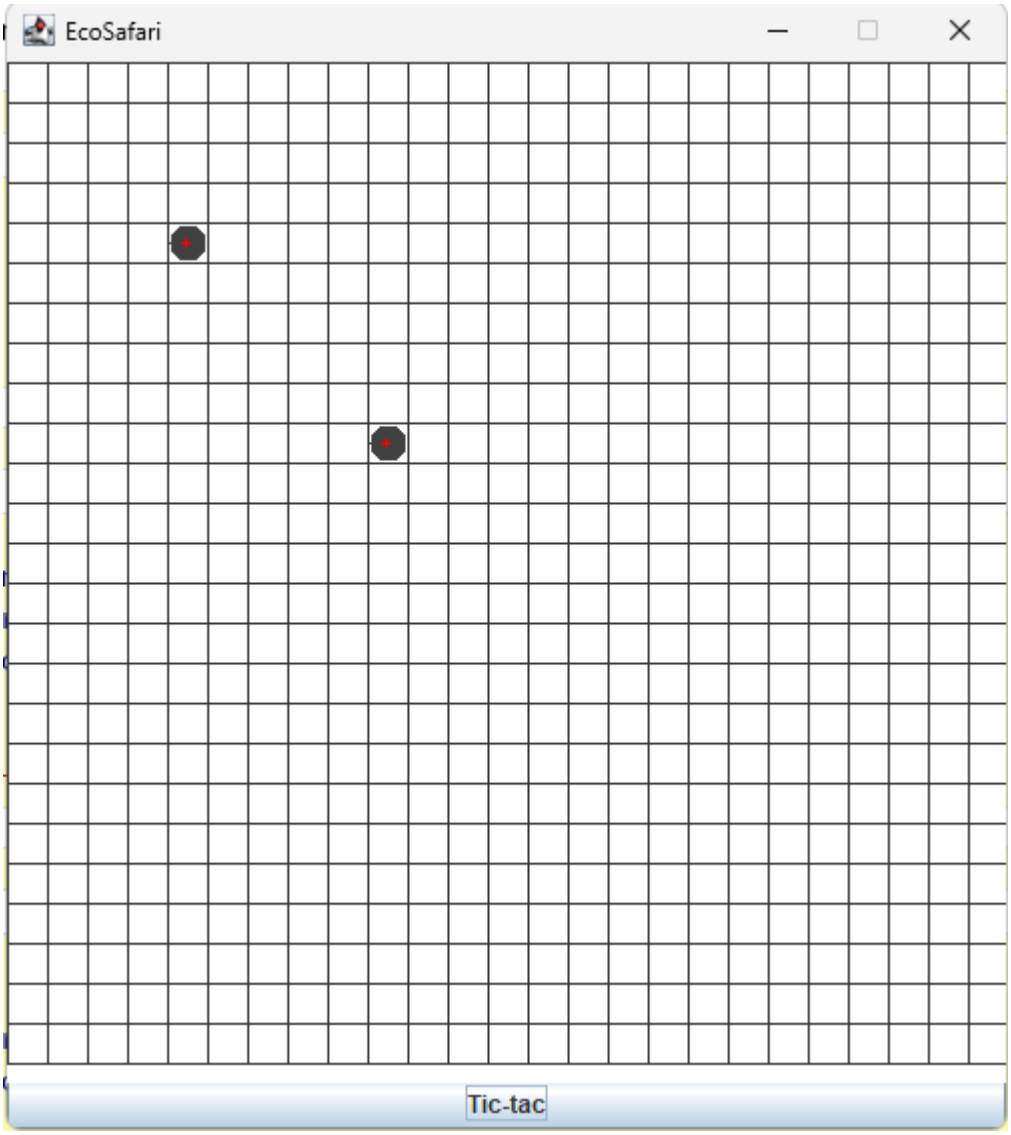
   El Elephant es un Organism, el cual tiene como tope de energía 100. Por ser un organismo, puede cambiar su energía, se puede saber su energía. Además, por implementar a Entity, reacciona ante el tic y tac, además de que se puede obtener su color, se puede saber su hábitat, este puede desaparecer y moverse. Por último, al ser un Elephant, sabe que su figura será un circulo, sabe como actuar específicamente ante el tic y tac, sabe su hábitat y si ha actuado, además de que su color irá variando, dependiendo de la energía que tenga.

6. Ahora vamos a crear dos elefantes en diferentes posiciones (5,5), (10,10), llámelos dumbo y babar, usando el método someEntities. Ejecutan el programa,

```
/**
 * Pupulates the EcoSafari with some entities
 */
public void someEntities(){
    //We create dumbo and babar:
    //Dumbo will be in position (5,5), but in a matrix it represents [4][4]
    Elephant dumbo = new Elephant(this, 4,4);

    //Babar will be in position (10,10) but in a matrix it represents [9][9]
    Elephant babar = new Elephant(this, 9,9);

}
```



a) ¿Qué pasa con los elefantes?

Son círculos grises oscuro que tienen un + en la mitad de ellos.

b) ¿Por qué?

Son de color gris oscuro, porque al iniciar su creación tienen 100 de energy, por tanto cuando se pide el color de estos, se ejecuta el siguiente método dentro de elephant:

```
@Override
public final Color getColor(){
    return(getEnergy()>=80? Color.DARK_GRAY: Color.LIGHT_GRAY);
}
```

Y pues como su energía es mayor o igual a 80, dice que el color que debe devolver es DARK_GRAY.

Por otro lado tiene un “+” encima porque, en EcoSafariGUI cuando va a pintar en PhotoEcoSafari, en el método de paintComponent:

```
}
if (theEcoSafari.get(f,c).isOrganism()){
    g.setColor(Color.red);
    if (((Organism)theEcoSafari.get(f,c)).getEnergy()>=50){
        g.drawString("+",gui.SIDE*c+6,gui.SIDE*f+15);
    } else {
        g.drawString("~",gui.SIDE*c+6,gui.SIDE*f+17);
    }
}
```

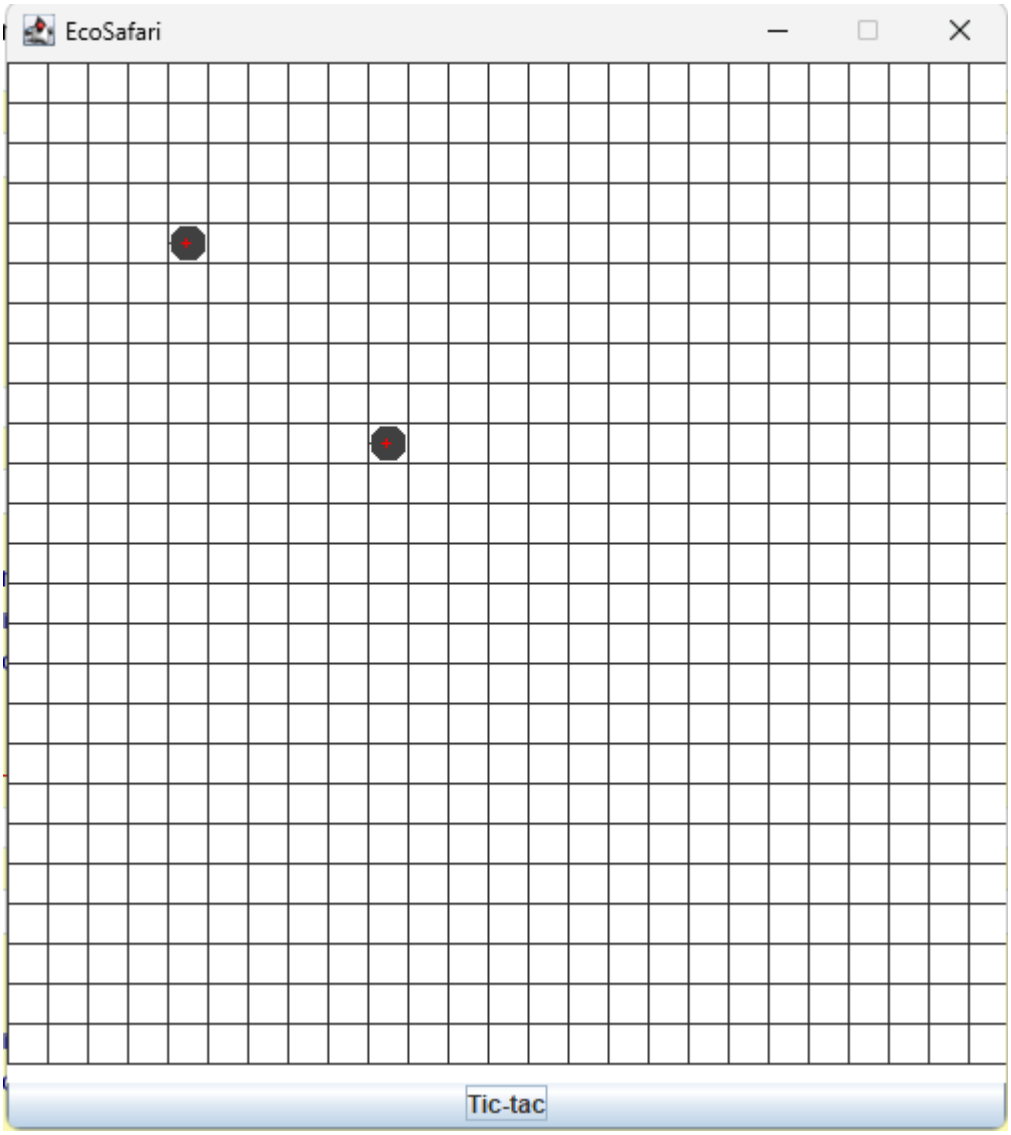
Se dice que si su energía es mayor o igual a 50 tendrá un símbolo de +. Además el color del signo se define por ese g.setColor(Color.red), por eso es rojo

c) Capturen una pantalla significativa.

```
/**
 * Pupulates the EcoSafari with some entities
 */
public void someEntities(){
    //We create dumbo and babar:
    //Dumbo will be in position (5,5), but in a matrix it represents [4][4]
    Elephant dumbo = new Elephant(this, 4,4);

    //Babar will be in position (10,10) but in a matrix it represents [9][9]
    Elephant babar = new Elephant(this, 9,9);

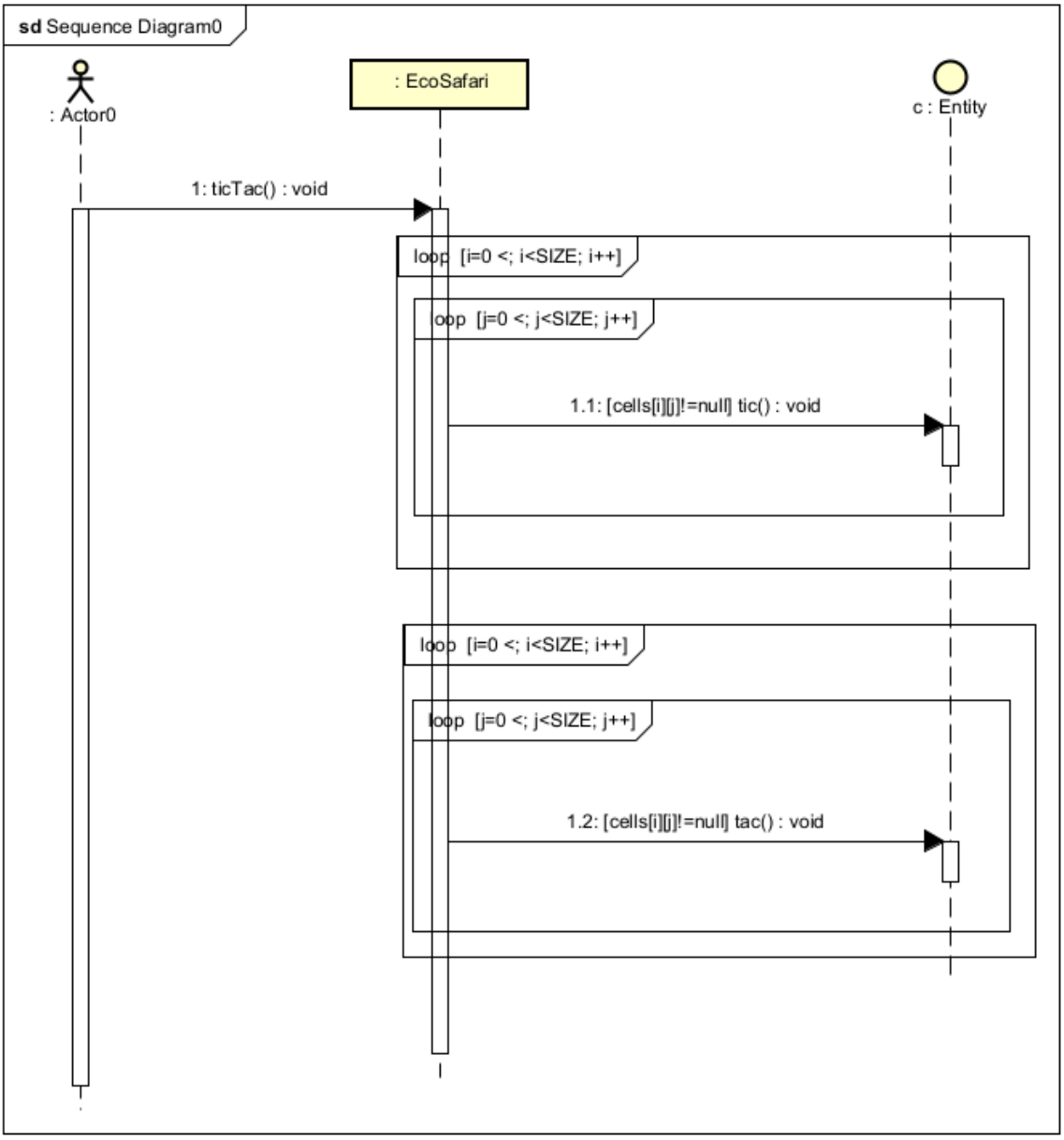
}
```




```
@Override
public final Color getColor(){
    return(getEnergy()>=80? Color.DARK_GRAY: Color.LIGHT_GRAY);
}
```

```
    }
    if (theEcoSafari.get(f,c).isOrganism()){
        g.setColor(Color.red);
        if (((Organism)theEcoSafari.get(f,c)).getEnergy()>=50){
            g.drawString("+",gui.SIDE*c+6,gui.SIDE*f+15);
        } else {
            g.drawString("~",gui.SIDE*c+6,gui.SIDE*f+17);
        }
    }
```

7. Diseñen, construyan y prueben el método llamado tictac() de la clase EcoSafari.



```java
/**
 * Advances the simulation by one time step
 */
//First, all entities execute their tic() action
//Then, all entities execute their tac() actions
public void ticTac(){
    for(int i=0; i<SIZE; i++){
        for(int j=0; j<SIZE; j++){
            if(cells[i][j] != null){
                cells[i][j].tic();
            }
        }
    }
    for(int i=0; i<SIZE; i++){
        for(int j=0; j<SIZE; j++){
            if(cells[i][j] != null){
                cells[i][j].tac();
            }
        }
    }
}
```

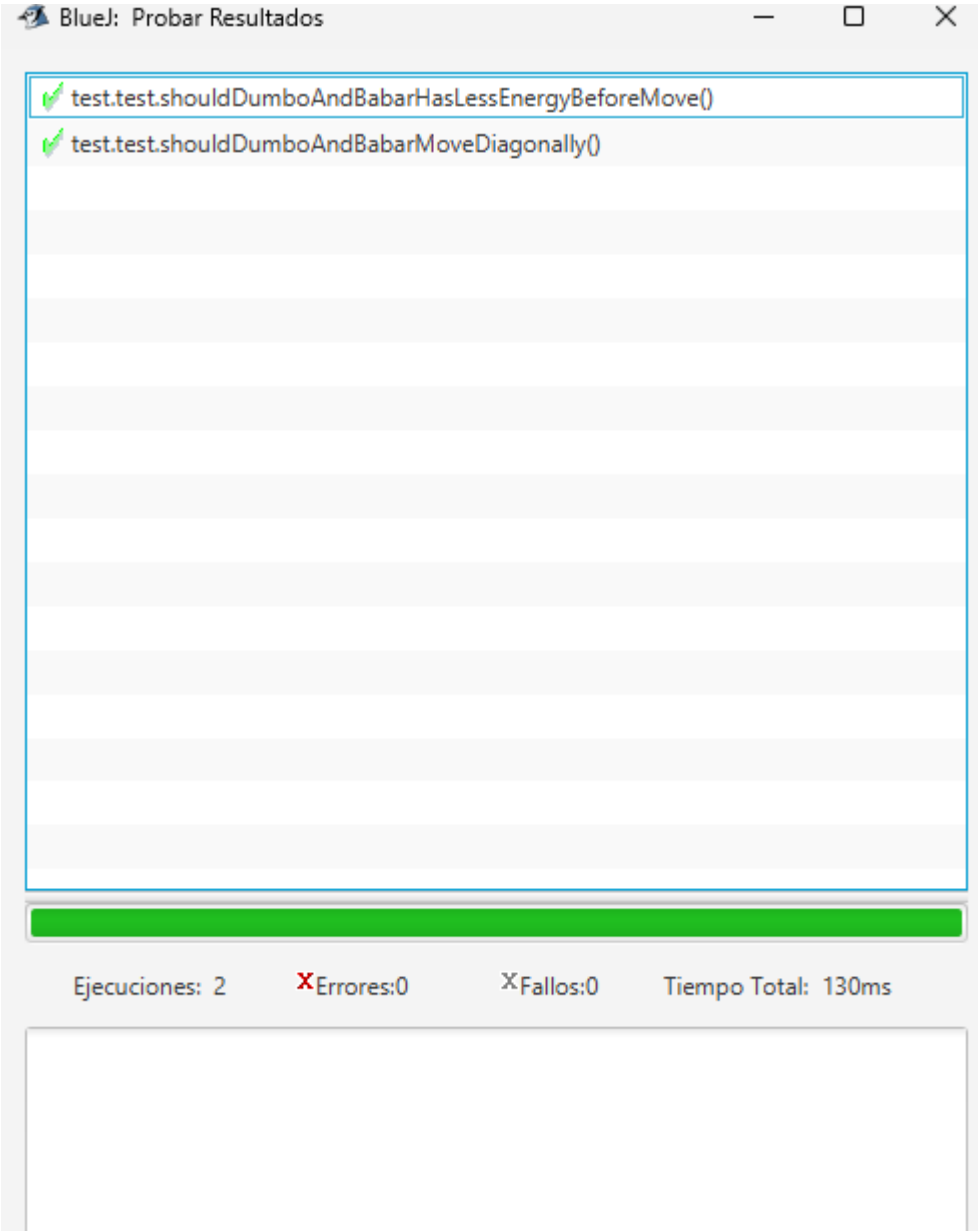
```java
/**
 * Dumbo and babar should change their position and move diagonally.
 */
@Test
public void shouldDumboAndBabarMoveDiagonally(){
    es.ticTac();
    assertTrue(es.get(4,4) == null && es.get(9,9) == null);
    assertTrue(es.get(5,5) != null && es.get(10,10) != null);
}
```

```java
/**
 * Dumbo and babar should have less energy after they move.
 */
@Test
public void shouldDumboAndBabarHasLessEnergyAfterMove(){
    Elephant dumbo = (Elephant)es.get(4,4);
    int startDumboEnergy = dumbo.getEnergy();

    Elephant babar = (Elephant)es.get(9,9);
    int startBabarEnergy = babar.getEnergy();

    es.ticTac();

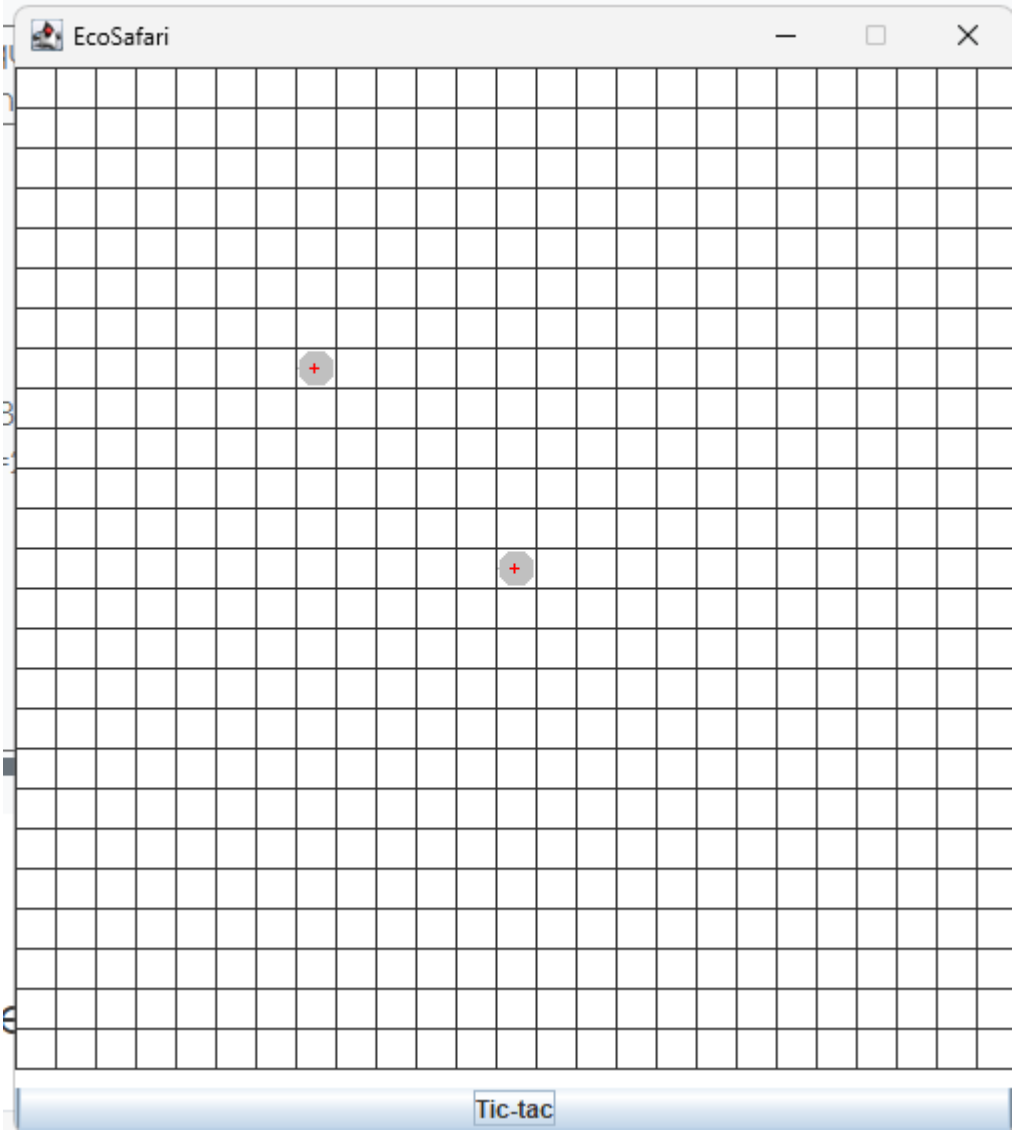
    assertTrue(startDumboEnergy > dumbo.getEnergy());
    assertTrue(startBabarEnergy > babar.getEnergy());
}
```



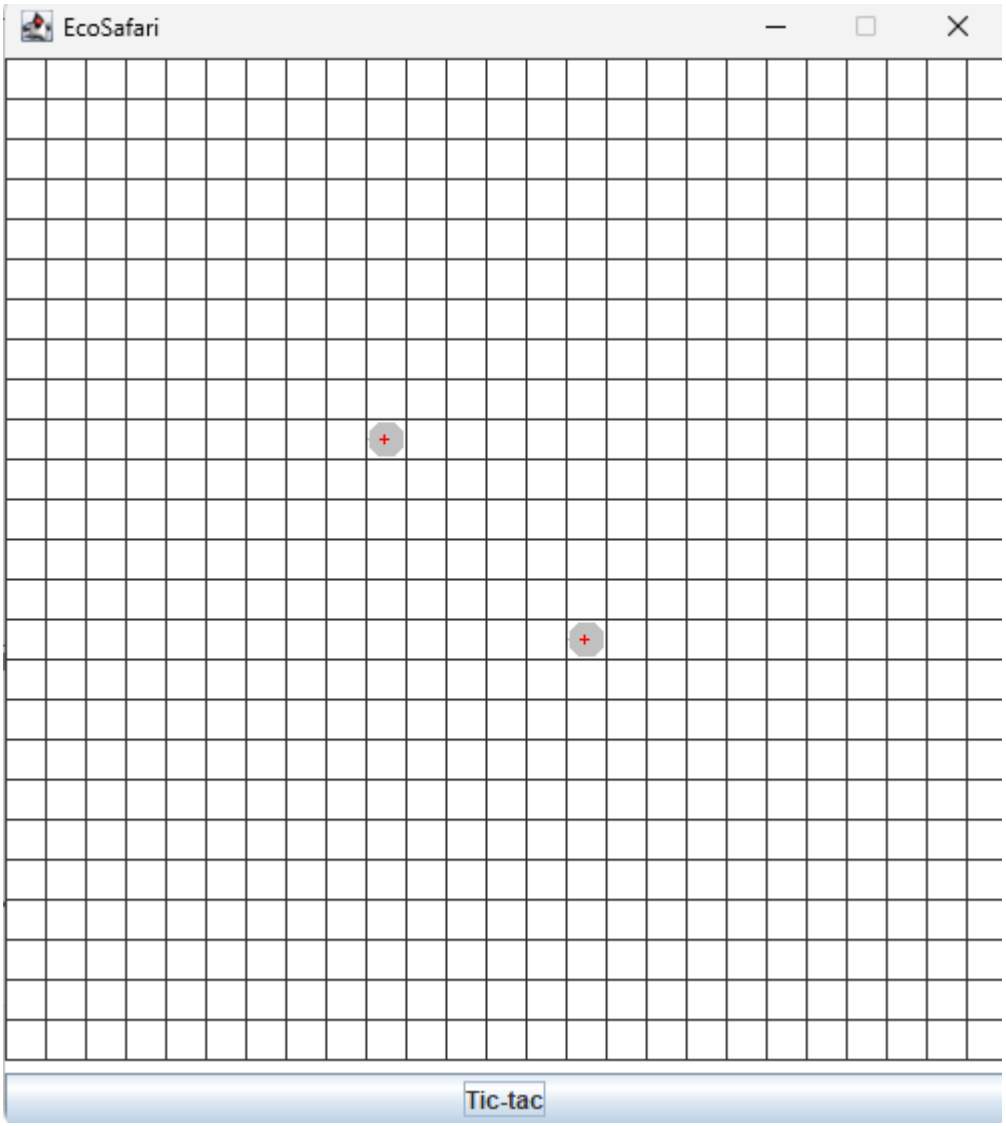
8. a) ¿Cómo quedarían Dumbo y babar después de 3, 5, 10 tic-tac?

   Ejecuten el programa. Capturen pantallas significativas en los momentos correspondientes.

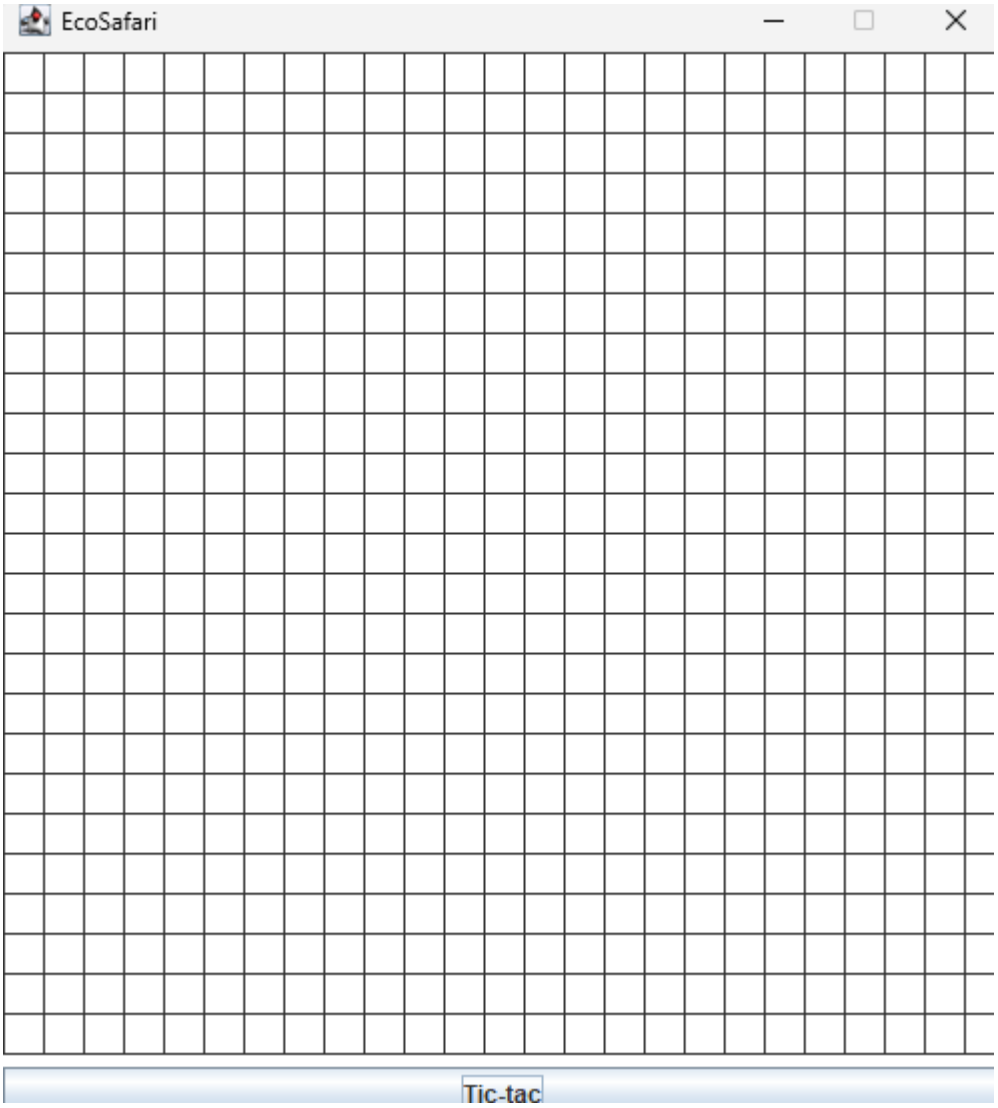
   Después de 3 tic-tac:



Después de 5 tic-tac



Después de 10 tic-tacs



b) ¿Es correcto?

Sí, porque cada ves que hace tic, el Elephant va perdiendo 10 de energía, como lo dice esta parte del código:

```
@Override
public void tic(){
    if ((! hasActed) && (move(1, 1))) {
        changeEnergy(-10);
        if (getEnergy()==0){
            disappear();
        }
    }
    hasActed=true;
}
```

Pero se me hace curiosa, la prueba de los 5 tic, porque intuyo que se quería ver que apareciera la virgulilla (“~”), pero eso solo ocurrirá si tiene menos de 50, no exactamente 50, porque ahí seguirá mostrando el más (“+”), eso se afirma por esta parte del código:

```
if (((Organism)theEcoSafari.get(f,c)).getEnergy()>=50){
    g.drawString("+",gui.SIDE*c+6,gui.SIDE*f+15);
} else {
    g.drawString("~",gui.SIDE*c+6,gui.SIDE*f+17);
}
```

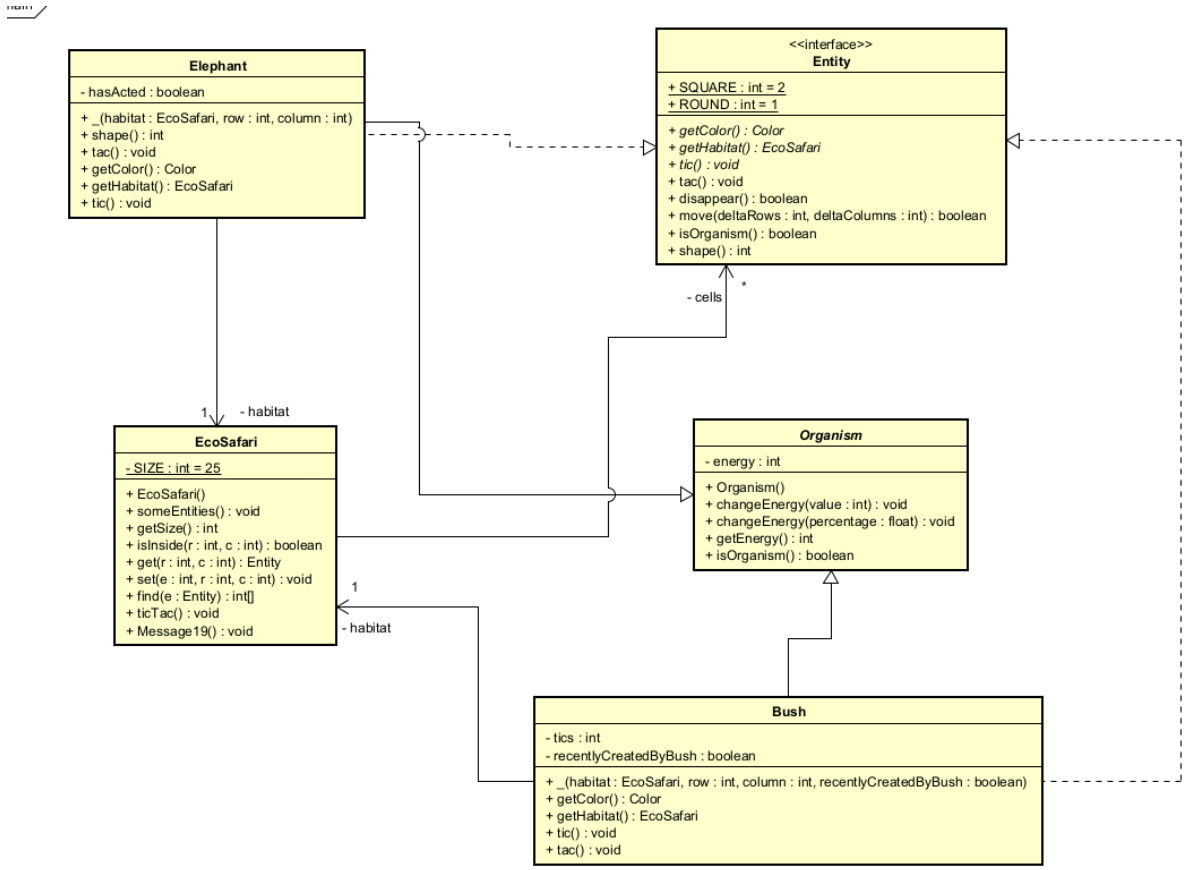
Y pues al final tiene sentido que desaparezca porque tiene 0 de energía, entonces muere.

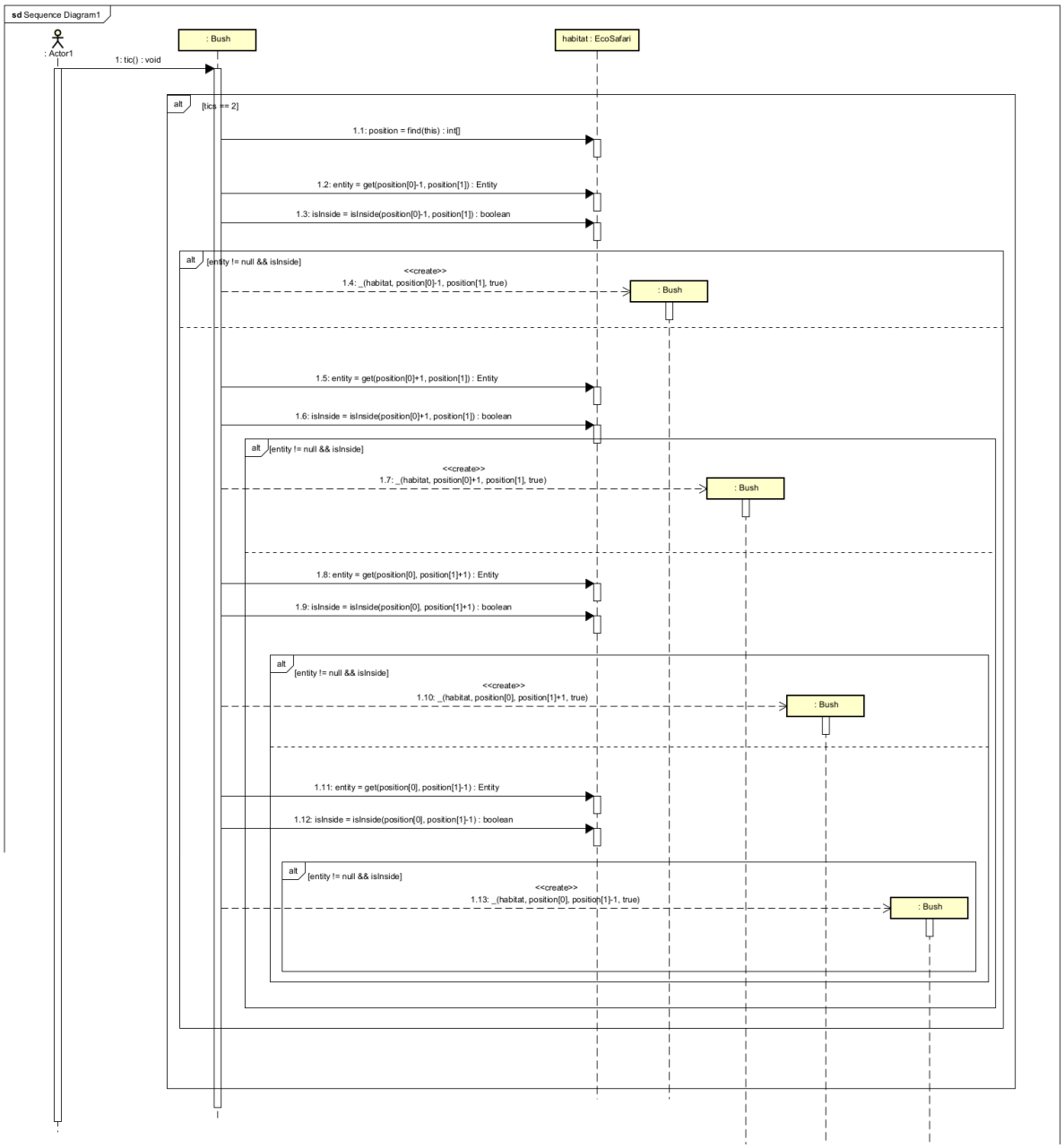
B. Ciclo 2. Incluyendo arbustos.

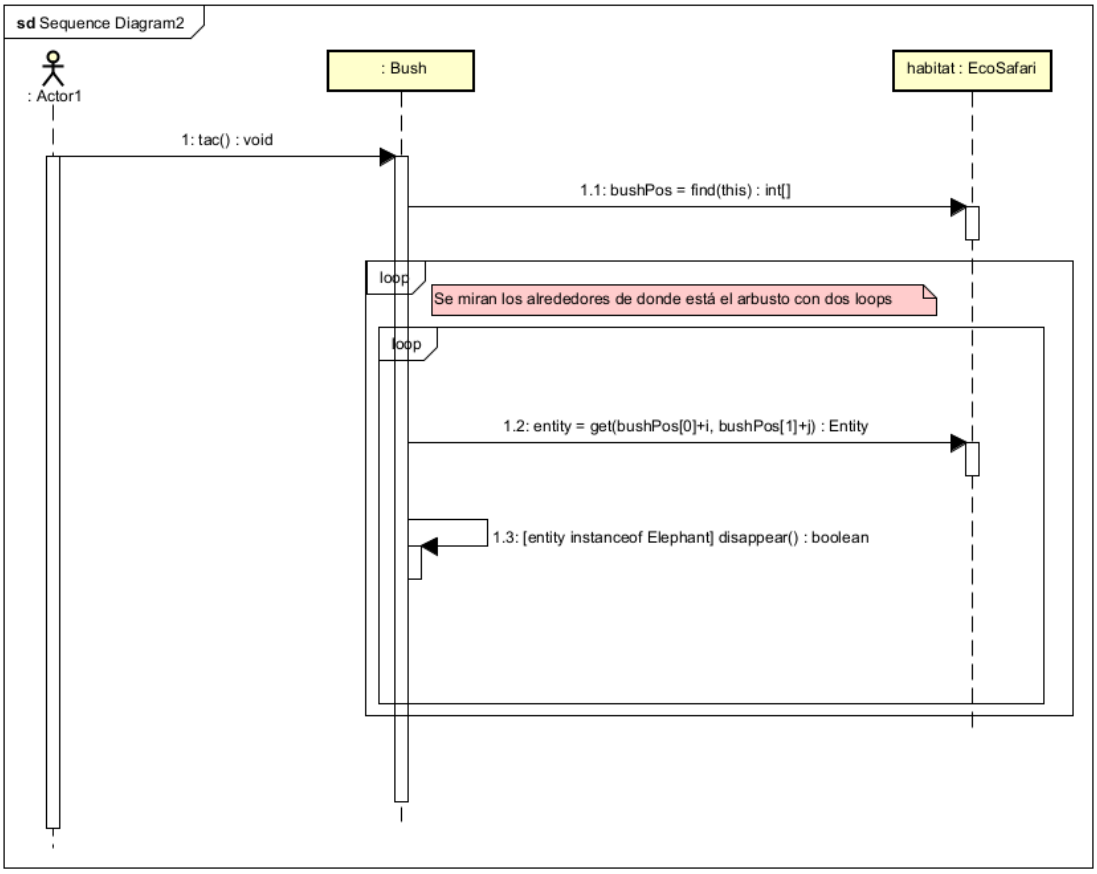
El objetivo de este punto es permitir tener arbustos en el bosque. Ellos (i) son verdes, cuando son jóvenes y amarillos al envejecer (4 tics) (ii) cuando son jóvenes, después de 2 tics retornan si hay espacio vecino vacío con el siguiente orden de prioridad: norte, sur, este y oeste, (iv) si quedan cerca a un elefante desaparecen (el elefante se los come)

1. Para implementar esta nueva clase Organism.
   a) ¿Cuáles métodos se sobre-escriben (overriding)?

      Ninguno, porque no vemos necesario hacer overriding, los métodos que ahí están definidos ya funcionan. Simplemente va a haber una implementación de los métodos de Entity.
2. Diseñen, construyan y prueben esta nueva clase (Mínimo dos pruebas de unidad)


Elephant
- hasActed : boolean
+ _(habitat : EcoSafari, row : int, column : int)
+ shape() : int
+ tac() : void
+ getColor() : Color
+ getHabitat() : EcoSafari
+ tic() : void
<<interface>>
Entity
+ SQUARE : int = 2
+ ROUND : int = 1
+ getColor() : Color
+ getHabitat() : EcoSafari
+ tic() : void
+ tac() : void
+ disappear() : boolean
+ move(deltaRows : int, deltaColumns : int) : boolean
+ isOrganism() : boolean
+ shape() : int
- cells
*
1
- habitat
EcoSafari
- SIZE : int = 25
+ EcoSafari()
+ someEntities() : void
+ getSize() : int
+ isInside(r : int, c : int) : boolean
+ get(r : int, c : int) : Entity
+ set(e : int, r : int, c : int) : void
+ find(e : Entity) : int[]
+ ticTac() : void
+ Message19() : void
Organism
- energy : int
+ Organism()
+ changeEnergy(value : int) : void
+ changeEnergy(percentage : float) : void
+ getEnergy() : int
+ isOrganism() : boolean
1
- habitat
Bush
- tics : int
- recentlyCreatedByBush : boolean
+ _(habitat : EcoSafari, row : int, column : int, recentlyCreatedByBush : boolean)
+ getColor() : Color
+ getHabitat() : EcoSafari
+ tic() : void
+ tac() : void


sd Sequence Diagram1
: Actor1
: Bush
habitat : EcoSafari
1: tic() : void
alt
[tics == 2]
1.1: position = find(this) : int[]
1.2: entity = get(position[0]-1, position[1]) : Entity
1.3: isInside = isInside(position[0]-1, position[1]) : boolean
alt
[entity != null && isInside]
<<create>>
1.4: _(habitat, position[0]-1, position[1], true)
: Bush
1.5: entity = get(position[0]+1, position[1]) : Entity
1.6: isInside = isInside(position[0]+1, position[1]) : boolean
alt
[entity != null && isInside]
<<create>>
1.7: _(habitat, position[0]+1, position[1], true)
: Bush
1.8: entity = get(position[0], position[1]+1) : Entity
1.9: isInside = isInside(position[0], position[1]+1) : boolean
alt
[entity != null && isInside]
<<create>>
1.10: _(habitat, position[0], position[1]+1, true)
: Bush
1.11: entity = get(position[0], position[1]-1) : Entity
1.12: isInside = isInside(position[0], position[1]-1) : boolean
alt
[entity != null && isInside]
<<create>>
1.13: _(habitat, position[0], position[1]-1, true)
: Bush


sd Sequence Diagram2
: Actor1
: Bush
habitat : EcoSafari
1: tac() : void
1.1: bushPos = find(this) : int[]
loop
Se miran los alrededores de donde está el arbusto con dos loops
loop
1.2: entity = get(bushPos[0]+i, bushPos[1]+j) : Entity
1.3: [entity instanceof Elephant] disappear() : boolean

```java
/**
 * Should create correct new bushes at north and south
 */
@Test
public void shouldCreateCorrectNewBushesAtNorthAndSouth(){
    Bush bush1 = new Bush(es,0,0,false); //Should create a new bush at south
    Bush bush2= new Bush(es,24,24,false); //Should create a new bush at north

    es.ticTac();
    es.ticTac();

    assertTrue(es.get(23,24) != null);
    assertTrue(es.get(1,0) != null);
}
```

```java
/**
 * Should create correct new bushes at east and west
 */
@Test
public void shouldCreateCorrectNewBushesAtEastAndWest(){
    Bush bush1 = new Bush(es,0,0,false);
    Bush bush2= new Bush(es,24,24,false);

    es.ticTac();

    Bush bush3 = new Bush(es,1,0,false); //Now bush1 should create a new bush at east.
    Bush bush4 = new Bush(es,23,24,false); //Now bus2 should create a new bush at west.
    es.ticTac();

    assertTrue(es.get(0,1) != null);
    assertTrue(es.get(24,23) != null);
}
```

```java
/**
 * Bush should change its color after four tics.
 */
@Test
public void shouldBushChangeColorAfterFourTics(){
    Bush bush1 = new Bush(es,0,0,false);
    for(int i = 0; i < 4; i++){
        es.ticTac();
    }

    assertEquals(bush1.getColor(), Color.yellow);
}
```

```java
/**
 * Should bush be eaten if there's an elephant near
 */
@Test
public void shouldBushBeEatenIfThereIsAnElephantNear(){
    Bush bush1 = new Bush(es,2,2,false);
    Elephant eater = new Elephant(es,0,0);
    es.ticTac();
    assertTrue(es.get(2,2) == null);
}
```

```java
package domain;

import java.awt.Color;

/**
 * Represents a Bush that is an organism and implements Entity<br>
 * <b>(habitat, tics, recentlyCreatedByBush)</b>
 */
public class Bush extends Organism implements Entity{
    private int tics;
    private boolean recentlyCreatedByBush;
    private EcoSafari habitat;

    /**
     * Constructs a new Bush.
     * @param habitat is Bushes' habitat.
     * @param row is the row position.
     * @param column is the column position.
     */
    public Bush(EcoSafari habitat,int row, int column, boolean recentlyCreatedByBush){
        this.habitat=habitat;
        habitat.set((Entity)this, row, column);
        tics = 0;
        this.recentlyCreatedByBush = recentlyCreatedByBush;
    }

    @Override
    /**
     * Gives the Bushes' habitat.
     * @return Bushes' habitat.
     */
    public EcoSafari getHabitat(){
        return habitat;
```

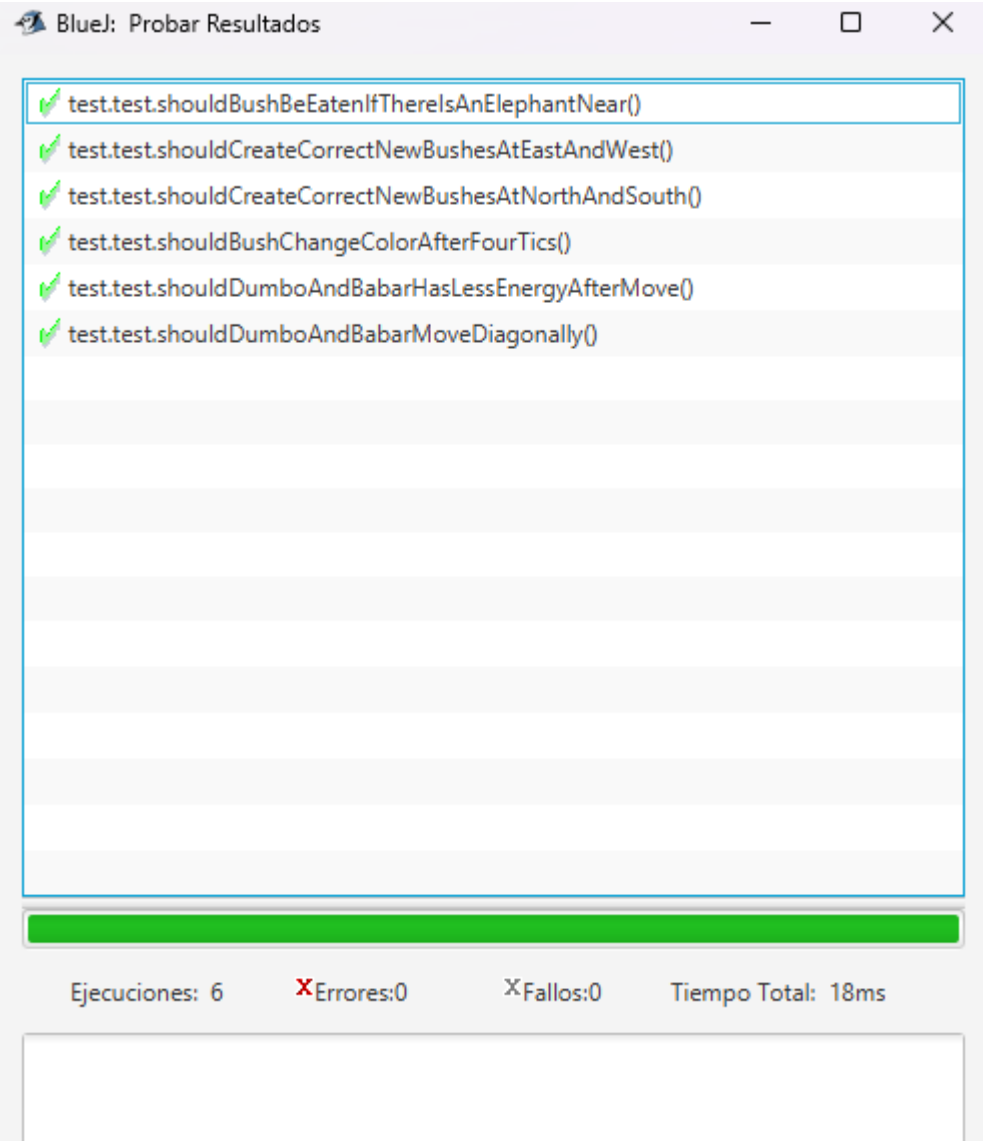
```java
@Override
/**
 * Gives Bushes' color depending its energy.
 * @return Bushes' color.
 */
public final Color getColor(){
    if(tics<4){
        return Color.green;
    }
    else{
        return Color.yellow;
    }
}

@Override
/**
 * Creates a new bush near to it, if a place is empty, first looks the North, then South, then
 * East and finally West. This just happens if there are two tics.
 * If there are four tics it changes it colors.
 */
public void tic(){
    if(!recentlyCreatedByBush){
        tics++;
    }

    if(tics ==2){
        int[] position = habitat.find(this);
        if(habitat.get(position[0]-1, position[1]) == null && habitat.isInside(position[0]-1, position[1])){
            new Bush(habitat, position[0]-1, position[1],true);
        }
        else if(habitat.get(position[0]+1, position[1]) == null && habitat.isInside(position[0]+1, position[1])){
            new Bush(habitat, position[0]+1, position[1],true);
        }
        else if(habitat.get(position[0], position[1]+1) == null && habitat.isInside(position[0], position[1]+1)){
            new Bush(habitat, position[0], position[1]+1, true);
        }
        else if(habitat.get(position[0], position[1]-1) == null && habitat.isInside(position[0], position[1]-1)){
            new Bush(habitat, position[0], position[1]-1,true);
        }
    }
}
```

```java
@Override
/**
 * Looks if there's an elephant near, if it happens then the bush disappears.
 */
public void tac(){
    if(recentlyCreatedByBush){
        recentlyCreatedByBush = false;
    }

    int[] bushPos = habitat.find(this);
    for(int i = -1; i <=1; i++){
        for(int j= -1; j <=1; j++){
            if(i != 0 || j != 0){
                if(habitat.get(bushPos[0]+i, bushPos[1]+j) instanceof Elephant){
                    disappear();
                    break;
                }
            }
        }
    }
}
```



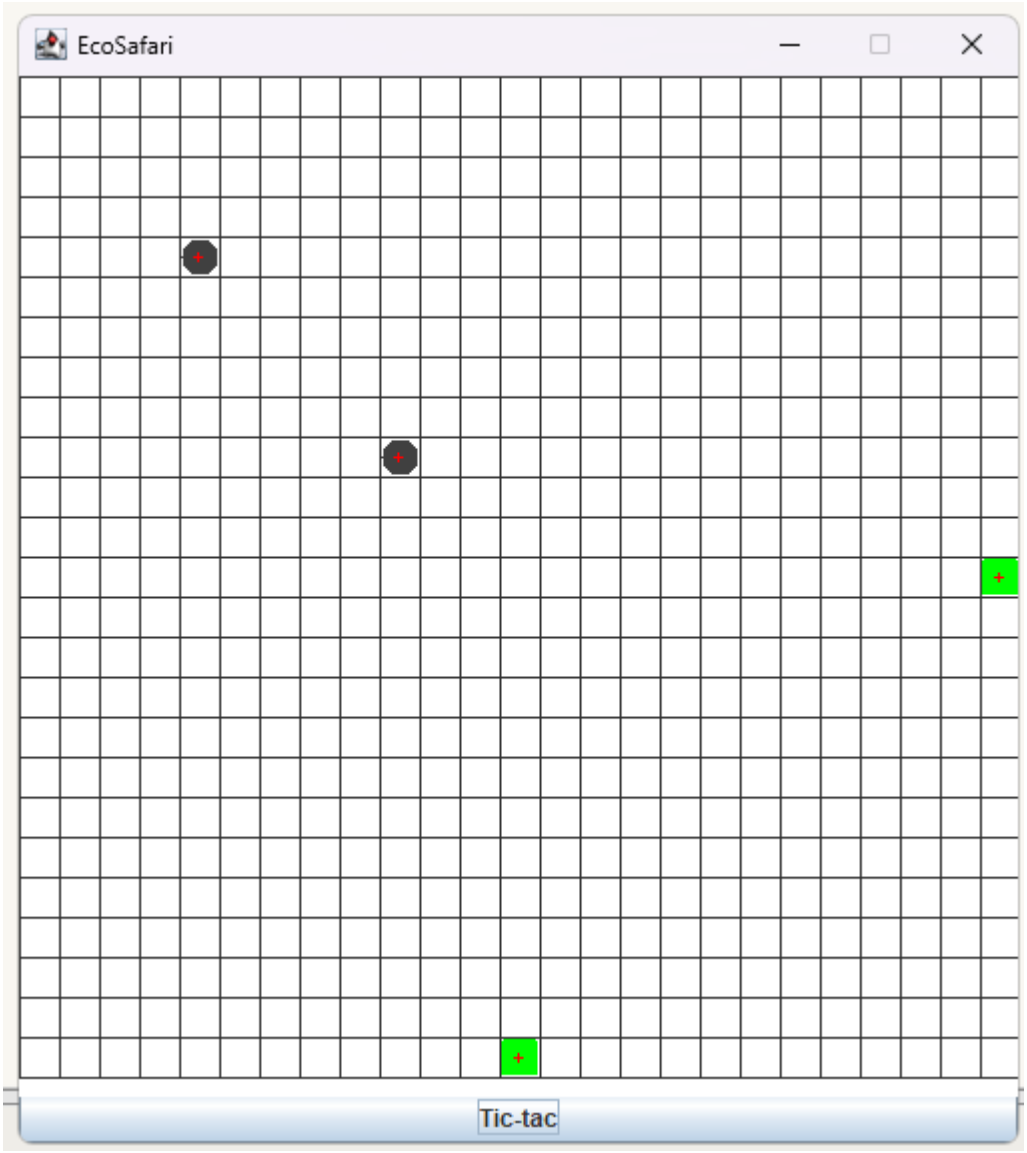
3. Adicione una pareja de arbustos, de nombre mopane y acacia:

```
/**
 * Populates the EcoSafari with some entities
 */
public void someEntities(){
    //We create dumbo and babar:
    //Dumbo will be in position (5,5), but in a matrix it represents [4][4]
    Elephant dumbo = new Elephant(this, 4,4);

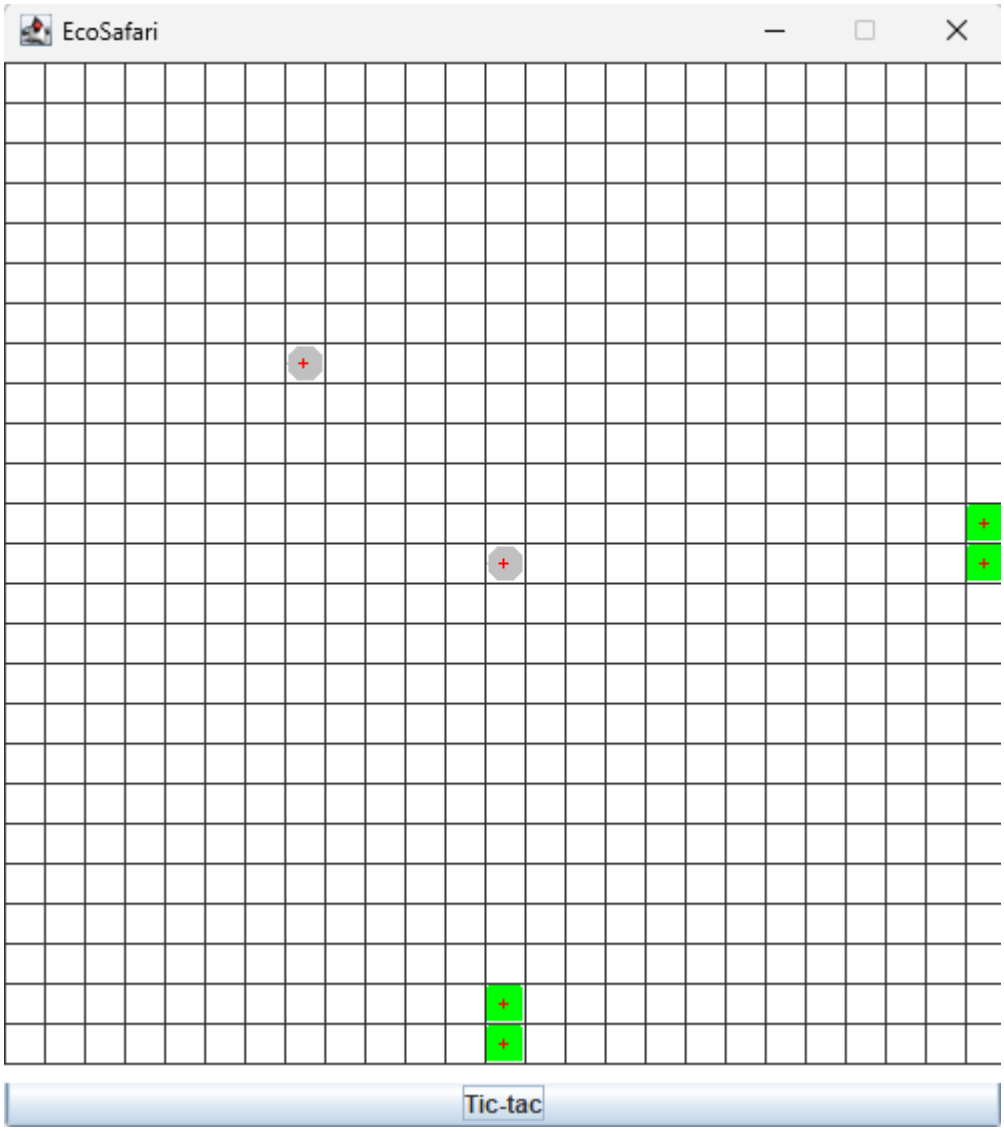
    //Babar will be in position (10,10) but in a matrix it represents [9][9]
    Elephant babar = new Elephant(this, 9,9);

    //We create mopane and acacia in positions near to the border:
    Bush mopane = new Bush(this, 24,12,false);
    Bush acacia = new Bush(this, 12,24,false);
```

a) ¿Cómo quedarían después de 3 Tic-tac? Ejecuten el programa y hagan 3 clics en el botón. Capturen una pantalla significativa



Después de 3 tics:



b) ¿Es correcto?
Sí, porque tiene como prioridad la dirección del norte.

C. Ciclo 3. Adicionando tormentas

El objetivo de este punto es incluir tormentas (sólo vamos a permitir el tipo básico). En una tormenta (i) su centro se desplaza circularmente en diagonal en dirección noreste, (ii) se afecta un área diámetro tres, (iii) destruye lo que encuentra en su centro, (iv) su centro es negro y vuelve gris el área afectada

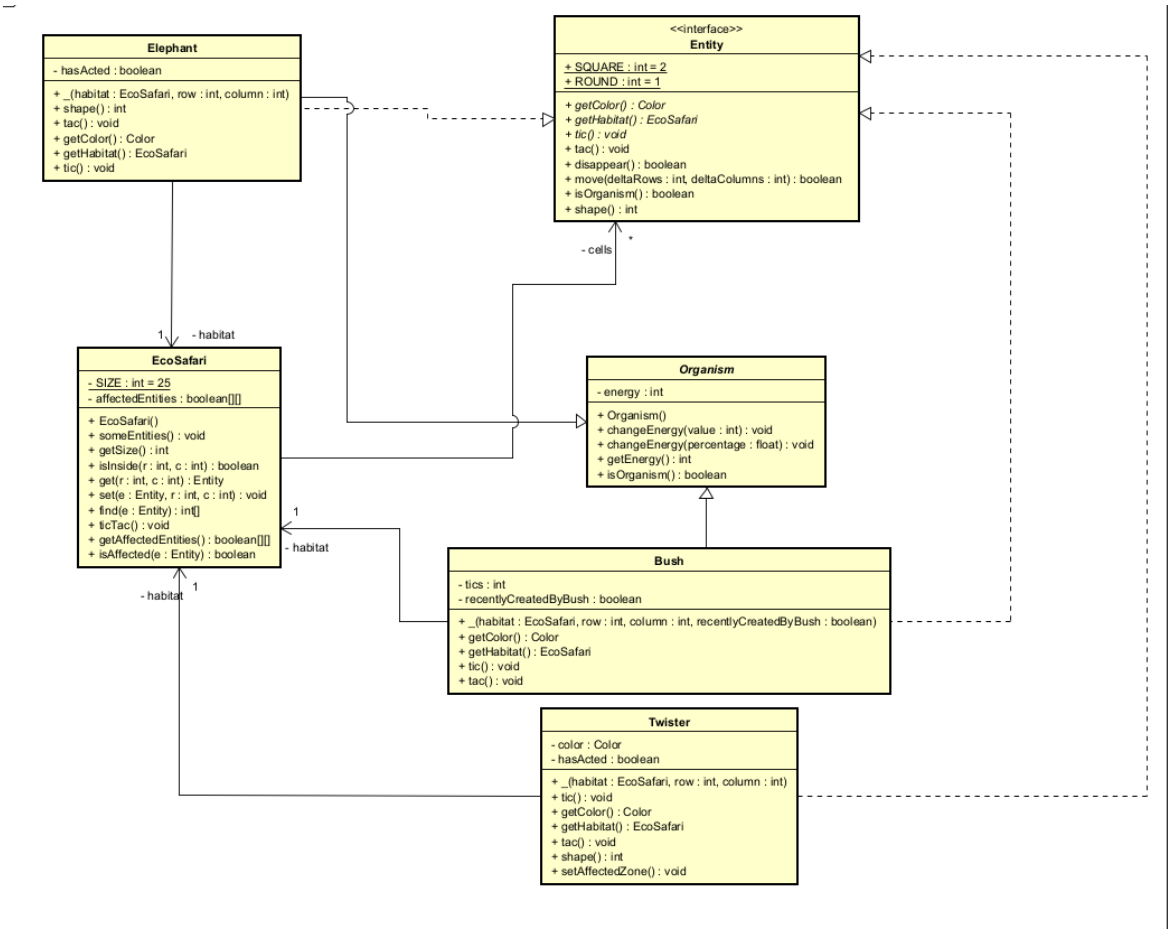
1. Para poder adicionar tormentas,
   a) ¿Debe cambiar en el código de EcoSafari en algo?
   Sí
   b) ¿Por qué?
   Porque a la hora de hacer un set, si hay un tornado en esa posición no se puede poner, porque el tornado lo destruye.

```java
/**
 * Places an entity at a specified position there must not be a twister there.
 * @param r the row
 * @param c the column
 */
public void set(Entity e, int r, int c){
    if (isInside(r,c) && !(get(r,c) instanceof Twister)){
        cells[r][c]=e;
    }
}
```

c) ¿Debe extender Entity?
No, no es necesario.

d) ¿Por qué?
Porque todo lo que está en Entity ya es suficiente para que Twister pueda funcionar.

2. Diseñen, construyan y prueben esta nueva clase (Mínimo dos pruebas de unidad).



(Se arreglaron algunos return de EcoSafari) Se agrega el nuevo concepto twister, además de algunos métodos, el más interesante es setAffectedZone(), donde el Twister se encarga de poner la zona afectada en EcoSafari. Puesto que era

necesario ver la zona que afecta el tornado, en EcoSafari se creo el atributo de affectedEntitities, además de dos métodos isAffected y getAffectedEntitites.

```java
/**
 * Twister should move diagonally in direction northeast
 */
@Test
public void shouldTwisterMoveDiagonally(){
    Twister twister = new Twister(es, 7,7);
    es.ticTac();

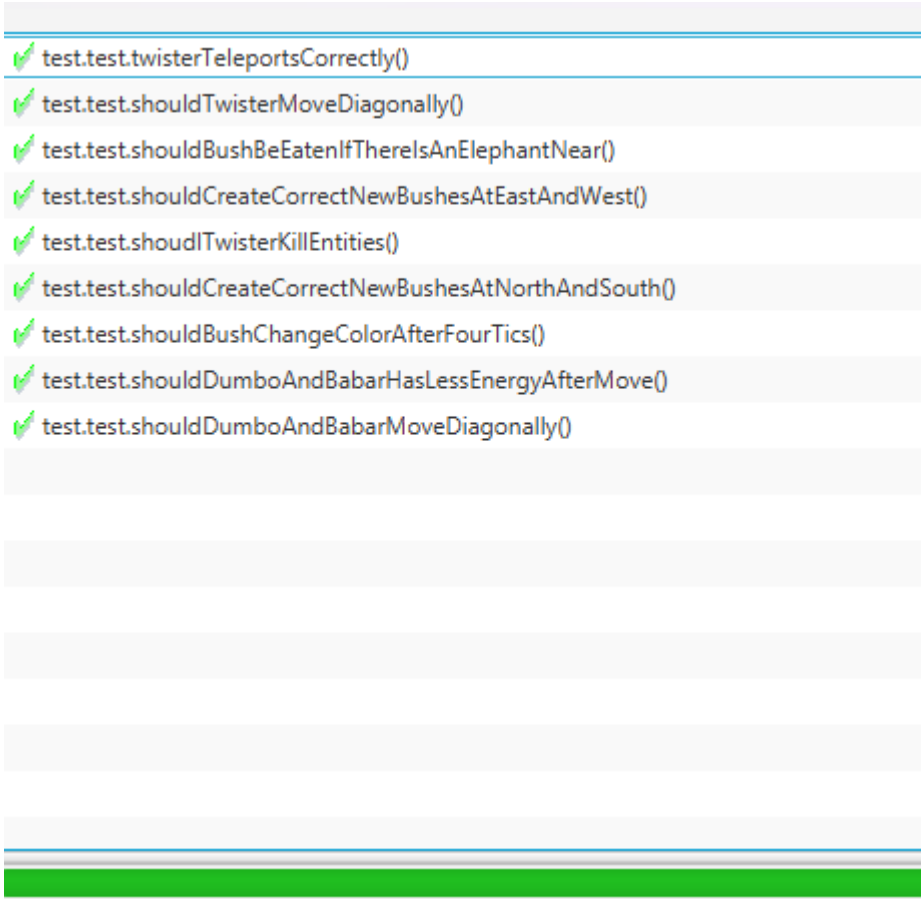
    assertEquals(es.get(7,7), null);
    assertEquals(twister, es.get(6,8));
}

/**
 * Twister should kill entities.
 */
@Test
public void shoudlTwisterKillEntities(){
    Twister twister = new Twister(es, 7,7);
    Bush bush1 = new Bush(es,6,8,false);
    es.ticTac();
    es.ticTac();

    assertEquals(es.get(6,8), null);
}

/**
 * Twister teleports correctly to the other side of the map if it is in a border.
 */
@Test
public void twisterTeleportsCorrectly(){
    Twister twister1 = new Twister(es, 0,1);
    Twister twister2 = new Twister(es, 23,24);
    es.ticTac();

    assertEquals(es.get(1,0), twister1);
    assertEquals(es.get(24,23), twister2);
}
```


test.test.twisterTeleportsCorrectly()
test.test.shouldTwisterMoveDiagonally()
test.test.shouldBushBeEatenIfThereIsAnElephantNear()
test.test.shouldCreateCorrectNewBushesAtEastAndWest()
test.test.shoudlTwisterKillEntities()
test.test.shouldCreateCorrectNewBushesAtNorthAndSouth()
test.test.shouldBushChangeColorAfterFourTics()
test.test.shouldDumboAndBabarHasLessEnergyAfterMove()
test.test.shouldDumboAndBabarMoveDiagonally()

```java
/**
 * Represents a Twister implements Entity<br>
 * <b>(habitat, tics, color)</b>
 */
public class Twister implements Entity{

    private Color color;
    private EcoSafari habitat;
    private boolean hasActed;

    /**
     * Constructs a new Twister.
     * @param habitat is Twister's habitat.
     * @param row is the row position.
     * @param column is the column position.
     */
    public Twister(EcoSafari habitat,int row, int column){
        this.habitat=habitat;
        habitat.set((Entity)this, row, column);
        color = Color.black;
    }

    @Override
    /**
     * Gives the Twister's habitat.
     * @return Twister's habitat.
     */
    public EcoSafari getHabitat(){
        return habitat;
    }
```

```java
@Override
/**
 * Gives Twister's color depending its energy.
 * @return Twister's color.
 */
public final Color getColor(){
    return color;
}
```

```java
@Override
/**
 * Creates a new bush near to it, if a place is empty, first looks the North, then South, then
 * East and finally West. This just happens if there are two tics
 */
public void tic(){
    if(!hasActed){
        if(!move(-1,1)){
            int[] posTwister = habitat.find(this);
            if(!move(posTwister[1],-posTwister[1])){
                move(-posTwister[0]+posTwister[1],posTwister[0]-posTwister[1]);
            }
        }
    }
    hasActed = true;
}
```

```java
@Override
/**
 * It change hasActed to false.
 */
public void tac(){
    setAffectedZone();
    hasActed=false;
}

@Override
/**
 * Gives how Twister looks like.
 * @return the number that represents the shape.
 */
public final int shape(){
    return Entity.ROUND;
}

/**
 * Changes the affectedZone for entities
 */
public void setAffectedZone(){
    boolean[][] affectedEntities = habitat.getAffectedEntities();
    int[] posTwister = habitat.find(this);

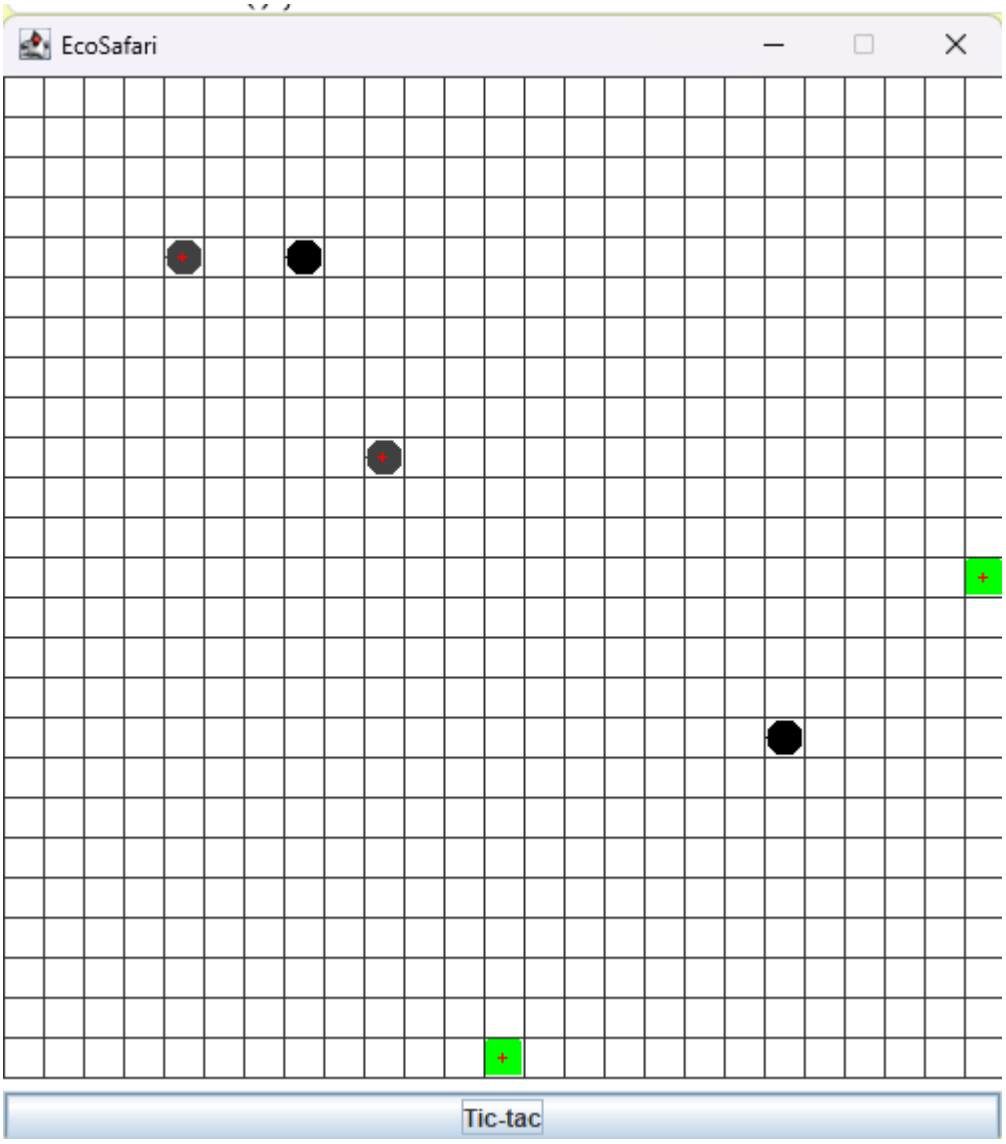
    for(int i = -1+posTwister[0]; i<=1+posTwister[0];i++){
        for(int j = -1+posTwister[1]; j<=1+posTwister[1]; j++){
            if(habitat.isInside(i,j) && !(habitat.get(i,j) instanceof Twister)){
                affectedEntities[i][j] = true;
            }
        }
    }
}
```

3. Adicionen dos objetos tormentas en EcoSafari, llámenlos Thor y Tempest,

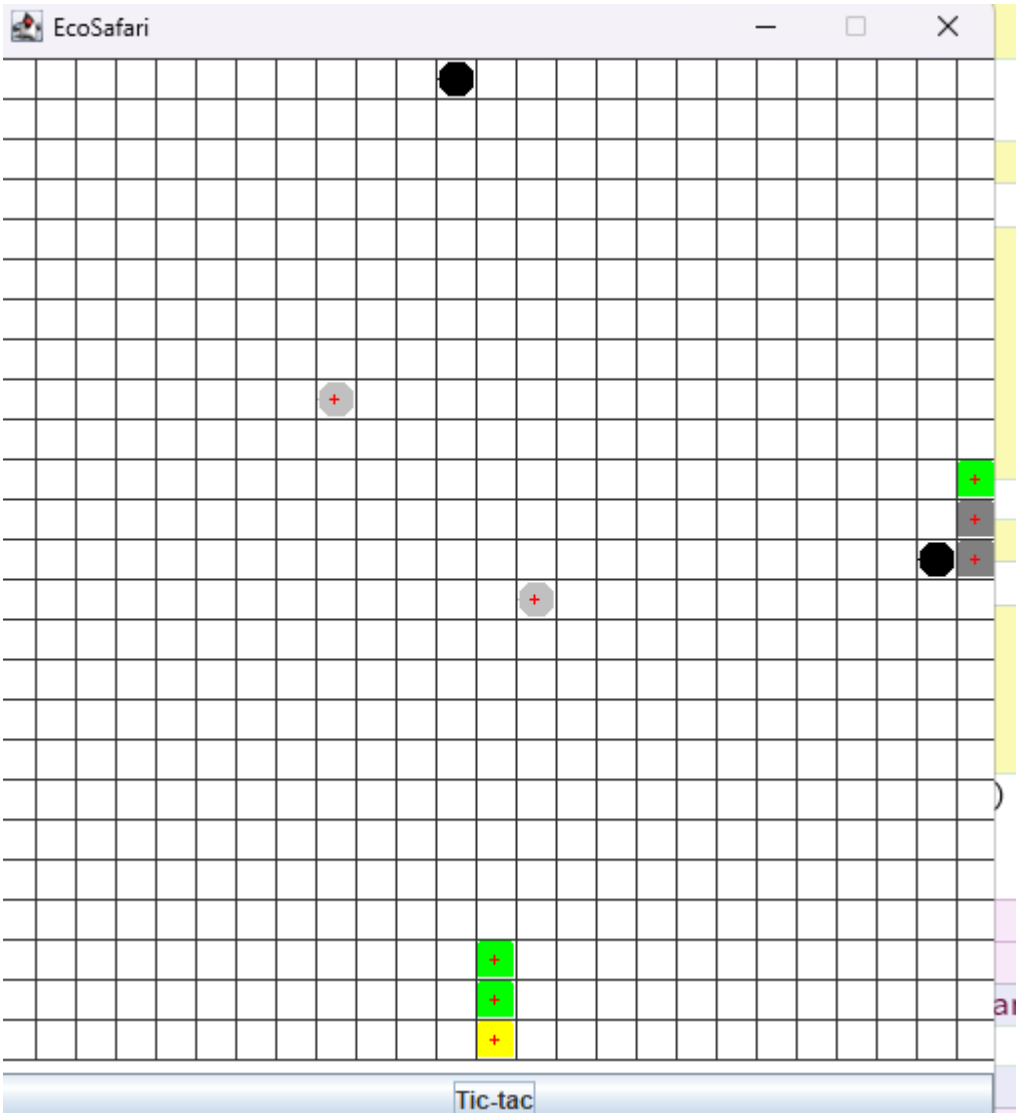
```java
21
22      /**
23       * Populates the EcoSafari with some entities
24       */
25      public void someEntities(){
26          //We create dumbo and babar:
27          //Dumbo will be in position (5,5), but in a matrix it represents [4][4]
28          Elephant dumbo = new Elephant(this, 4,4);
29
30          //Babar will be in position (10,10) but in a matrix it represents [9][9]
31          Elephant babar = new Elephant(this, 9,9);
32
33          //We create mopane and acacia in positions near to the border:
34          Bush mopane = new Bush(this, 24,12,false);
35          Bush acacia = new Bush(this, 12,24,false);
36
37
38          //We put two twisters Thor y Tempest
39          Twister thor = new Twister(this, 4,7);
40          Twister tempest = new Twister(this, 16,19);
41      }
```

a) ¿Cómo quedarían después de cuatro Tic-tac? Ejecuten el programa y hagan 5 clics en el botón. Capturen una pantalla significativa.

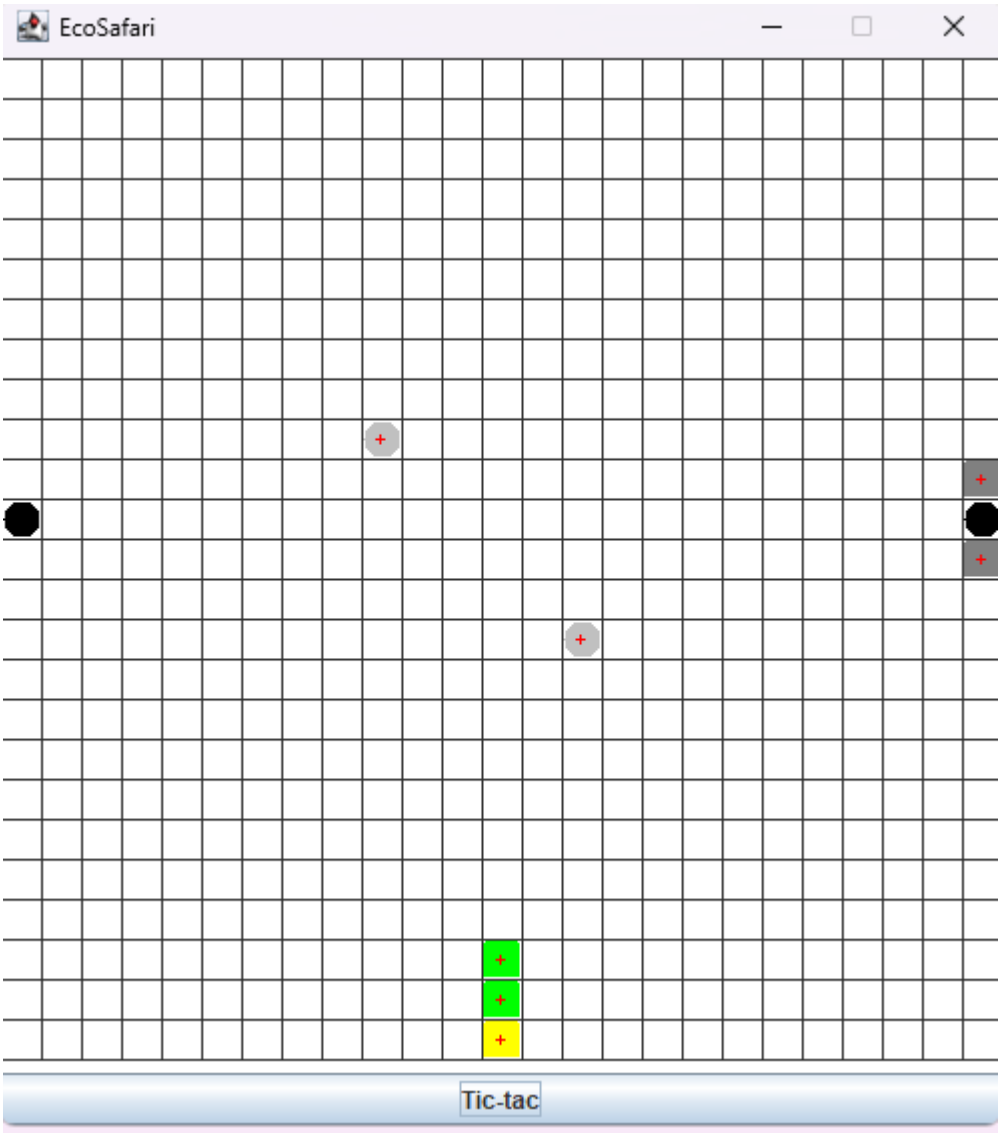
0 Tic-Tac:



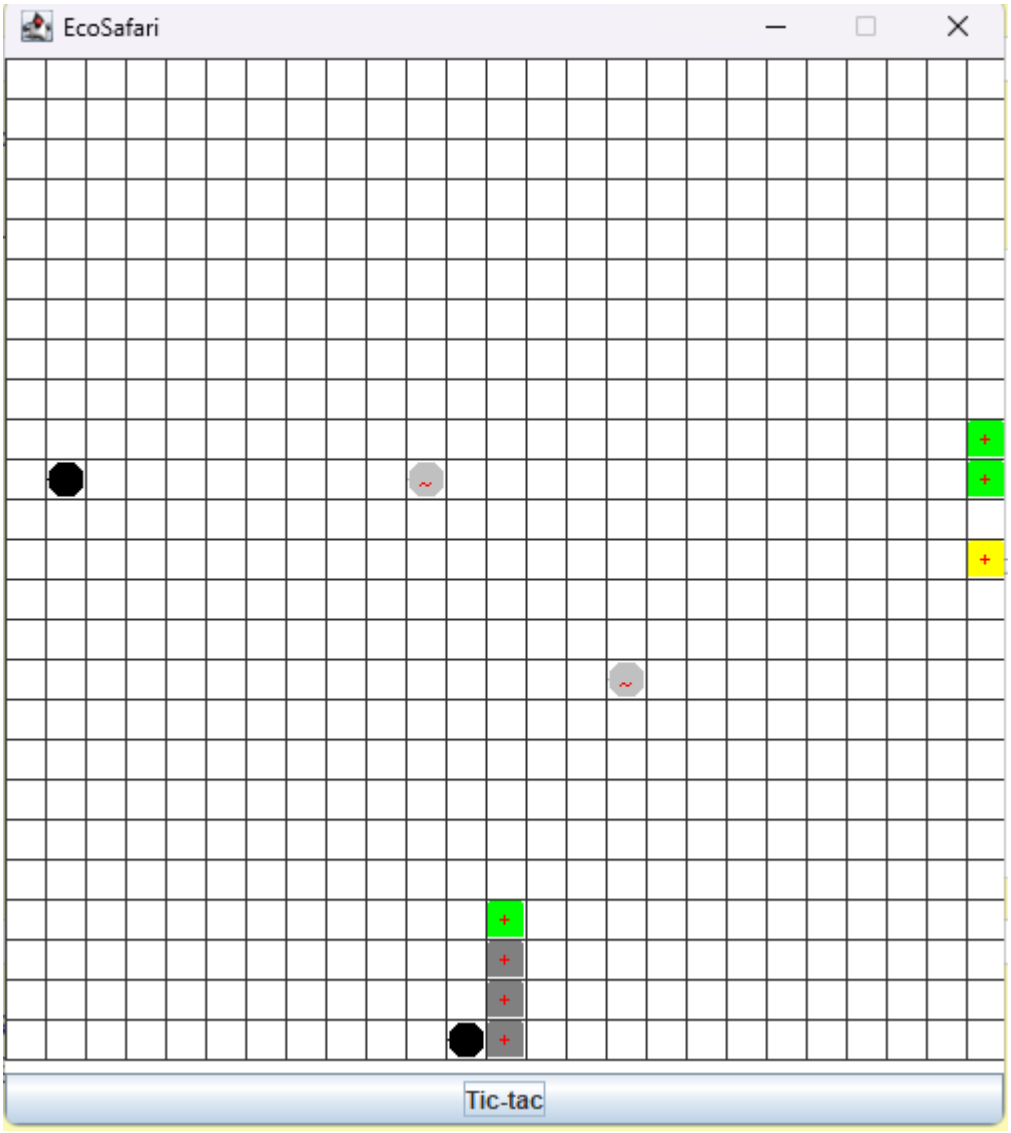
Cuatro Tic-Tac:



5to tic-Tac:



6to Tic-tac:



b) ¿Es correcto?
Sí, porque probamos varias cosas:

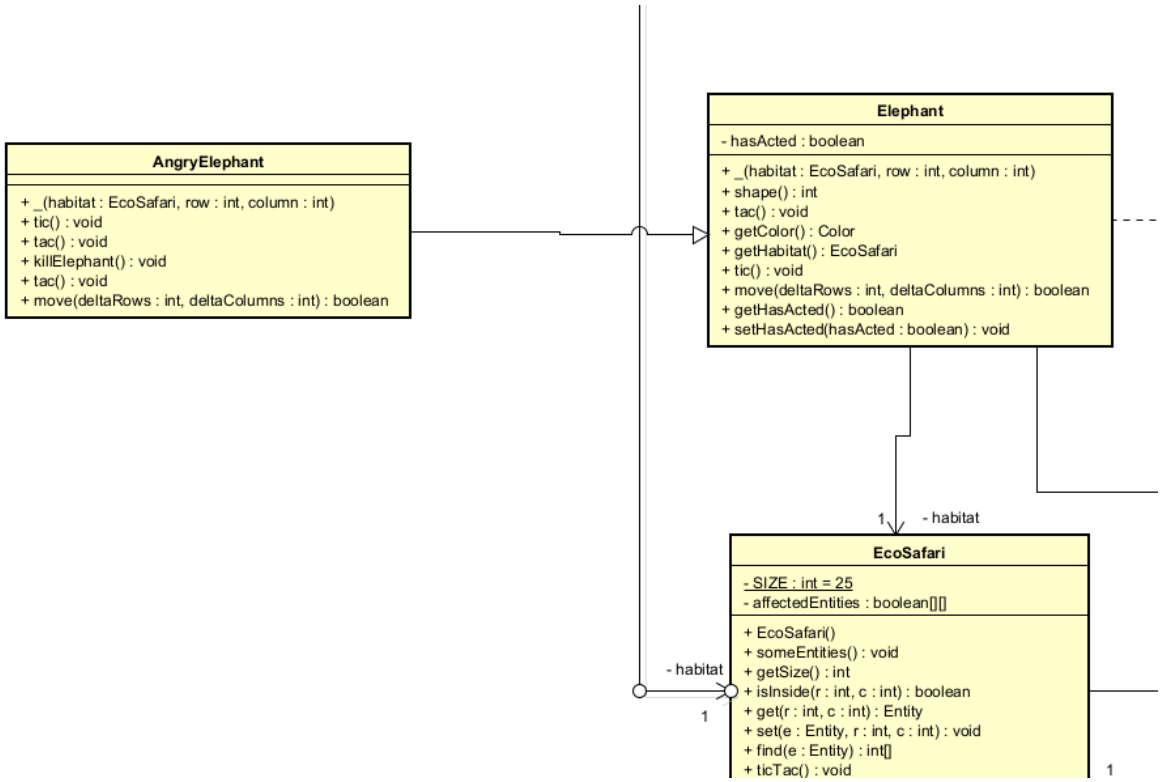
1. Se mueve diagonalmente como debe ser
2. Si llega al borde se teletransporta al otro lado del mapa, pero siempre en la misma línea horizontal
3. Genera sombra en las entidades que tiene alrededor
4. Se come todo lo que toca.

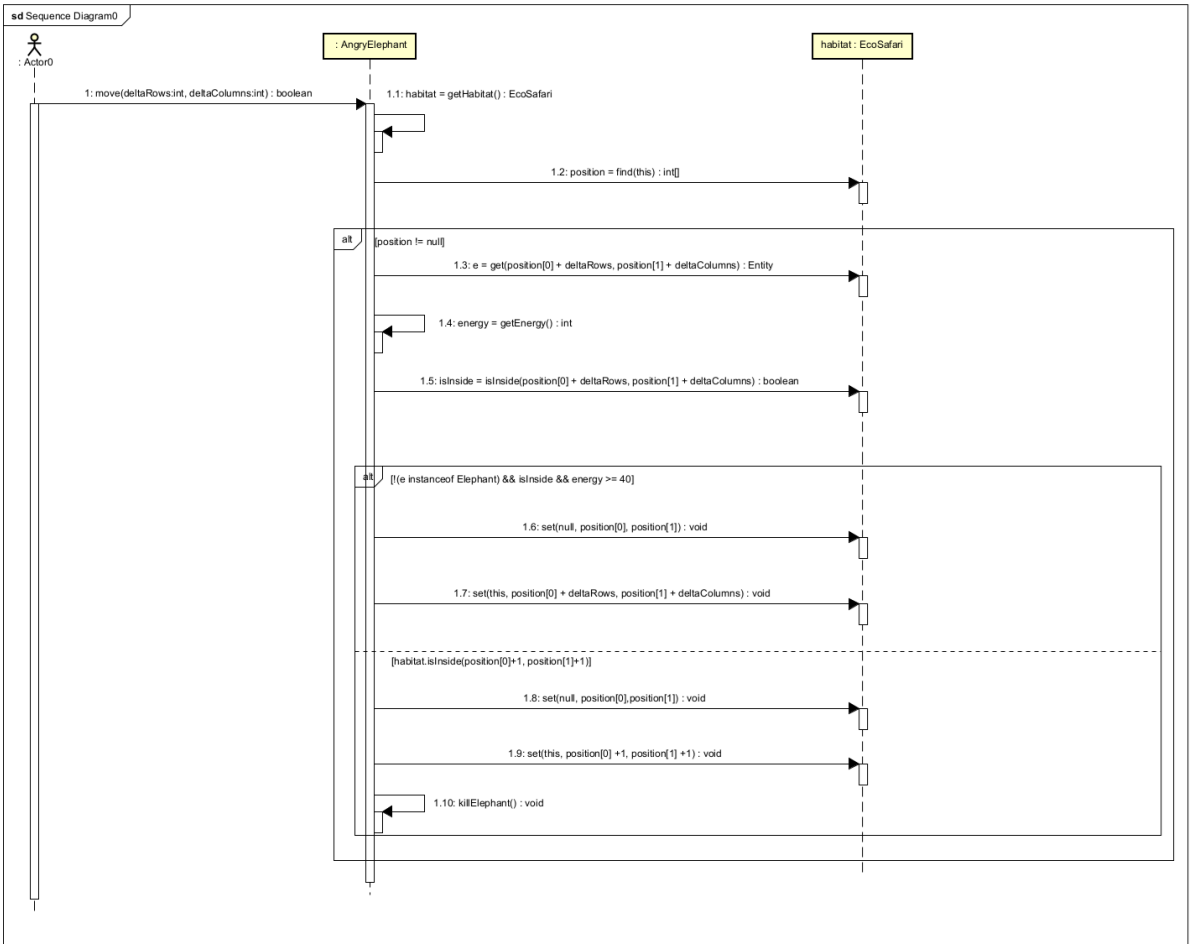
D Ciclo 4. Proponiendo y diseñando: Nuevo elefante

El objetivo de este punto es permitir recibir un nuevo tipo de Elephant

1. Propongan, describan e implementen el nuevo tipo de Elephant . (Mínimo dos pruebas de unidad)

-La propuesta es que realizaremos una nueva clase llamada **AngryElephant** el cual es una nueva entidad para “extender” Elephant, este tendrá características particulares, como poder matar a elefantes normales, poder correr (en cada tic moverse 2 casillas) siempre que tenga una energía superior a 40, cada vez que mate a otro elefante, toma la energía que tenía y la agrega a la que tiene actualmente. Implementacion:






```java
/**
 * Angry elephant must kill a normal elephant and walk one step if an elephant is 2 squares away
 */
@Test
public void shouldAngryElephantKillaElephantAndWalkOneStep(){
    Elephant elephant1 = new Elephant(es, 20, 20);
    AngryElephant elephant2 = new AngryElephant(es, 18, 18);
    es.ticTac();

    assertEquals(es.get(19,19), elephant2);
    assertEquals(es.get(20,20), null);

}
```

```java
/**
 * Angry Elephats must walk two steps
 */
@Test
public void shouldWalkTwoSteps(){
    AngryElephant elephant1 = new AngryElephant(es, 0, 0);
    es.ticTac();

    assertEquals(es.get(2,2), elephant1);
}
```

```java
package domain;
import java.awt.Color;

/**
 * Write a description of class AngryElephant here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class AngryElephant extends Elephant{
    /**
     * Constructs a new AngryElephant.
     * @param habitat is Angry Elephant's habitat.
     * @param row is the row position.
     * @param column is the column position.
     */
    public AngryElephant(EcoSafari habitat, int row, int column){
       super(habitat, row, column);
    }

    /**
     * It moves the Angry Elephant if it hasn't acted and it loses energy. If energy is 0 it disappears.
     */
    @Override
    public void tic(){
        if ((!this.getHasActed())){
            killElephant();
            if(move(2, 2)) {
                this.changeEnergy(-15);
            }
        }
        if (this.getEnergy()==0){
            this.disappear();
        }
        this.setHasActed(true);
    }

    /**
     * calls the killElephant method and change hasActed to false
     */
    @Override
    public void tac(){
        killElephant();
        this.setHasActed(false);
    }
```

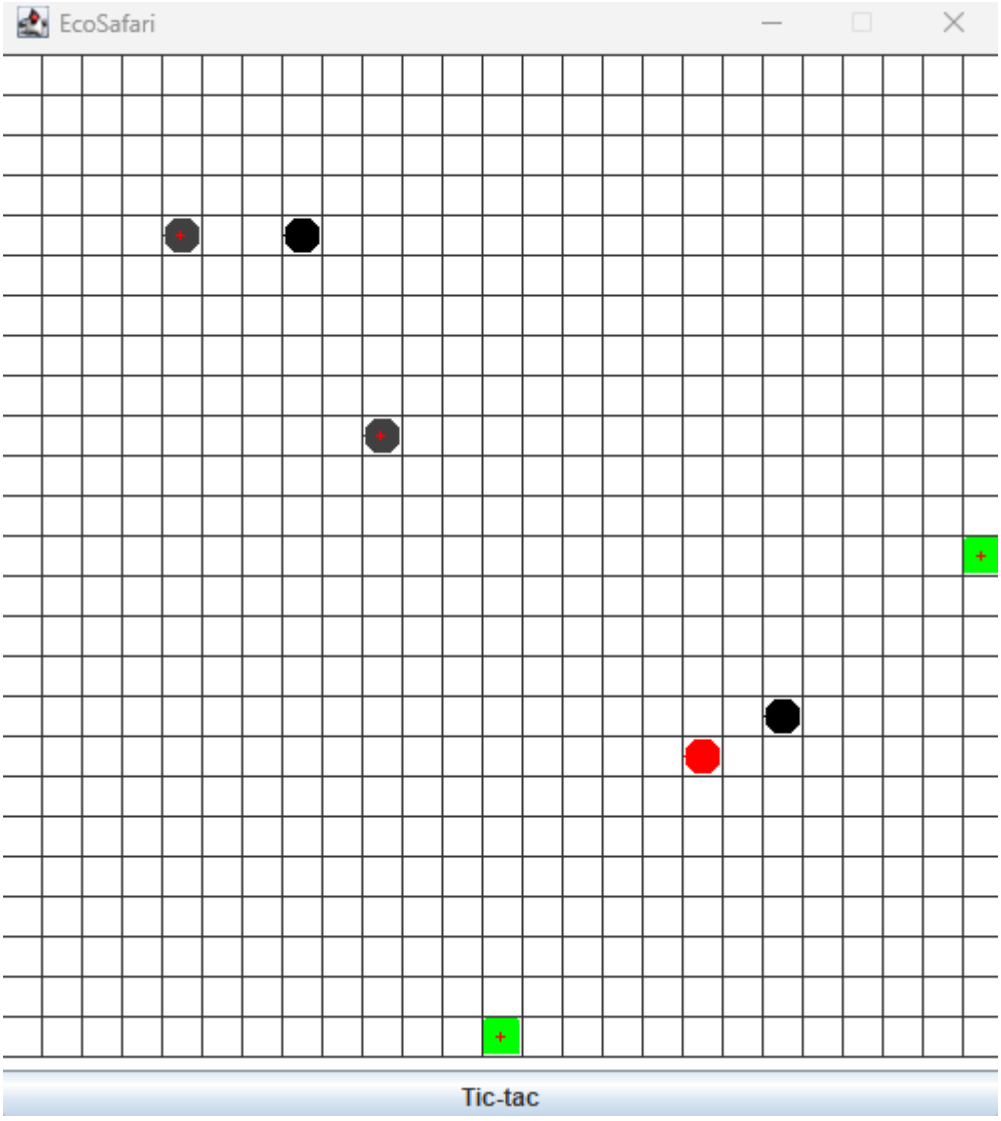
```java
/**
 * Angry Elephant kills a normal Elephant, provided that the elephant is one square away from the Angry Elephant
 */
public void killElephant(){
    EcoSafari habitat = this.getHabitat();
    int[] angryElephantPos = habitat.find(this);
    if(angryElephantPos != null){
        for(int i = -1; i <=1; i++){
            for(int j= -1; j <=1; j++){
                if((i != 0 || j != 0) && habitat.isInside(i,j)){
                    if(habitat.get(angryElephantPos[0]+i, angryElephantPos[1]+j) instanceof Elephant && !(habitat.get(angryElephantPos[0]+i, angryElephantPos[1]+j) instanceof AngryElephant)){
                        int a = this.getEnergy();
                        Elephant e = (Elephant)habitat.get(angryElephantPos[0]+i, angryElephantPos[1]+j);
                        this.changeEnergy(a+e.getEnergy());
                        e.disappear();

                    }
                }
            }
        }
    }
}
```

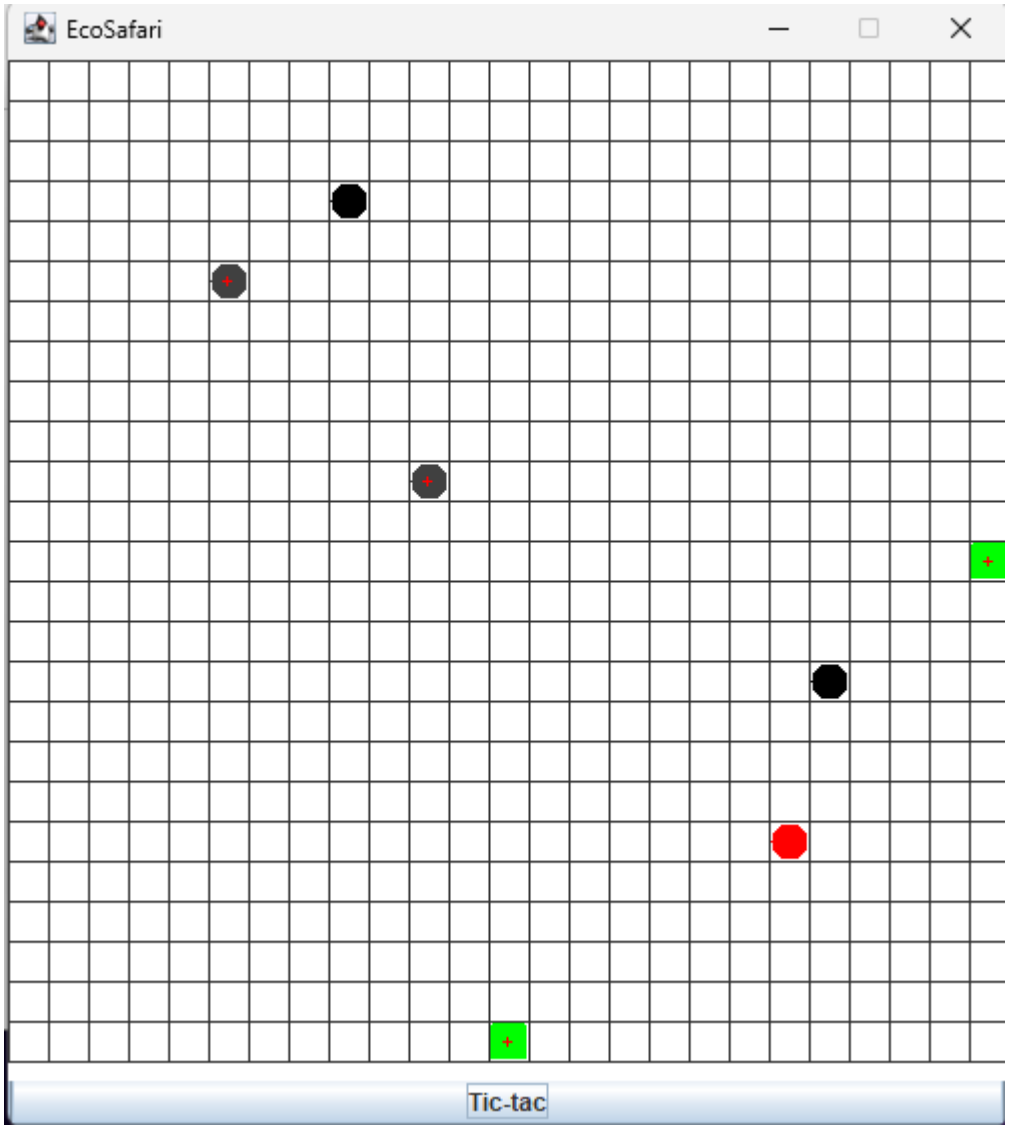
```java
@Override
/**
 * Gives AngryElephant's color depending its energy.
 * @return AngryElephant's color.
 */
public final Color getColor(){
    return(this.getEnergy()>=40? Color.RED: Color.ORANGE);
}
```

```java
@Override
/**
 * An Entity can move, by default it moves diagonally
 * @return if it was moved.
 */
public boolean move(int deltaRows, int deltaColumns){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.isInside(r+deltaRows,c+deltaColumns) && !(habitat.get(r+deltaRows,c+deltaColumns) instanceof Elephant) && this.getEnergy()>=40){
            habitat.set(null,r,c);
            habitat.set(this,r+deltaRows,c+deltaColumns);
            ok=true;
        }else if(habitat.isInside(r+1, c+1)){
            habitat.set(null,r,c);
            habitat.set(this,r+1,c+1);
            killElephant();
            ok=true;
        }
    }
    return ok;
}
```

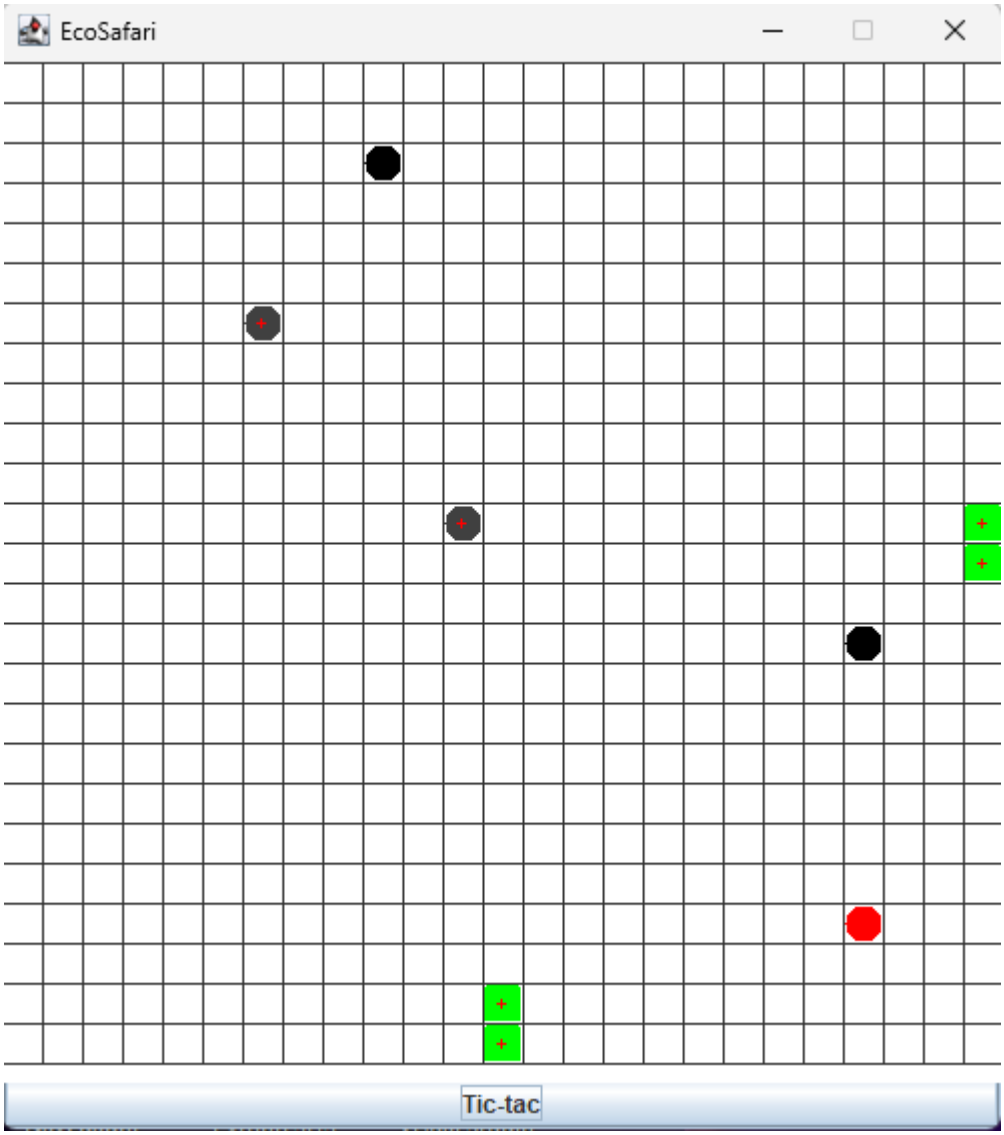
Se llama al main de EcoSafariGUI:



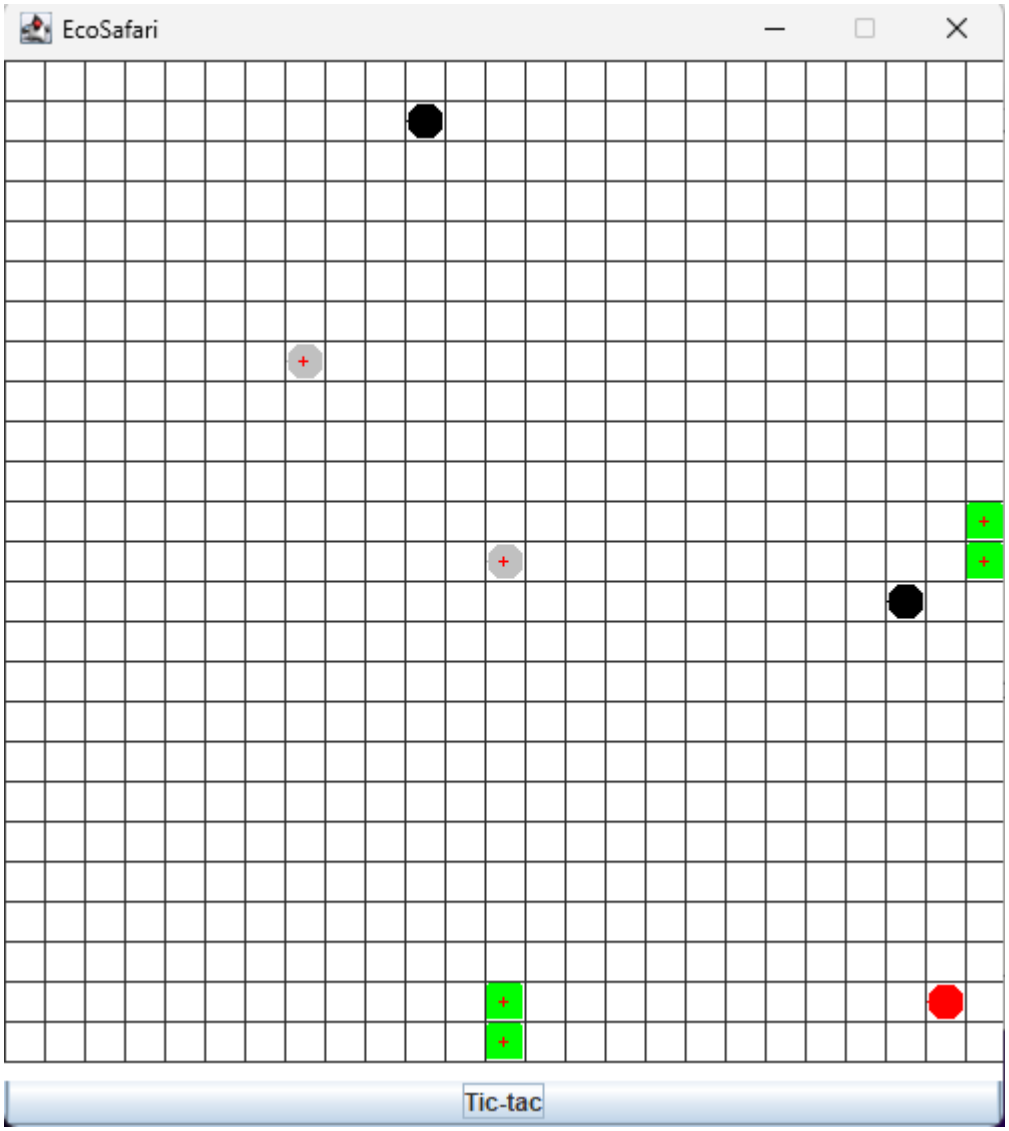
Primer tic-tac;



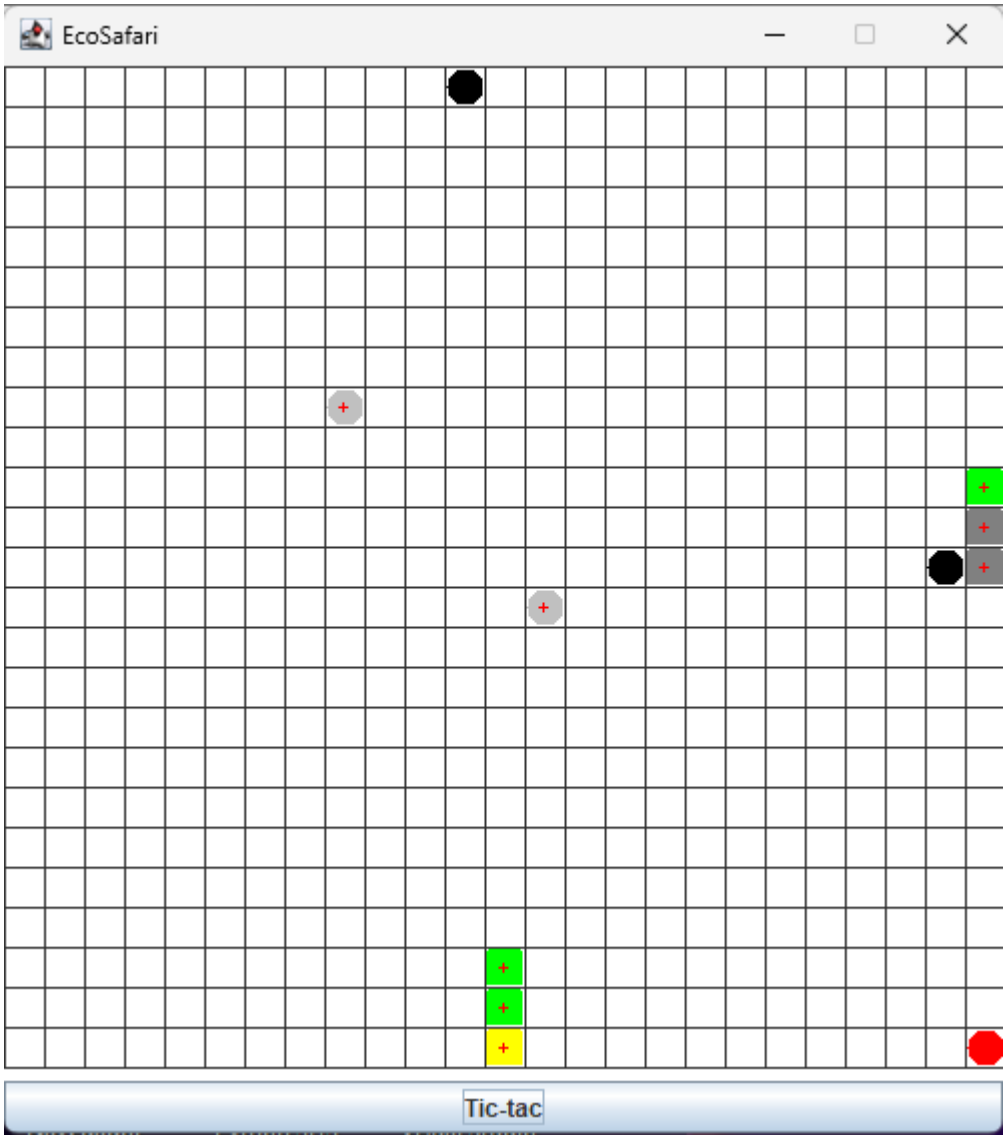
Segundo tic-tac:

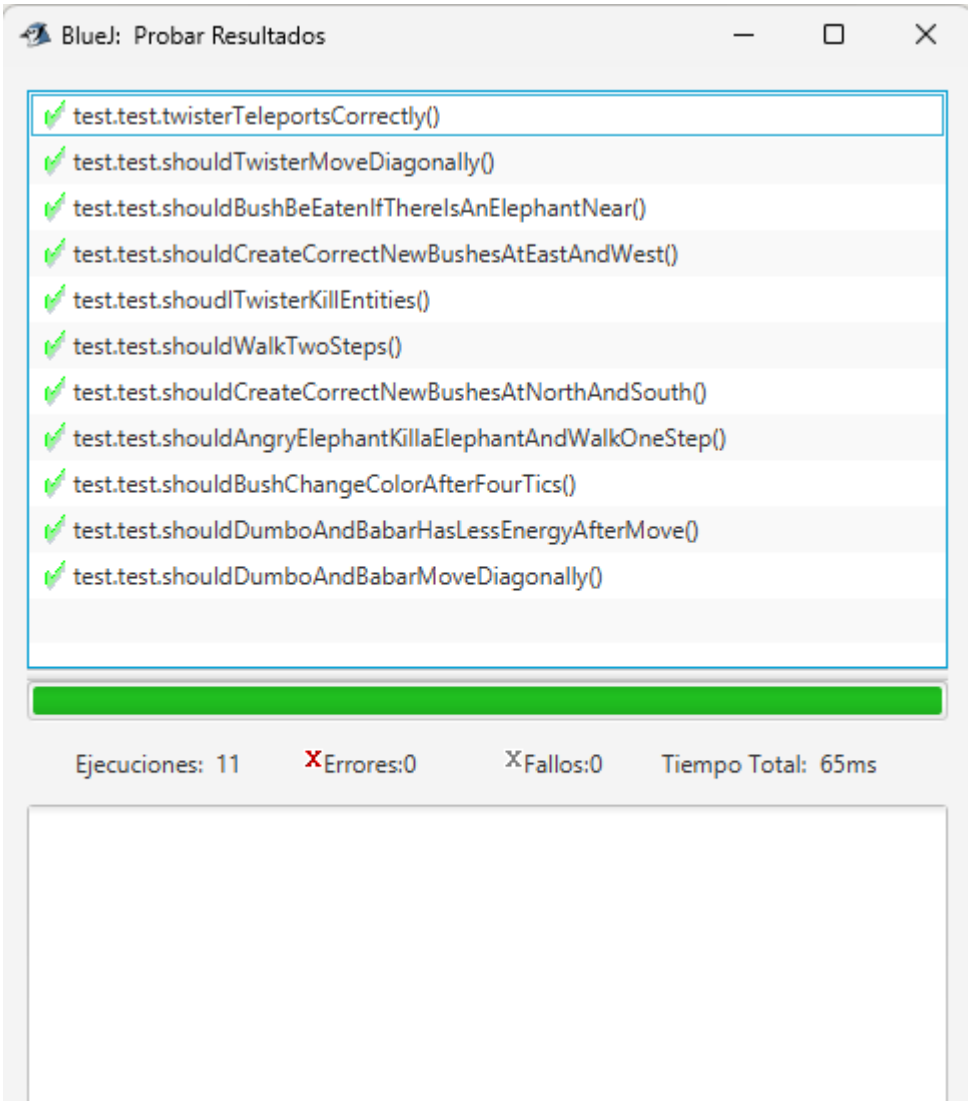


Tercer tic-tac:



Cuarto tic-tac:


EcoSafari
Tic-tac



2. Considerando una pareja de ellos con el apellido de ustedes. (a) Piensen en otra prueba significativa y expliquen la intención. (b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba. (c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.

A) La intención de la nueva prueba es que el nuevo objeto AngryElephant (Mendivelso) mate a 2 elefantes que están a una celda de distancia y que se desplace 2 celdas hacia abajo en diagonal

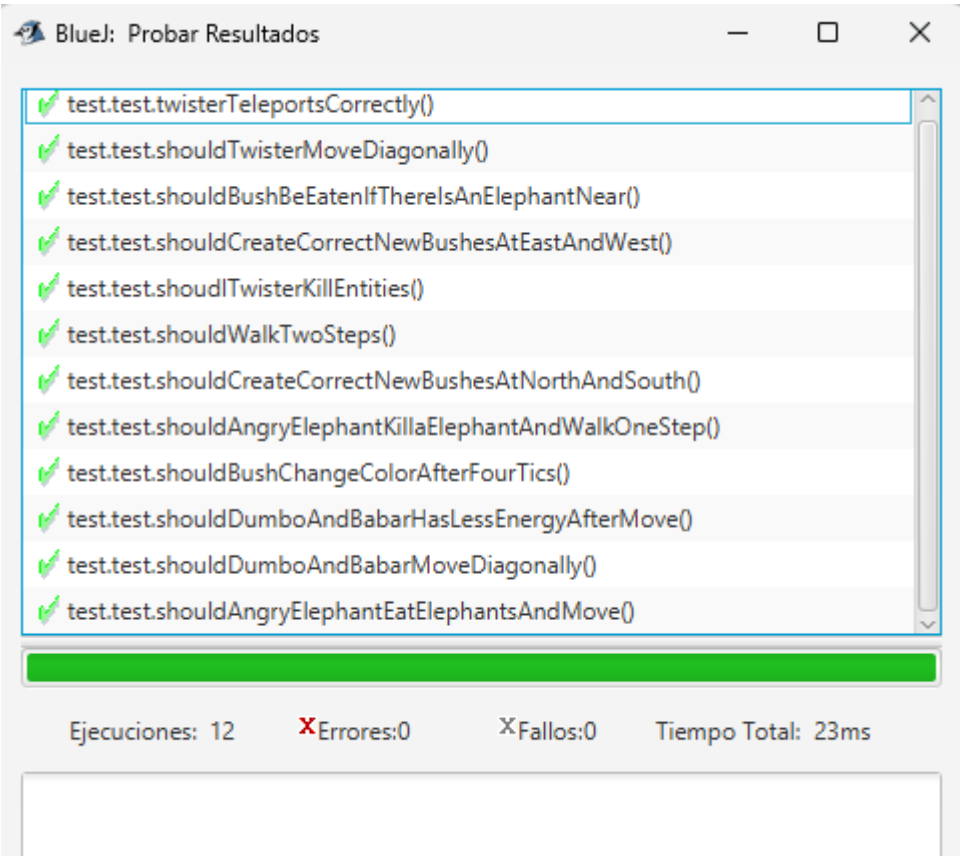
B)

```java
/**
 * Angry Elephant eats the elephants surrounding it and moves two spaces diagonally downwards.
 */
@Test
public void shouldAngryElephantEatElephantsAndMove(){
    AngryElephant Mendivelso = new AngryElephant(es, 7, 16);

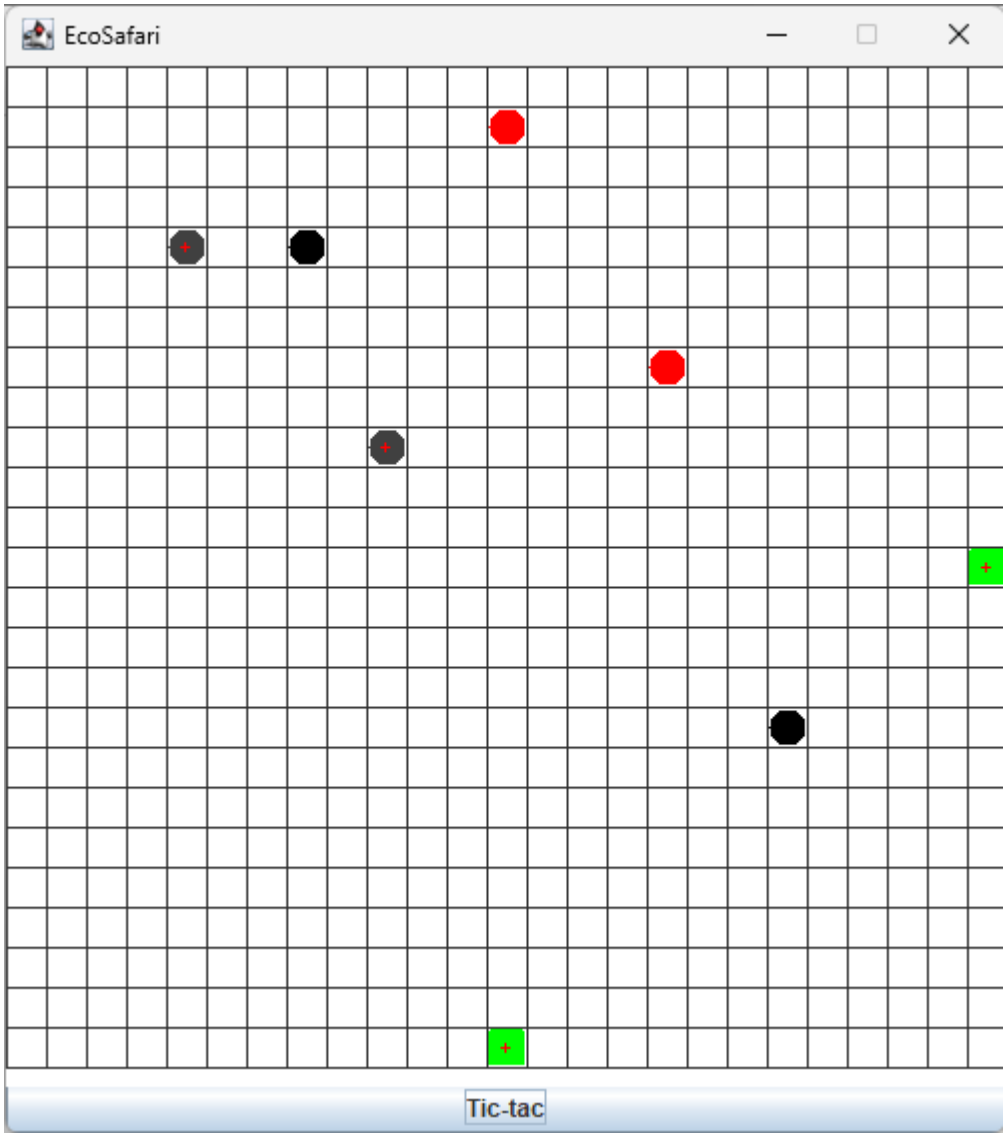
    Elephant e1 = new Elephant(es, 7, 17);
    Elephant e2 = new Elephant(es, 8, 15);
    es.ticTac();

    assertEquals(es.get( 7, 17), null);
    assertEquals(es.get( 8, 15), null);

    assertEquals(es.get(9,18), Mendivelso);
}
```



C)



En la prueba de unidad se van a crear dos elefantes, cuando hago tic-tac, el AngryElephant los mata, y avanza 2 casillas diagonal hacia abajo.

E Ciclo 5. Proponiendo y diseñando: Nueva entidad

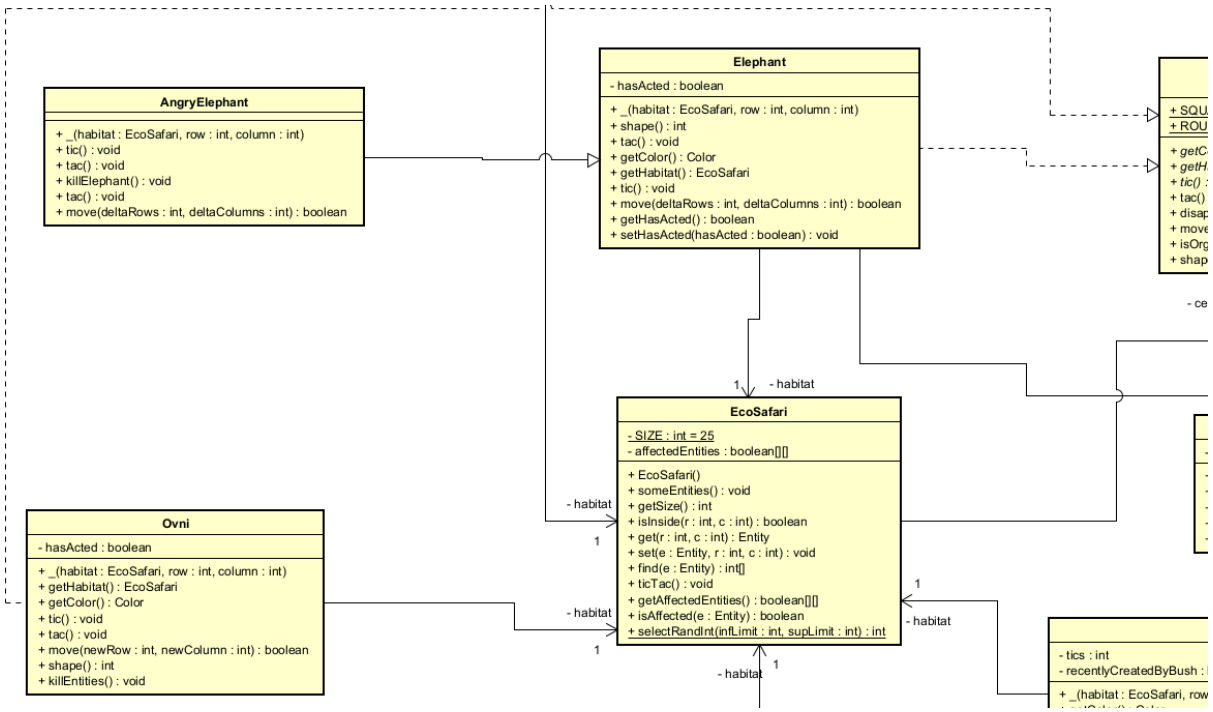
El objetivo de este punto es permitir recibir un tictac (clase de Entity (no Elephant) en el EcoSafari.

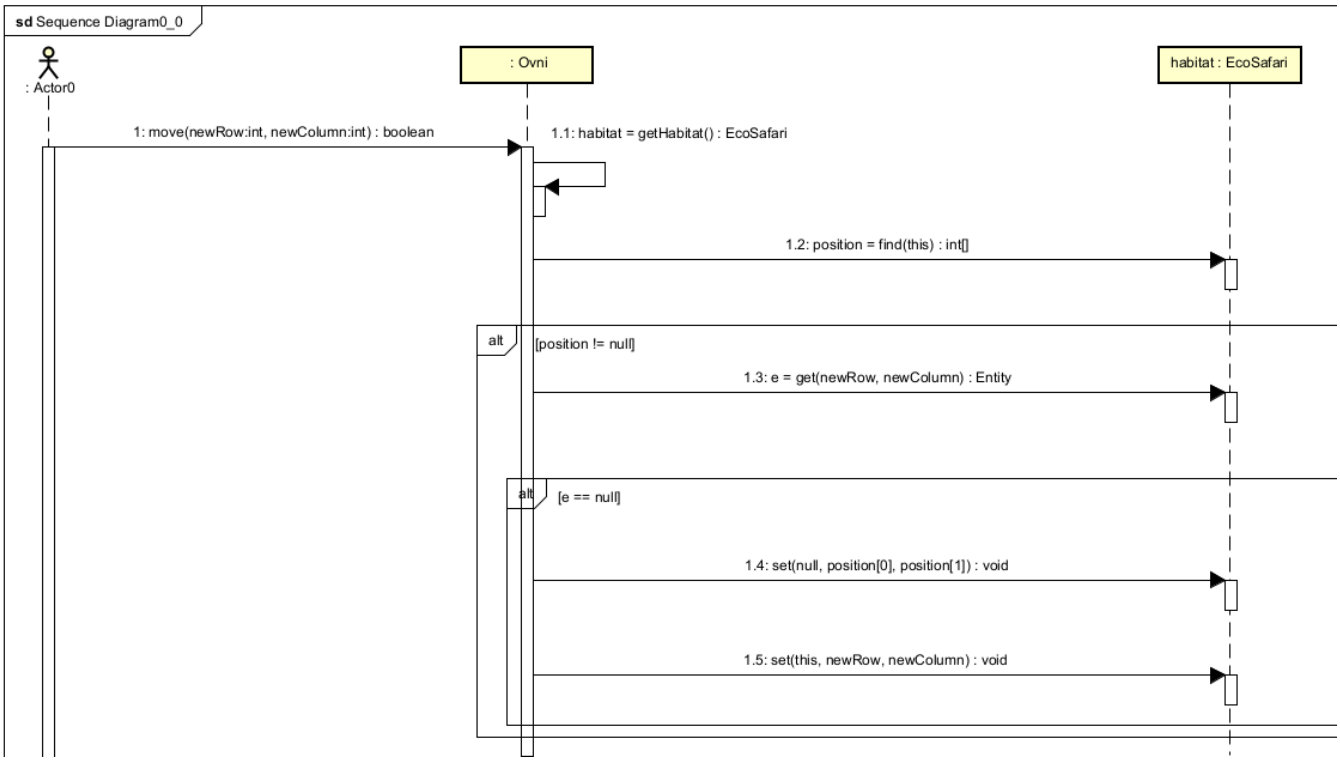
1. Propongan, describan e implementen la nueva clase de Entity . (Mínimo dos pruebas de unidad)

-Nuestra propuesta es realizar una nueva clase llamada Ovni, esta clase será una clase subhija de la clase Entity, El ovni se puede mover libremente de manera aleatoria por todo el hábitat, siempre verifica que cuando se teletransporte, donde caiga no haya nada, también, cuando se teletransporta, el ovni se encarga de matar a todos lo que

este a una casilla de distancia, menos el tornado, y el tornado es el único que puede matar al ovni.

Implementacion:





```java
/**
 * Ovni must kill all's entities except a Twisters
 */
@Test
public void shouldKillEntitys(){
    Ovni alien = new Ovni(es, 20, 3);
    es.ticTac();

    int[] position = es.find(alien);
    int f = position[0];
    int c = position[1];
    for(int i=-1 ; i<=1; i++){
        for(int j=-1 ; j<=1; j++){
            if(i!=0 || j!=0){
                assertTrue(es.get(f+i, c+j) == null || es.get(f+i, c+j)instanceof Twister);
            }
        }
    }
}

/**
 * Alien 1 and Alien 2 must always be in different positions.
 */
@Test
public void shouldNotBeOnTopOfAnotherOvni(){
    Ovni alien1 = new Ovni(es, 20, 3);
    alien1.tic();

    Ovni alien2 = new Ovni( es, 20, 8);
    alien2.tic();


    int[] p1= es.find(alien1);
    int[] p2= es.find(alien2);

    assertFalse(p1 == p2);

}
```

```java
package domain;
import java.awt.Color;


/**
 * Write a description of class Ovni here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Ovni implements Entity
{
    private final EcoSafari habitat;
    private boolean hasActed;
    /**
     * Constructor for objects of class Ovni
     */
    public Ovni(EcoSafari habitat, int row, int column){
        this.habitat = habitat;
        habitat.set((Entity)this, row, column);
        hasActed = false;
    }

    @Override
    /**
     * Gives the Ovni's habitat.
     * @return Ovni's habitat.
     */
    public EcoSafari getHabitat(){
        return habitat;
    }

    @Override
    /**
     * Gives Ovni's color.
     * @return Ovni's color
     */
    public Color getColor(){
        return Color.GRAY;
    }
```

```java
@Override
/**
 *  It moves freely (randomly) throughout the simulator if it hasn't acted and if in the square when he will be stay is null
 */
public void tic(){
    int size = habitat.getSize();
    int f= habitat.selectRandInt(0, size);
    int c= habitat.selectRandInt(0, size);
    if ((!hasActed)){
        killEntities();
        boolean temp = false;
        while(temp==false){
            if(move(f, c)){
                temp=true;
            }else{
                f= habitat.selectRandInt(0, size);
                c= habitat.selectRandInt(0, size);
            }
        }
    }
    hasActed=true;
}

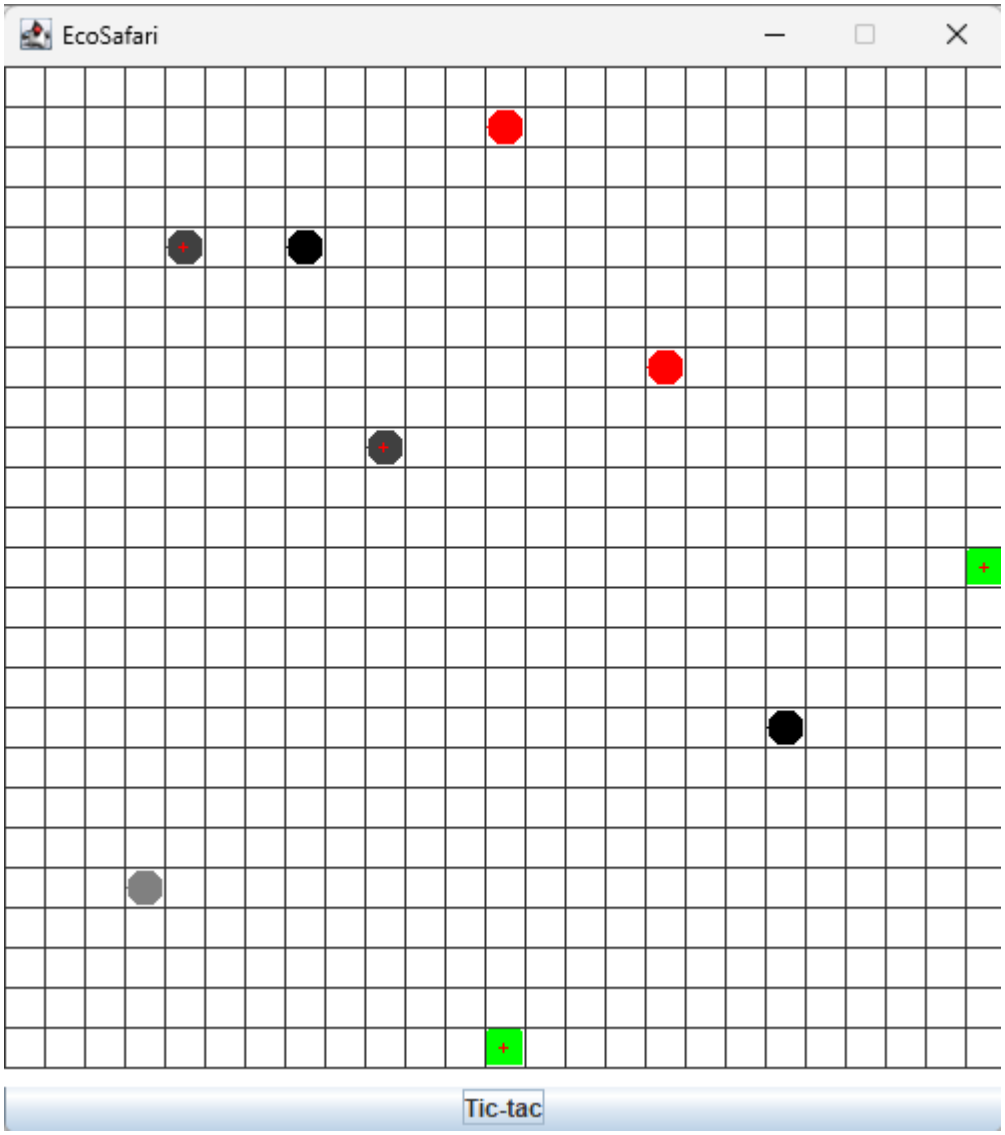
@Override
/**
 * calls the killEntities method and change hasActed to false
 */
public void tac(){
    killEntities();
    hasActed=false;
}
```

```java
@Override
/**
 * An Ovni can move
 * @return if it was moved.
 */
public boolean move(int newRow, int newColumn){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.get(newRow,newColumn)==null){
            habitat.set(null,r,c);
            habitat.set(this,newRow,newColumn);
            ok=true;
        }
    }
    return ok;
}
```

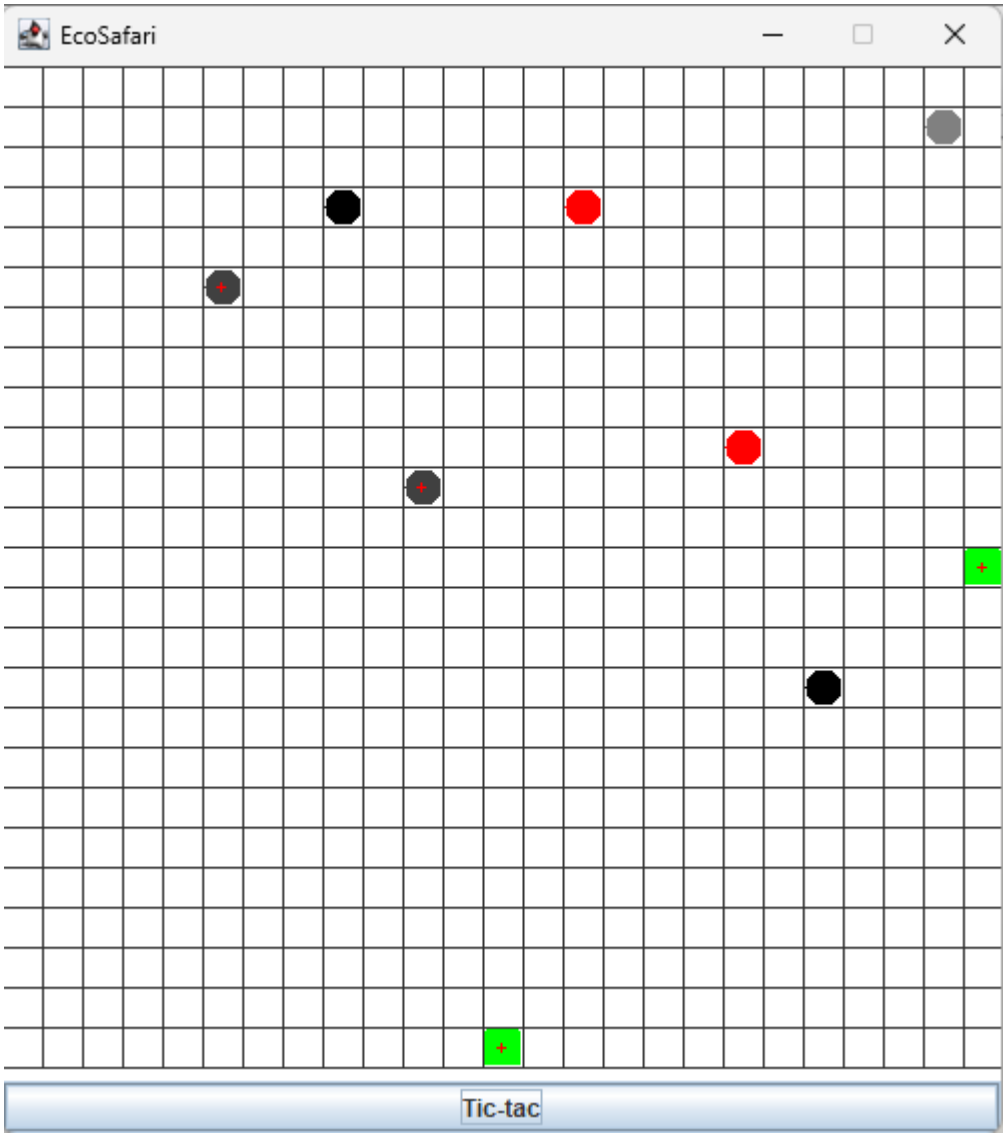
```java
@Override
/**
 * Gives how Ovni looks like.
 * @return the number that represents the shape.
 */
public final int shape(){
    return Entity.ROUND;
}
```

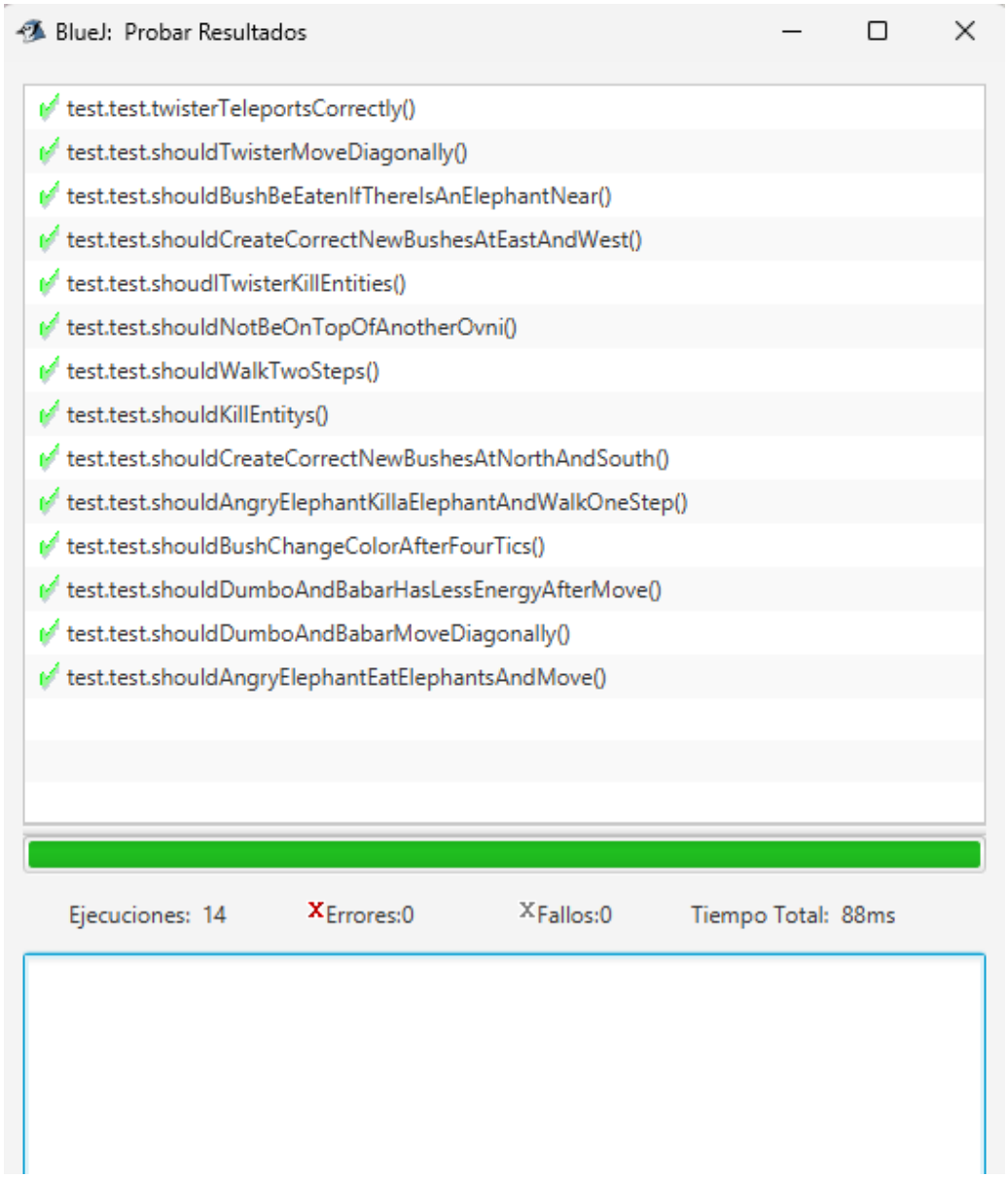
```java
/**
 * Kills any entity that is within a one square distance of him
 */
public void killEntities(){
    EcoSafari habitat = this.getHabitat();
    int[] OvniPos = habitat.find(this);
    if(OvniPos != null){
        for(int i = -1; i <=1; i++){
            for(int j= -1; j <=1; j++){
                if((i != 0 || j != 0)){
                    if(!(habitat.get(OvniPos[0]+i, OvniPos[1]+j) instanceof Twister) && !(habitat.get(OvniPos[0]+i, OvniPos[1]+j) instanceof Ovni) && habitat.get(OvniPos[0]+i, OvniPos[1]+j)!= null){
                        Entity e = (Entity)habitat.get(OvniPos[0]+i, OvniPos[1]+j);
                        e.disappear();
                    }
                }
            }
        }
    }
}
```

Llamado al main:



Primer tic-tac:


EcoSafari
Tic-tac



2. Considerando un par de ellos con el nombre de ustedes. (a) Piensen en otra prueba significativa y expliquen la intención. (b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba. (c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.

a) La intención de la nueva prueba es que el nuevo objeto Ovni (David) mate 2 arbustos y 1 elefante enojado que están a una celda de distancia

B)

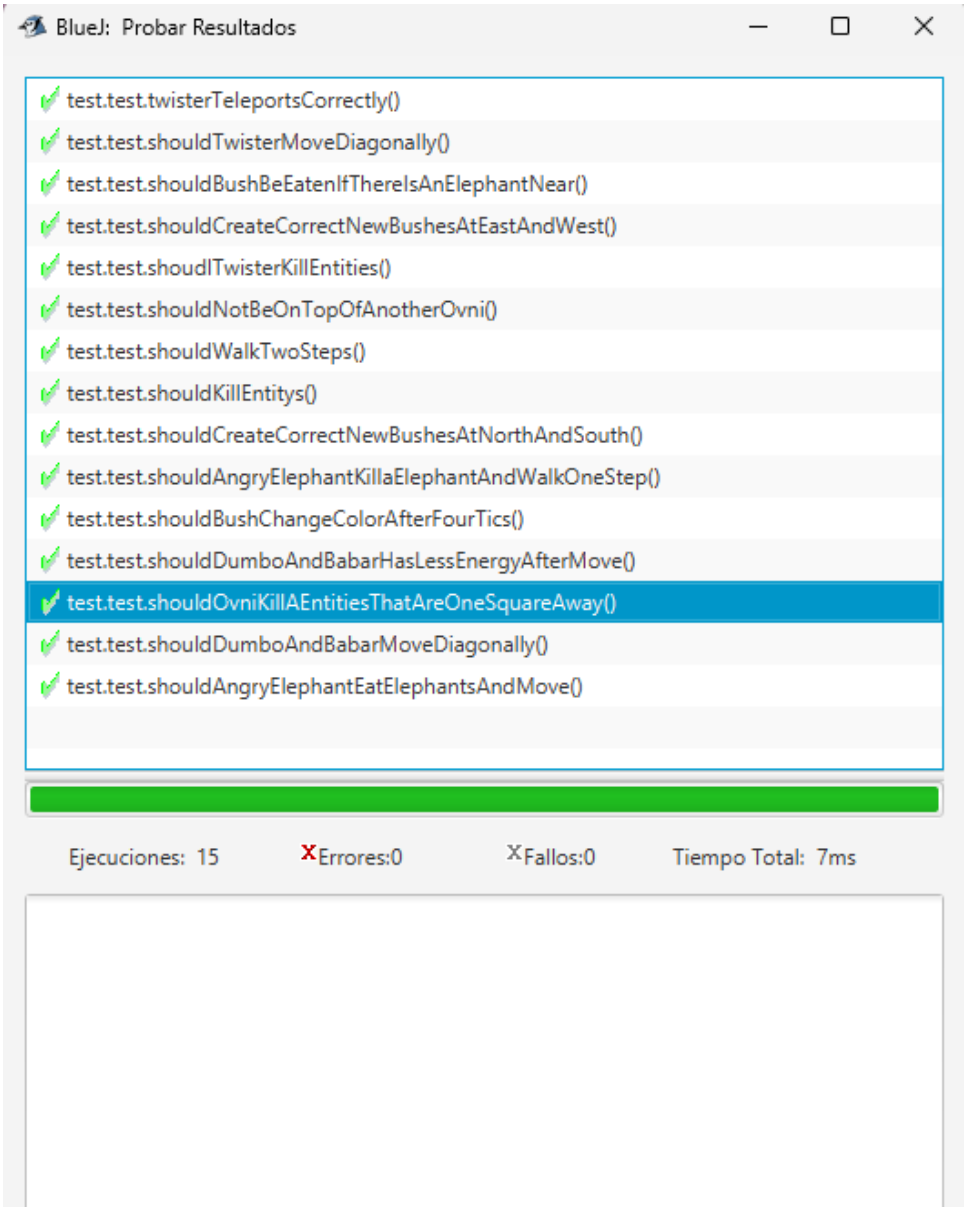
```java
/**
 * Ovni(David) must kill a Entities that are one square away.
 */
@Test
public void shouldOvniKillAEntitiesThatAreOneSquareAway(){
    Ovni David= new Ovni(es, 20, 3);

    Bush mopane = new Bush(es, 20,4,false);
    Bush acacia = new Bush(es, 21,3,false);

    AngryElephant elephant1 = new AngryElephant(es, 21, 4);
    es.ticTac();

    assertEquals(es.get(21,4), null);
    assertEquals(es.get(20,4), null);
    assertEquals(es.get(21,3), null);
}
```

c)



PARTE III. Desarrollando el reto

En este punto crearemos una zona en la pradera de EcoSafari dedicada a examinar lo que ocurre siguiendo un modelo depredador-presa-recurso que se comporta con las siguientes reglas:

- Cada celda tiene ocho celdas vecinas (vecindario de Moore).
- Las únicas entidades de la zona de la pradera son: tierra, pasto, cebra y león.
- Los animales se mueven libremente sobre tierra. Los leones avanzan una celda por paso y las zebras dos. Cada vez que se desplazan, pierden 10 % de su energía e intercambian su posición con la celda de tierra ocupada.
- Si la zebra queda vecina al pasto, se lo come y aumenta su energía en un 25 %.

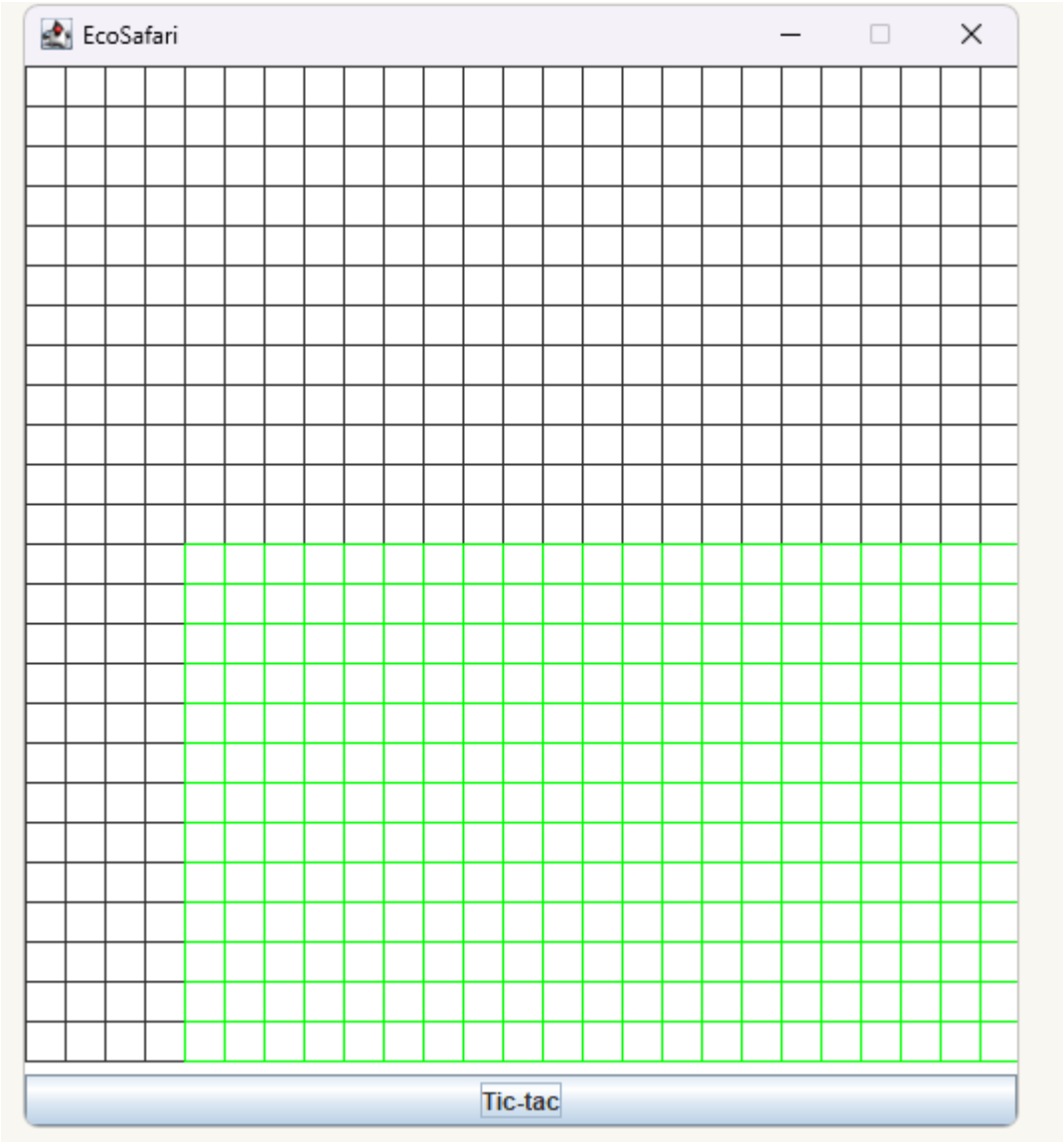
- Si el león queda vecino a una zebra, se la come y aumenta en un 50 % su energía.
- Si un animal queda a una casilla de tierra de otro de su misma especie, en la celda de tierra nace un nuevo animal de esa especie.
- Las celdas de tierra pueden generar pasto con una probabilidad del 10 %.
- Los animales no pueden tener más de 100 % de energía y mueren cuando se quedan sin energía.
- Cuando alguna entidad desaparece, es reemplazada por tierra.

Para la solución es indispensable el desarrollo de miniCiclos para ir adecuando todas las especificaciones necesarias, por tanto el orden de desarrollo será el siguiente:

0. Definir la zona donde iniciará el simulador
1. Crear la tierra y si la entidad desaparece, se remplaza por tierra.
2. Crear pasto y la tierra genera pasto con 10% de probabilidad
3. Implementación del vecindario de Moore
4. Crear zebra. Se mueve sobre la tierra. Avanzan dos celdas por paso. Pierde 10% de su energía, e intercambian su posición con la celda de tierra ocupada. Muere si se queda sin energía.
5. Si la zebra queda cerca al pasto se lo come y aumenta su energía en un 25%.
6. Crear león. Se mueve sobre la tierra. Avanza una celda por paso. Pierde 10% de su energía, e intercambian su posición con la celda de tierra ocupada. Muere si se queda sin energía.
7. Si el león queda cerca a la zebra se la come y aumenta su energía en un 50%.
8. Si un animal queda a una casilla de tierra de otro de su misma especie, en la celda de tierra nace un nuevo animal de esa especie.
9. Crear al azar el mapa de ecoSafari

MiniCiclo0 Definir la zona necesaria donde se hará el simulador:

Es necesario definir cuál será la zona especifica en la cual se desarrollará el simulador del reto, por tanto, inicialmente se define la zona verde, como esa zona donde se hará el simulador:

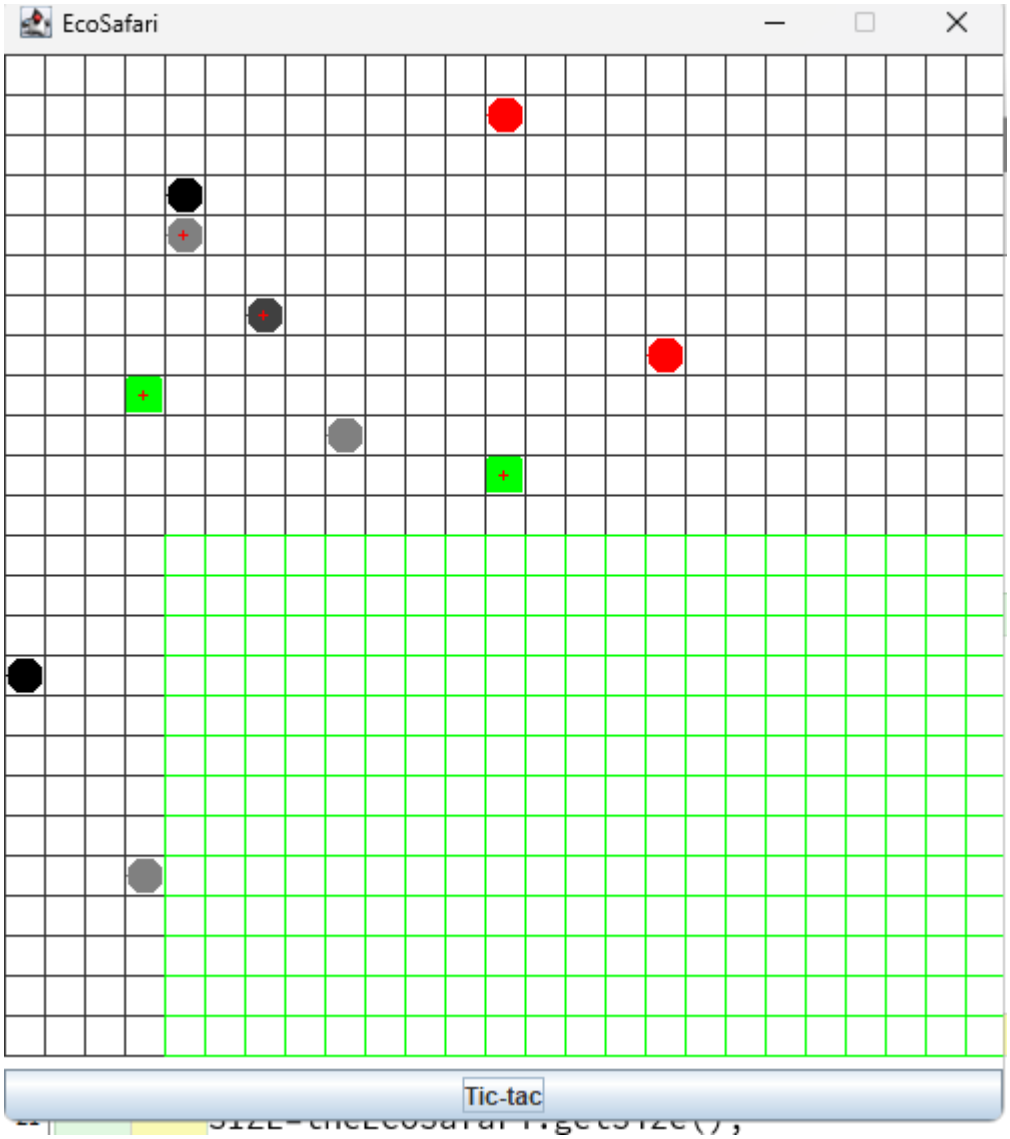


Esto plantea dos cuestiones interesantes:

- Definir que puede estar en esa zona y que no puede estar en esa zona. Sin embargo, el enunciado es muy preciso cuando dice que en la zona solo puede estar:

Tierra, pasto, cebra y león.

Por tanto es necesario hacer dos cosas, cambiar las posiciones de esos objetos que están actualmente. Por tanto en someEntities se pone:


EcoSafari
Tic-tac

```java
/**
 * Populates the EcoSafari with some entities
 */
public void someEntities(){
    //We create dumbo and babar:
    //Dumbo will be in position (5,5), but in a matrix it represents [4][4]
    Elephant dumbo = new Elephant(this, 4,4);

    //Babar will be in position (6,6) but in a matrix it represents [5][5]
    Elephant babar = new Elephant(this, 6,6);

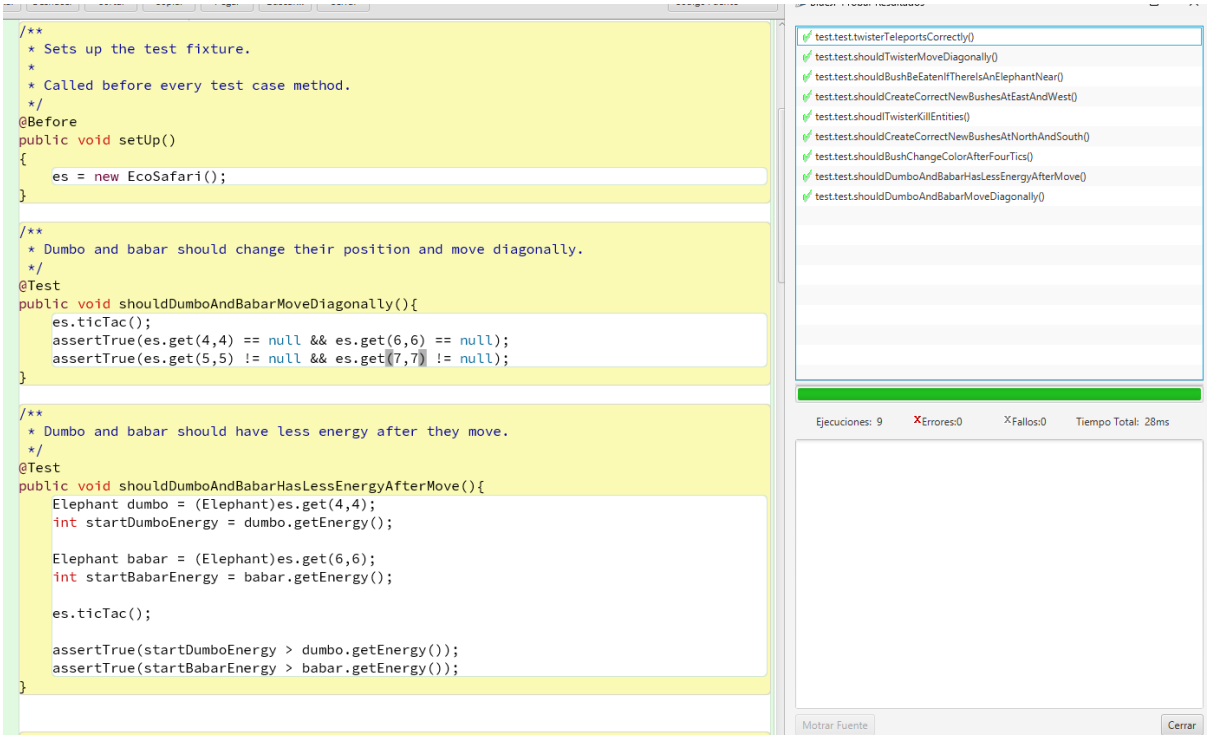
    //We create mopane and acacia.
    Bush mopane = new Bush(this, 10,12,false);
    Bush acacia = new Bush(this, 8,3,false);


    //We put two twisters Thor y Tempest
    Twister thor = new Twister(this, 3,4);
    thor.tac();
    Twister tempest = new Twister(this, 15,0);
    tempest.tac();

    //We create Garzon and Mendivelso in positions (1, 12) and (7, 16)
    AngryElephant Garzon = new AngryElephant(this, 1, 12);
    AngryElephant Mendivelso = new AngryElephant(this, 7, 16);

    //We create david and wilson
    Ovni david= new Ovni(this, 20, 3);
    Ovni wilson = new Ovni(this,9,8);
}
```

Esto también implicó el cambió de las pruebas, puesto que babar no iba a estar en el mismo lugar:



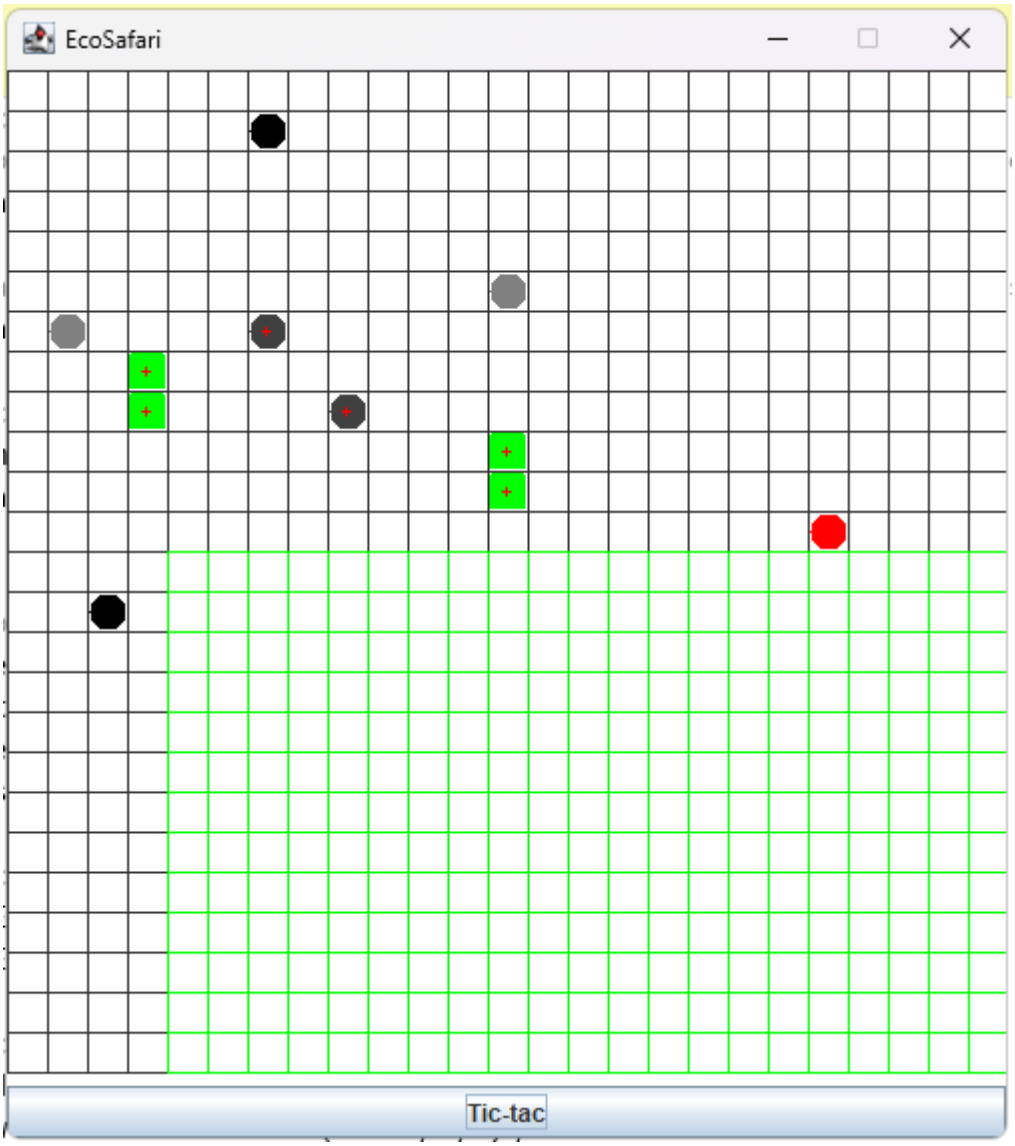
Fue necesario crear una función para verificar si algo está dentro de la zona de pradera:

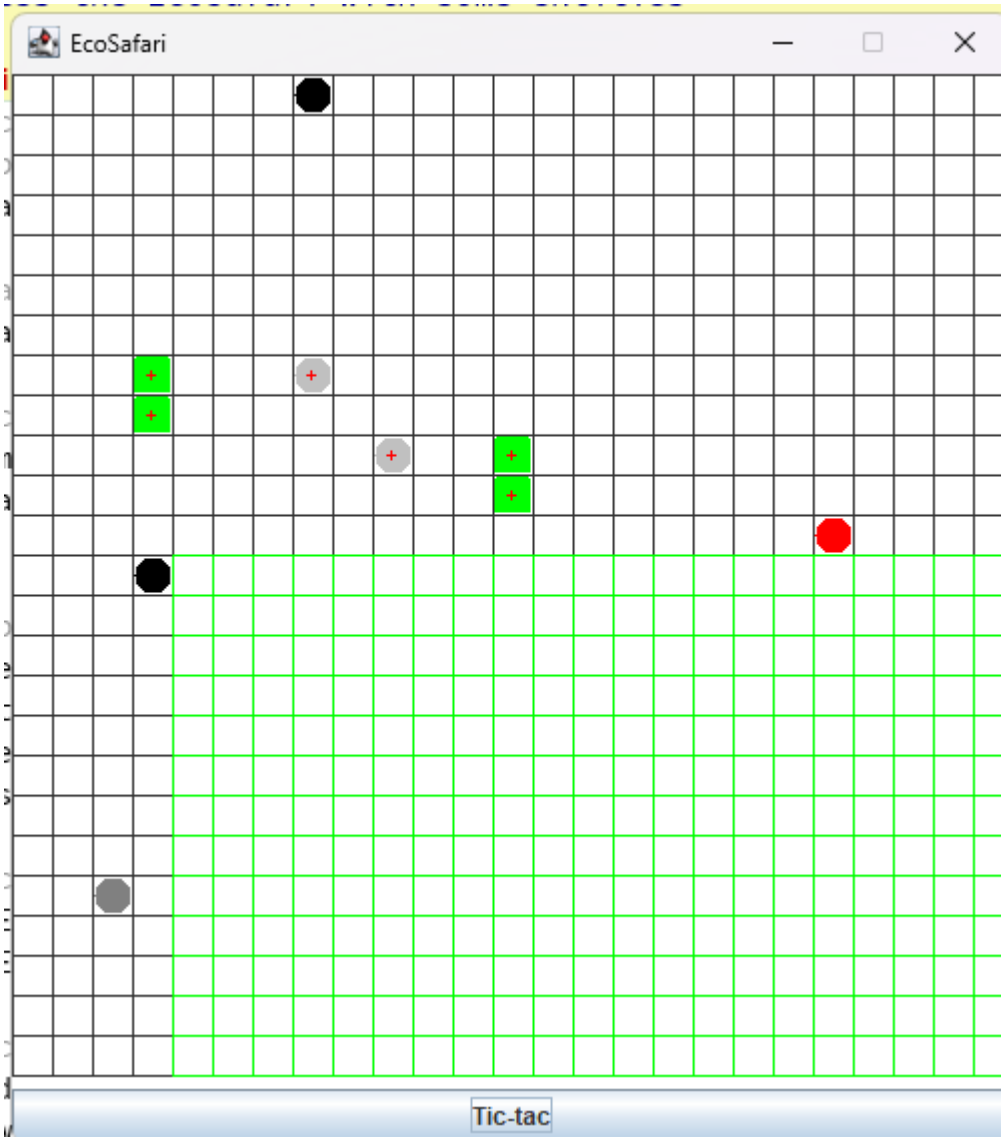
```
/**
 * Determines whether a position is inside the grasslandZone
 * @param r the row
 * @param c the column
 * @return if is inside the zone
 */
public boolean isInsideGrasslandZone(int r, int c){
    return ((12<=r) && (r<SIZE) && (4<=c) && (c<SIZE));
}
```

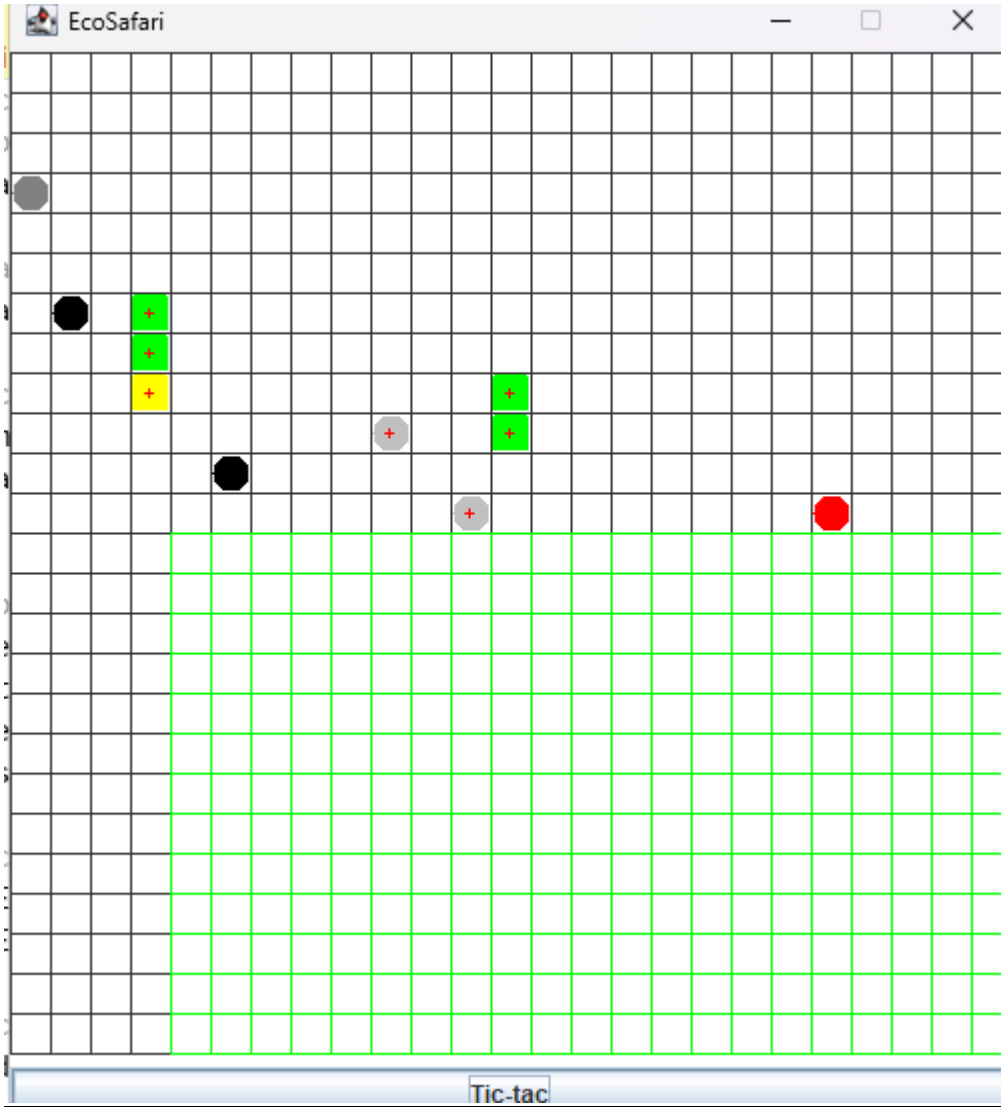
Lo cual hace que sea necesario redefinir varios comportamientos como los son el movimiento de los elefantes:

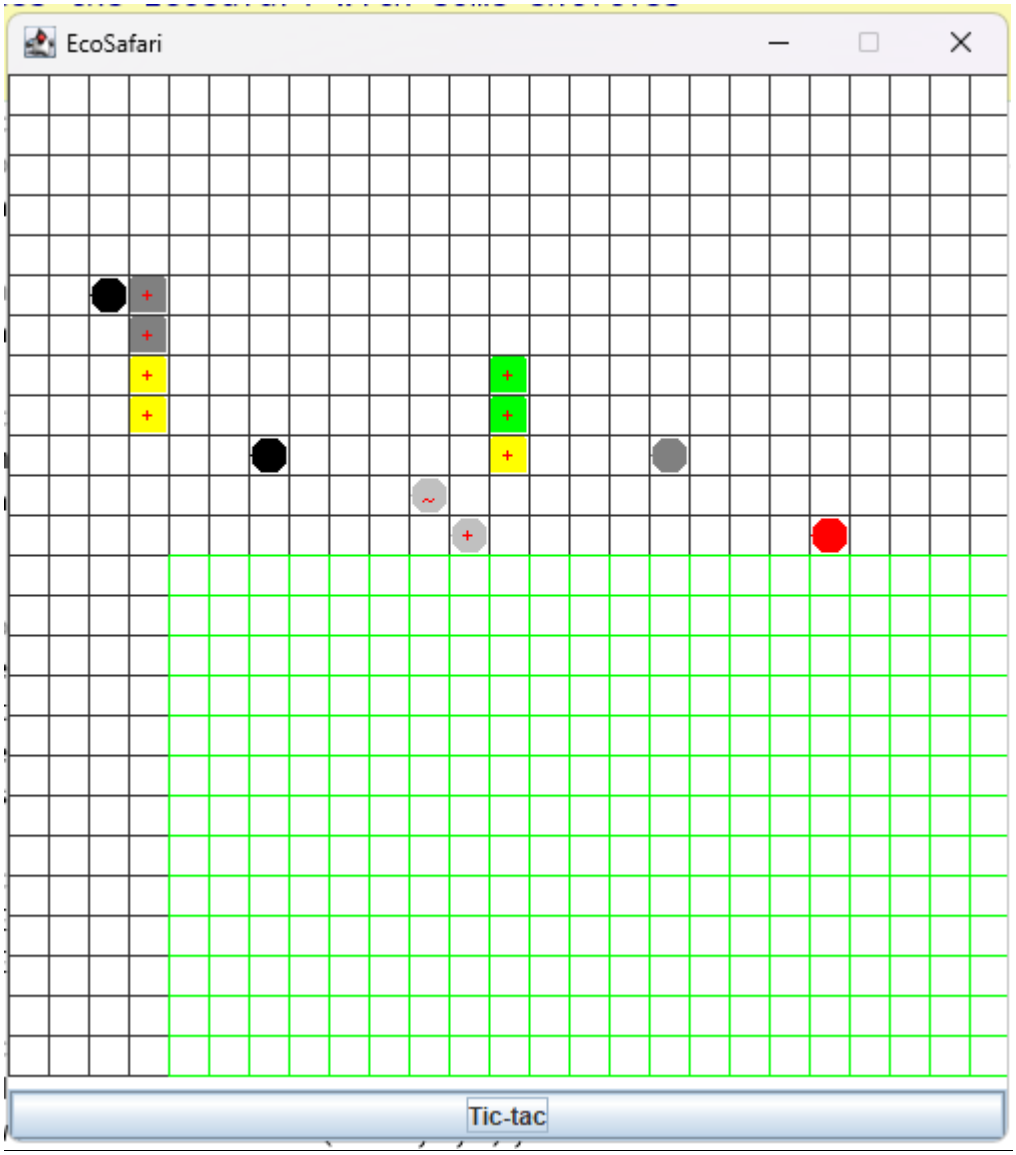
```
@Override
/**
 * Moves the elephant to a specific position, but it must be outside the grassland zone.
 * @param deltaRows are rows quantity it will move
 * @param deltaColumns are columns quantity it will move
 * @return if it could move.
 */
public boolean move(int deltaRows, int deltaColumns){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.isInside(r+deltaRows,c+deltaColumns) && !habitat.isInsideGrasslandZone(r+deltaRows,c+deltaColumns) && habitat.get(r+deltaRows,c+deltaColumns) ==null){
            habitat.set(null,r,c);
            habitat.set(this,r+deltaRows,c+deltaColumns);
            ok=true;
        }
    }
    return ok;
}
```

Ahora revisa que el elefante esté fuera de la zona de pradera, además de que, antes de moverse, confirme que esa zona está vacia, porque podía pasar que ambos elefantes se pasaban encima unos de otros:




EcoSafari
Tic-tac


EcoSafari
Tic-tac



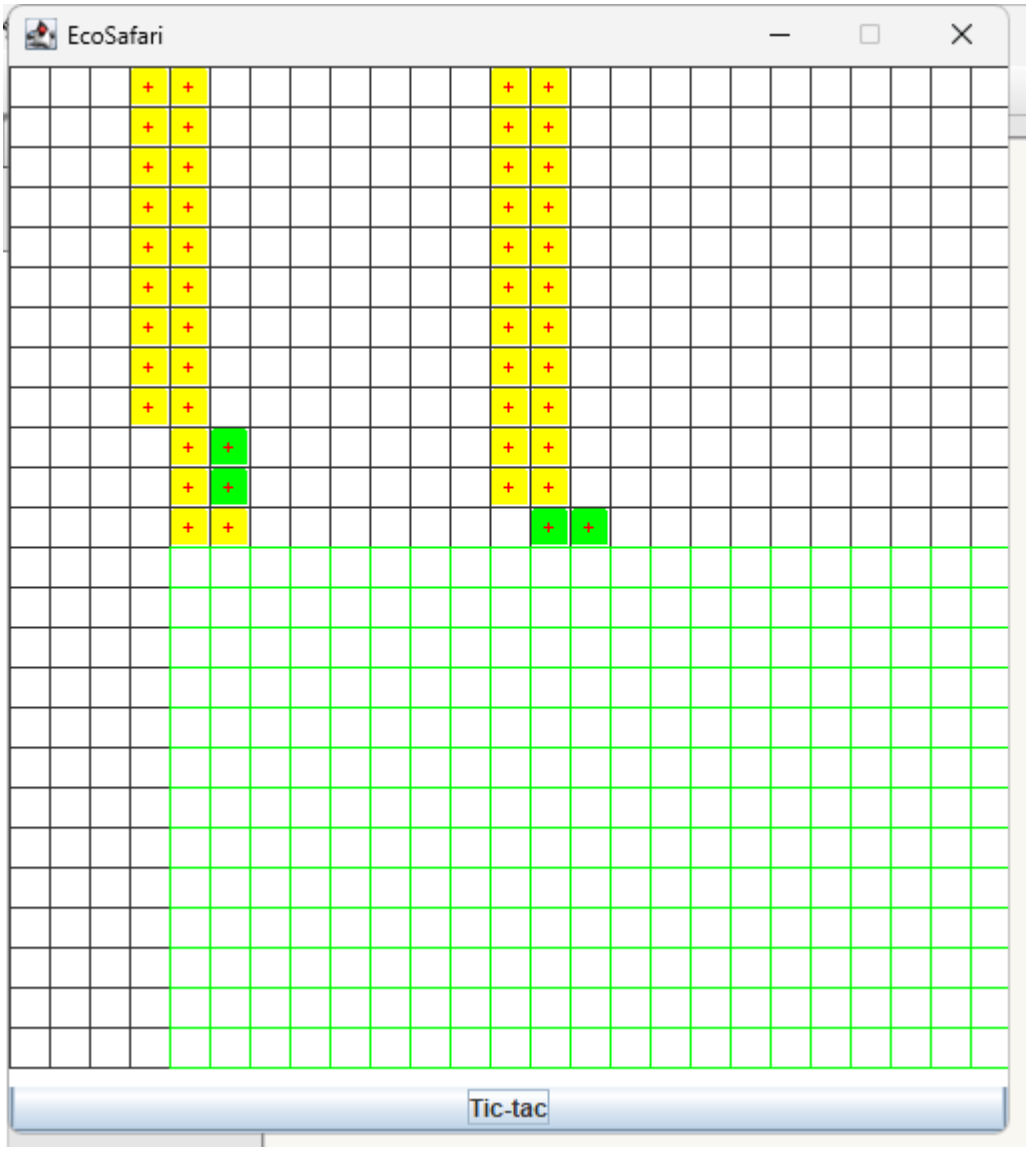
Entonces acá se ve como los elefantes no pasan a la zona segura, y no se pisan unos a otros

Por otro lado, fue necesario hacer el mismo análisis con los arbustos para que no entre a la zona de pradera:

```java
@Override
/**
 * Creates a new bush near to it, if a place is empty, first looks the North, then South, then
 * East and finally West. This just happens if there are two tics. But, it must be outside the grasslandZone.
 * If there are four tics it changes it colors.
 */
public void tic(){
    if(!recentlyCreatedByBush){
        tics++;
    }

    if(tics ==2){
        int[] position = habitat.find(this);
        if(habitat.get(position[0]-1, position[1]) == null && habitat.isInside(position[0]-1, position[1]) && !habitat.isInsideGrasslandZone(position[0]-1, position[1])){
            new Bush(habitat, position[0]-1, position[1],true);
        }
        else if(habitat.get(position[0]+1, position[1]) == null && habitat.isInside(position[0]+1, position[1]) && !habitat.isInsideGrasslandZone(position[0]+1, position[1])){
            new Bush(habitat, position[0]+1, position[1],true);
        }
        else if(habitat.get(position[0], position[1]+1) == null && habitat.isInside(position[0], position[1]+1) && !habitat.isInsideGrasslandZone(position[0], position[1]+1)){
            new Bush(habitat, position[0], position[1]+1, true);
        }
        else if(habitat.get(position[0], position[1]-1) == null && habitat.isInside(position[0], position[1]-1) && !habitat.isInsideGrasslandZone(position[0], position[1]-1)){
            new Bush(habitat, position[0], position[1]-1,true);
        }
    }
}
```

Para demostrar que el arbusto no se pasa:



Fue necesario cambiar algunas cosas de AngryElephant para que no entre en la zona segura:



```
    @Override
    /**
     * An Entity can move, by default it moves diagonally
     * @return if it was moved.
     */
    public boolean move(int deltaRows, int deltaColumns){
        int [] position=getHabitat().find(this);
        EcoSafari habitat=getHabitat();
        boolean ok=false;
        if (position!=null){
            int r = position[0];
            int c = position[1];
            if (habitat.isInside(r+deltaRows,c+deltaColumns) && !(habitat.get(r+deltaRows,c+deltaColumns) instanceof Elephant)&& habitat.get(r+deltaRows,c+deltaColumns) ==null && !habitat.isInsid
                habitat.set(null,r,c);
                habitat.set(this,r+deltaRows,c+deltaColumns);
                ok=true;
            }else if(habitat.isInside(r+1, c+1) && habitat.get(r+1,c+1) ==null && !habitat.isInsideGrasslandZone(r+1,c+1)){
                habitat.set(null,r,c);
                habitat.set(this,r+1,c+1);
                killElephant();
                ok=true;
            }
        }
        return ok;
    }
}
```

Donde se mira que no pueda entrar a la zona segura.

Lo mismo en el ovni es necesario evitar que se pueda teletransportar a la zona segura:



```
@Override
/**
 * An Ovni can move
 * @return if it was moved.
 */
public boolean move(int newRow, int newColumn){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.get(newRow,newColumn)==null && !habitat.isInsideGrasslandZone(newRow,newColumn) && habitat.isInside(newRow, newColumn)){
            habitat.set(null,r,c);
            habitat.set(this,newRow,newColumn);
            ok=true;
        }
    }
    return ok;
}
```

Además toca cambiar varias de las pruebas, puesto que algunas tenían los objetos ahí adentro, por tanto para evitar cualquier conflicto:

```java
/**
 * Should create correct new bushes at north and south
 */
@Test
public void shouldCreateCorrectNewBushesAtNorthAndSouth(){
    Bush bush1 = new Bush(es,0,0,false); //Should create a new bush at south
    Bush bush2= new Bush(es,1,1,false); //Should create a new bush at north

    es.ticTac();
    es.ticTac();

    assertTrue(es.get(0,1) != null);
    assertTrue(es.get(1,0) != null);
}
```

```java
/**
 * Should create correct new bushes at east and west
 */
@Test
public void shouldCreateCorrectNewBushesAtEastAndWest(){
    Bush bush1 = new Bush(es,0,0,false);
    Bush bush2= new Bush(es,0,24,false);

    es.ticTac();

    Bush bush3 = new Bush(es,1,0,false); //Now bush1 should create a new bush at east.
    Bush bush4 = new Bush(es,0,23,false); //Now bus2 should create a new bush at west.
    es.ticTac();

    assertTrue((es.get(0,1) instanceof Bush));
    assertTrue((es.get(1,24) instanceof Bush));
}
```

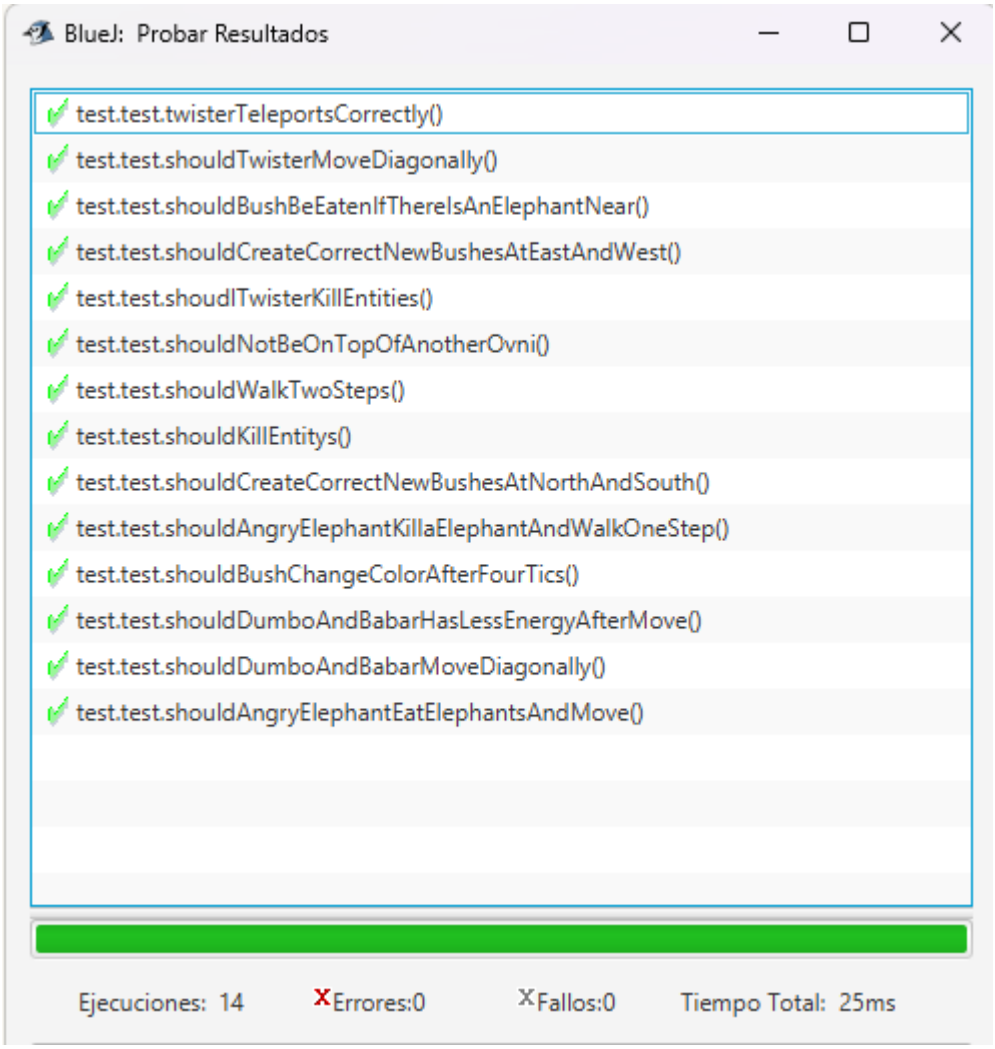
```java
/**
 * Angry Elephant eats the elephants surrounding it and moves two spaces diagonally downwards.
 */
@Test
public void shouldAngryElephantEatElephantsAndMove(){
    AngryElephant Mendivelso = new AngryElephant(es, 7, 16);

    Elephant e1 = new Elephant(es, 7, 17);
    Elephant e2 = new Elephant(es, 8, 15);
    es.ticTac();

    assertEquals(es.get( 7, 17), null);
    assertEquals(es.get( 8, 15), null);

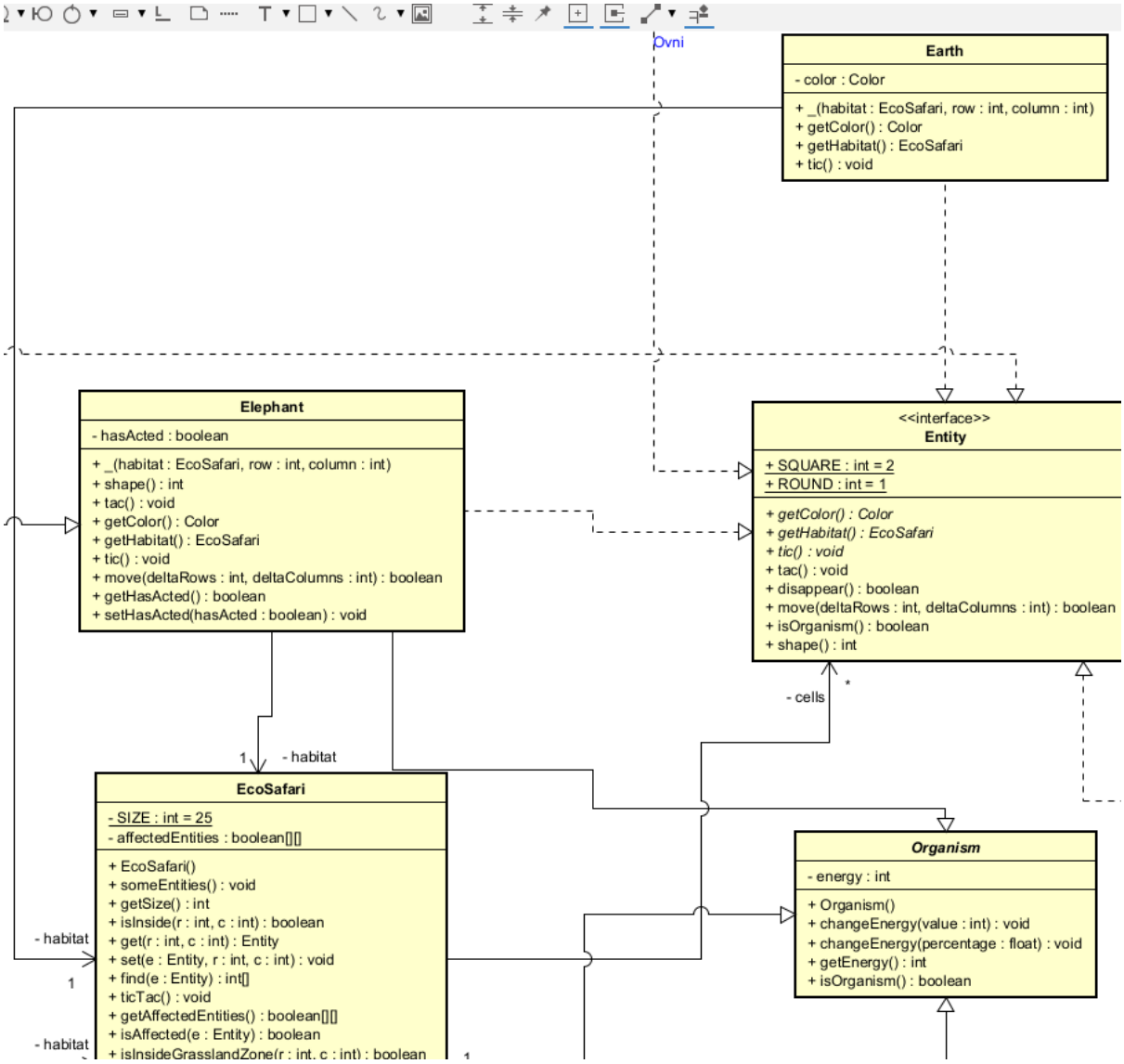
    assertEquals(es.get(9,18), Mendivelso);
}
```

Por tanto, hasta este punto se ve como pasan todas las pruebas:



Ciclo1. Crear la tierra

El diagrama de clases se le agrega esta nueva clase:



Se definirán algunas pruebas, enfocadas a ver que si una entidad desaparece, entonces se volverá en tierra:

```
@Before
public void setUp()
{
    es = new EcoSafari();
    for(int i=0; i<25; i++){
        for(int j=0; j<25; j++){
            new Earth(es, i,j);
        }
    }
}
```

```java
/**
 * If a bush dessapears by an elephant it will transform in earth
 */
@Test
public void shouldEntityBushDessapearsItWillBecomeEarth(){
    Bush bush = new Bush(es,3,15,false);
    Elephant elephant = new Elephant(es,1,13);

    es.ticTac();
    assertTrue(es.get(3,15) instanceof Earth);
}

/**
 * If an elephant walks in the place he were there will be earth
 */
@Test
public void shouldElephantWalksThePlaceHeWereWillBeEarth(){
    Elephant elephant = new Elephant(es,1,13);
    es.ticTac();

    assertTrue(es.get(1,13)instanceof Earth);
}

/**
 * If a twister kills an elephant the place the elephant were will be earth
 */
@Test
public void shouldElephantsDieThePlaceHeWereWillBeEarth(){
    Elephant elephant = new Elephant(es,1,13);
    Twister twister1 = new Twister(es,3,13);
    es.ticTac();
    es.ticTac();

    assertTrue(es.get(2,14)instanceof Earth);
}
```

Creando la clase Earth:

```java
package domain;
import java.awt.Color;


/**
 * Represents Earth that implements Entity<br>
 * <b>(color, habitat)</b>
 */
public class Earth implements Entity
{
    private Color color;
    private EcoSafari habitat;

    /**
     * Constructs a new Earth.
     * @param habitat is Earth's habitat.
     * @param row is the row position.
     * @param column is the column position.
     */
    public Earth(EcoSafari habitat,int row, int column){
        this.habitat=habitat;
        habitat.set((Entity)this, row, column);
        color = new Color(181, 138, 99);
    }


    @Override
    /**
     * Gives the Earth's habitat.
     * @return Earth's habitat.
     */
    public EcoSafari getHabitat(){
        return habitat;
    }
```

```java
@Override
/**
 * Gives Earth's color depending its energy.
 * @return Earth's color.
 */
public final Color getColor(){
    return color;
}
```

```java
@Override
/**
 * Reacts to tic
 */
public void tic(){
    //En el futuro, cuando ya exista la clase del pasto, la tierra tendrá un 10% de probabilidades de crear pasto
}
```

Fue necesario cambiar muchas partes del código que antes funcionaban con nulos, porque ahora cuando iniciamos EcoSafari se rellena el mapa con tierra.

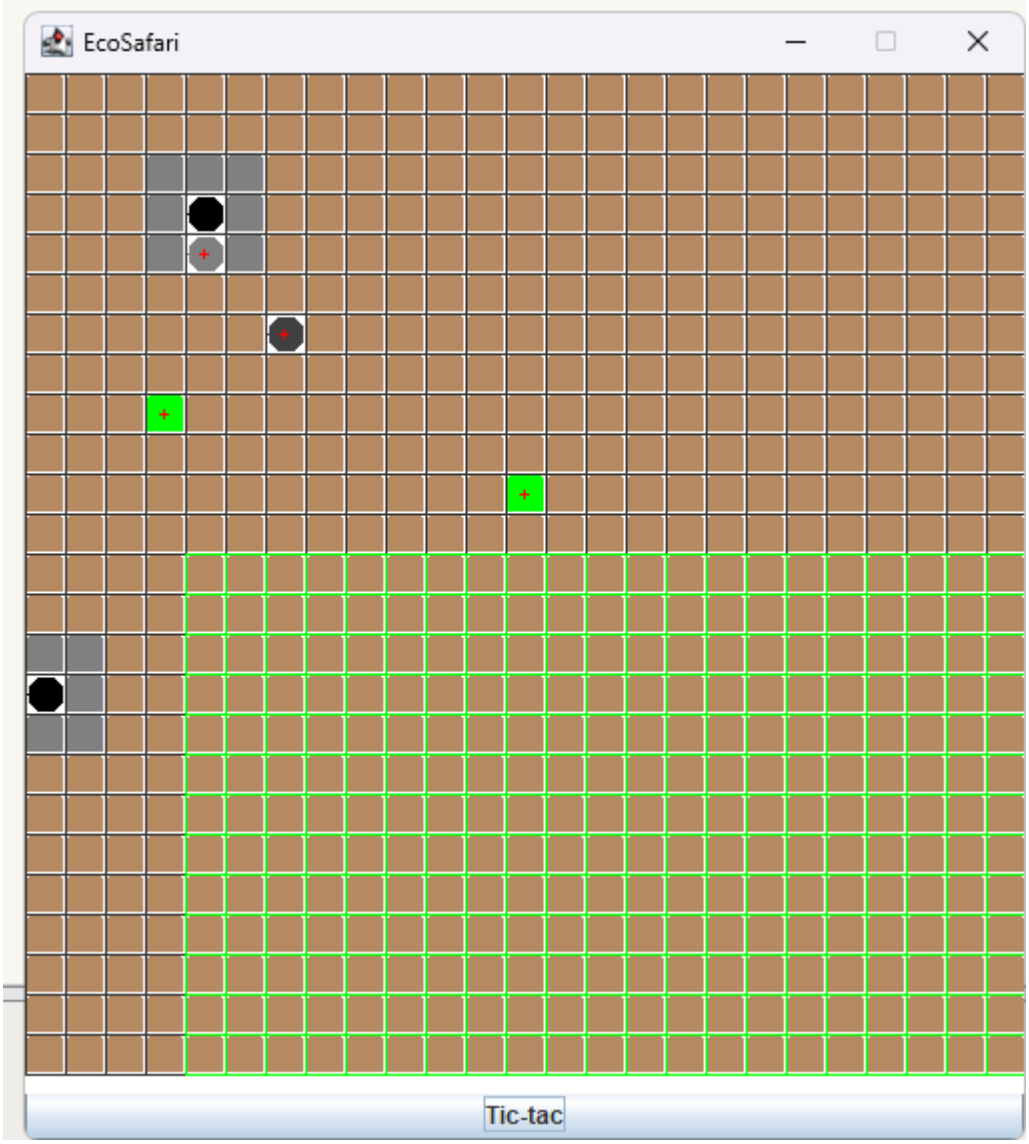
```java
/**
 * Constructs a new EcoSafari
 */
public EcoSafari() {
    cells=new Entity[SIZE][SIZE];
    affectedEntities = new boolean[SIZE][SIZE];
    placeEarth();
    someEntities();
}

/**
 * Puts all the EcoSafari with earth.
 */
public void placeEarth(){
    for(int i = 0; i < SIZE; i++){
        for(int j = 0; j < SIZE; j++){
            new Earth(this, i,j);
        }
    }
}
```

Además para que los tornados inicien con sombra:

```java
    Twister thor = new Twister(this, 3,4);
    thor.tac();
    Twister tempest = new Twister(this, 15,0);
    tempest.tac();
```

Se hace tac() para que revise las zonas afectadas y luego haga sombra con ella:



Fue necesario cambiar varias cosas del movimiento, para que el Elephant se mueva correctamente en la tierra.

```
@Override
/**
 * Moves the elephant to a specific position, but it must be outside the grassland zone.
 * @param deltaRows are rows quantity it will move
 * @param deltaColumns are columns quantity it will move
 * @return if it could move.
 */
public boolean move(int deltaRows, int deltaColumns){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.isInside(r+deltaRows,c+deltaColumns) && !habitat.isInsideGrasslandZone(r+deltaRows,c+deltaColumns) && habitat.get(r+deltaRows,c+deltaColumns)instanceof Earth){
            habitat.set(new Earth(getHabitat(), r, c),r,c);
            habitat.set(this,r+deltaRows,c+deltaColumns);
            ok=true;
        }
    }
    return ok;
}
```

Lo mismo con los arbustos:

```java
@Override
/**
 * Creates a new bush near to it, if a place is empty, first looks the North, then South, then
 * East and finally West. This just happens if there are two tics. But, it must be outside the grasslandZone.
 * If there are four tics it changes it colors.
 */
public void tic(){
    if(!recentlyCreatedByBush){
        tics++;
    }

    if(tics ==2){
        int[] position = habitat.find(this);
        if(habitat.get(position[0]-1, position[1]) instanceof Earth && habitat.isInside(position[0]-1, position[1]) && !habitat.isInsideGrasslandZone(position[0]-1, position[1])){
            new Bush(habitat, position[0]-1, position[1],true);
        }
        else if(habitat.get(position[0]+1, position[1]) instanceof Earth && habitat.isInside(position[0]+1, position[1]) && !habitat.isInsideGrasslandZone(position[0]+1, position[1])){
            new Bush(habitat, position[0]+1, position[1],true);
        }
        else if(habitat.get(position[0], position[1]+1) instanceof Earth && habitat.isInside(position[0], position[1]+1) && !habitat.isInsideGrasslandZone(position[0], position[1]+1)){
            new Bush(habitat, position[0], position[1]+1, true);
        }
        else if(habitat.get(position[0], position[1]-1) instanceof Earth && habitat.isInside(position[0], position[1]-1) && !habitat.isInsideGrasslandZone(position[0], position[1]-1)){
            new Bush(habitat, position[0], position[1]-1,true);
        }
    }
}
```

Lo mismo con Entity:

```java
/**
 * An Entity can move, by default it moves diagonally
 * @param deltaRows are rows quantity it will move
 * @param deltaColumns are columns quantity it will move
 * @return if it could move.
 */
public default  boolean move(int deltaRows, int deltaColumns){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.isInside(r+deltaRows,c+deltaColumns)){
            habitat.set(null,r,c);
            new Earth(getHabitat(), r, c);
            habitat.set(this,r+deltaRows,c+deltaColumns);
            ok=true;
        }
    }
    return ok;
}
```

Lo mismo con AngryElephant:



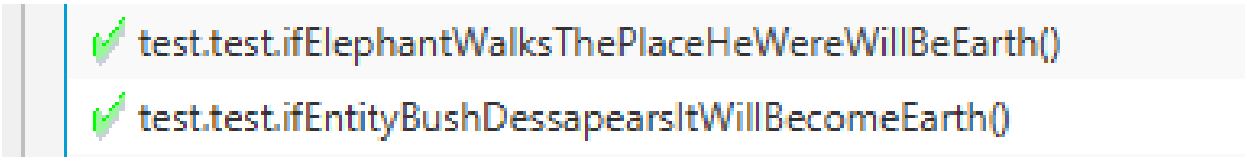
```
@Override
/**
 * An Entity can move, by default it moves diagonally
 * @return if it was moved.
 */
public boolean move(int deltaRows, int deltaColumns){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.isInside(r+deltaRows,c+deltaColumns) && habitat.get(r+deltaRows,c+deltaColumns) instanceof Earth && !habitat.isInsideGrasslandZone(r+deltaRows,c+deltaColumns) && this.getEnergy
            new Earth(getHabitat(), r, c);
            habitat.set(this,r+deltaRows,c+deltaColumns);
            ok=true;
        }else if(habitat.isInside(r+1, c+1) && habitat.get(r+1,c+1) instanceof Earth && !habitat.isInsideGrasslandZone(r+1,c+1)){
            new Earth(getHabitat(), r, c);
            habitat.set(this,r+1,c+1);
            killElephant();
            ok=true;
        }
    }
    return ok;
}
```

Lo mismo con Ovni:

```
@Override
/**
 * An Ovni can move
 * @return if it was moved.
 */
public boolean move(int newRow, int newColumn){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.get(newRow,newColumn)instanceof Earth && !habitat.isInsideGrasslandZone(newRow,newColumn) && habitat.isInside(newRow, newColumn)){
            new Earth(getHabitat(), r, c);
            habitat.set(this,newRow,newColumn);
            ok=true;
        }
    }
    return ok;
}
```

De este modo, todo sigue funcionando de modo que sea capaz de poner tierra en zonas del mapa.

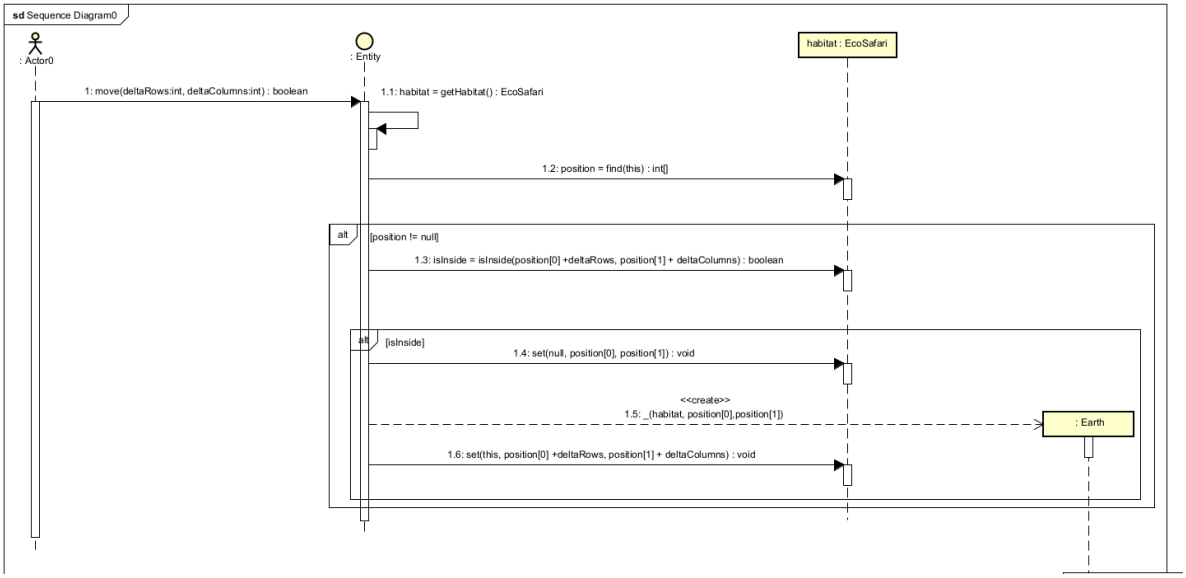
Por último se adjunta como pasó las pruebas:

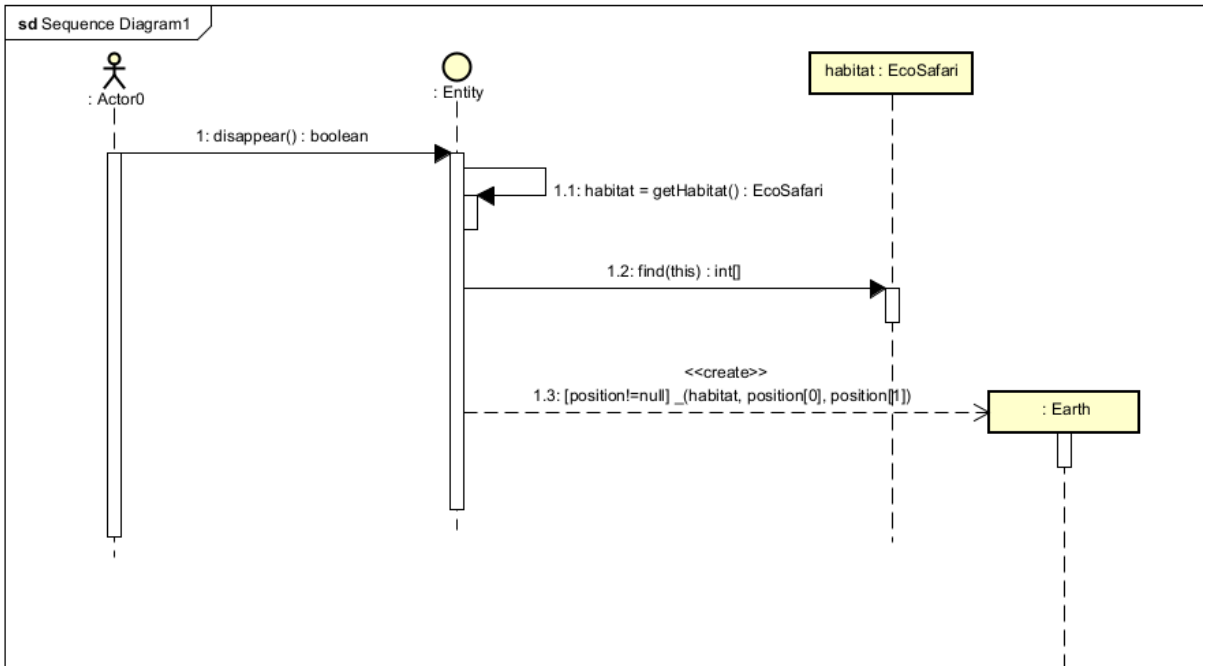


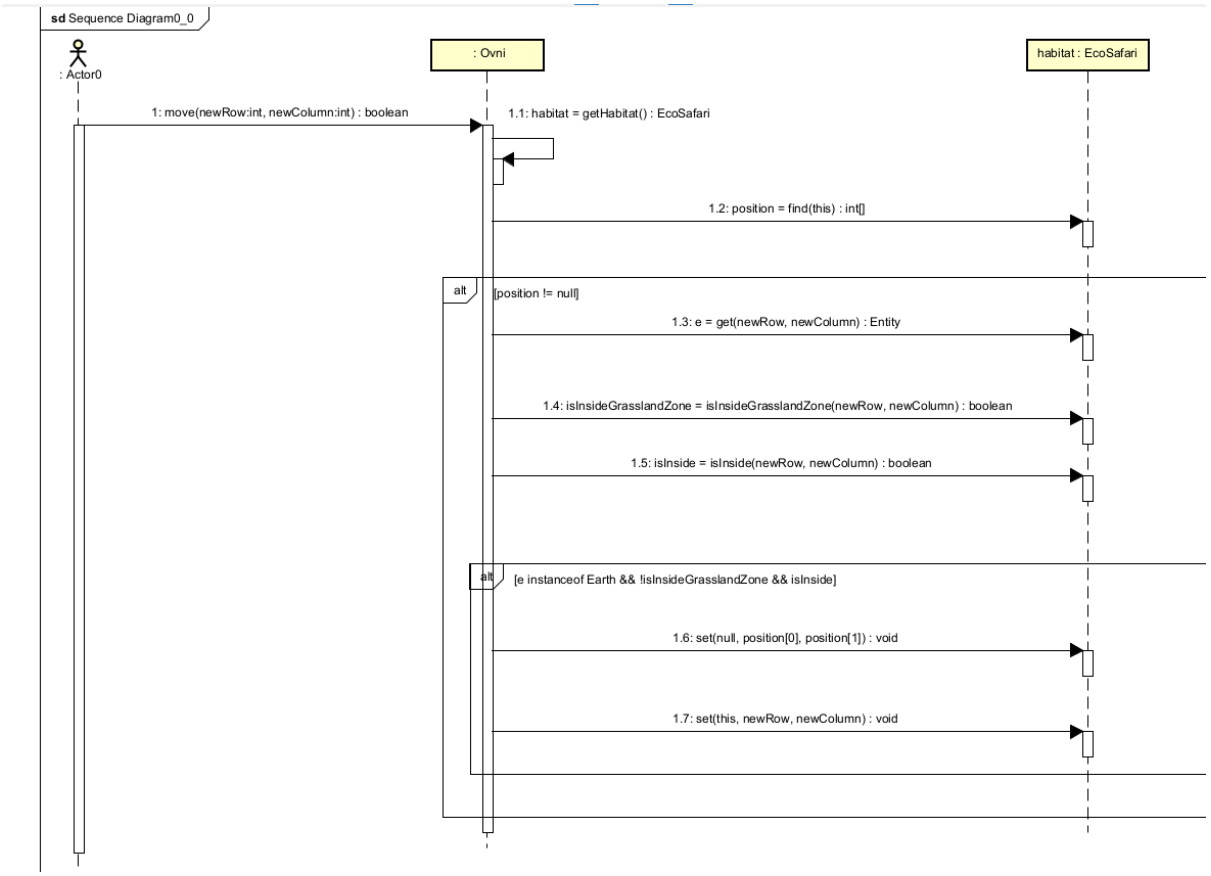


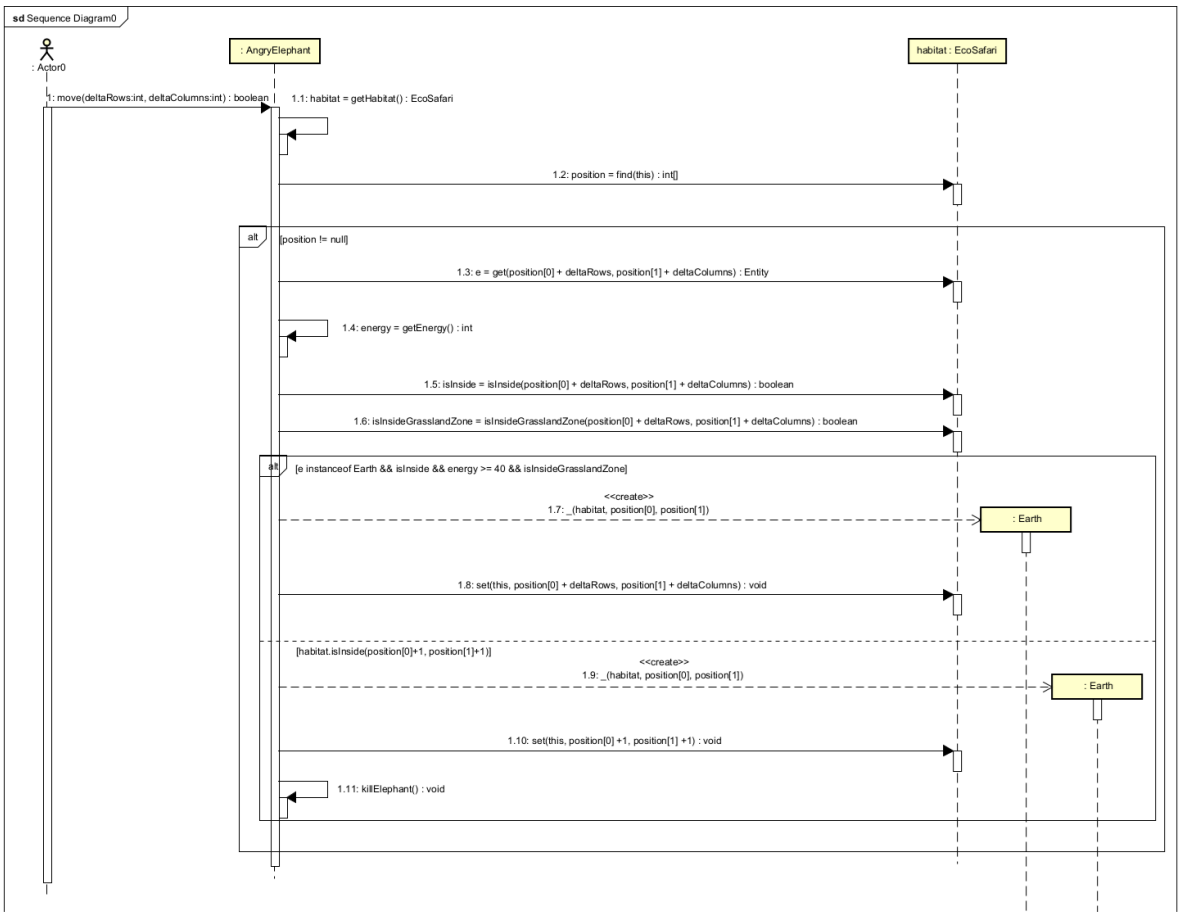
Además, al tener en cuenta la tierra fue necesario crear/implementar/cambiar algunos diagramas de secuencia para que tuvieran en cuenta este tema de la creación de Earth en vez de poner un null:

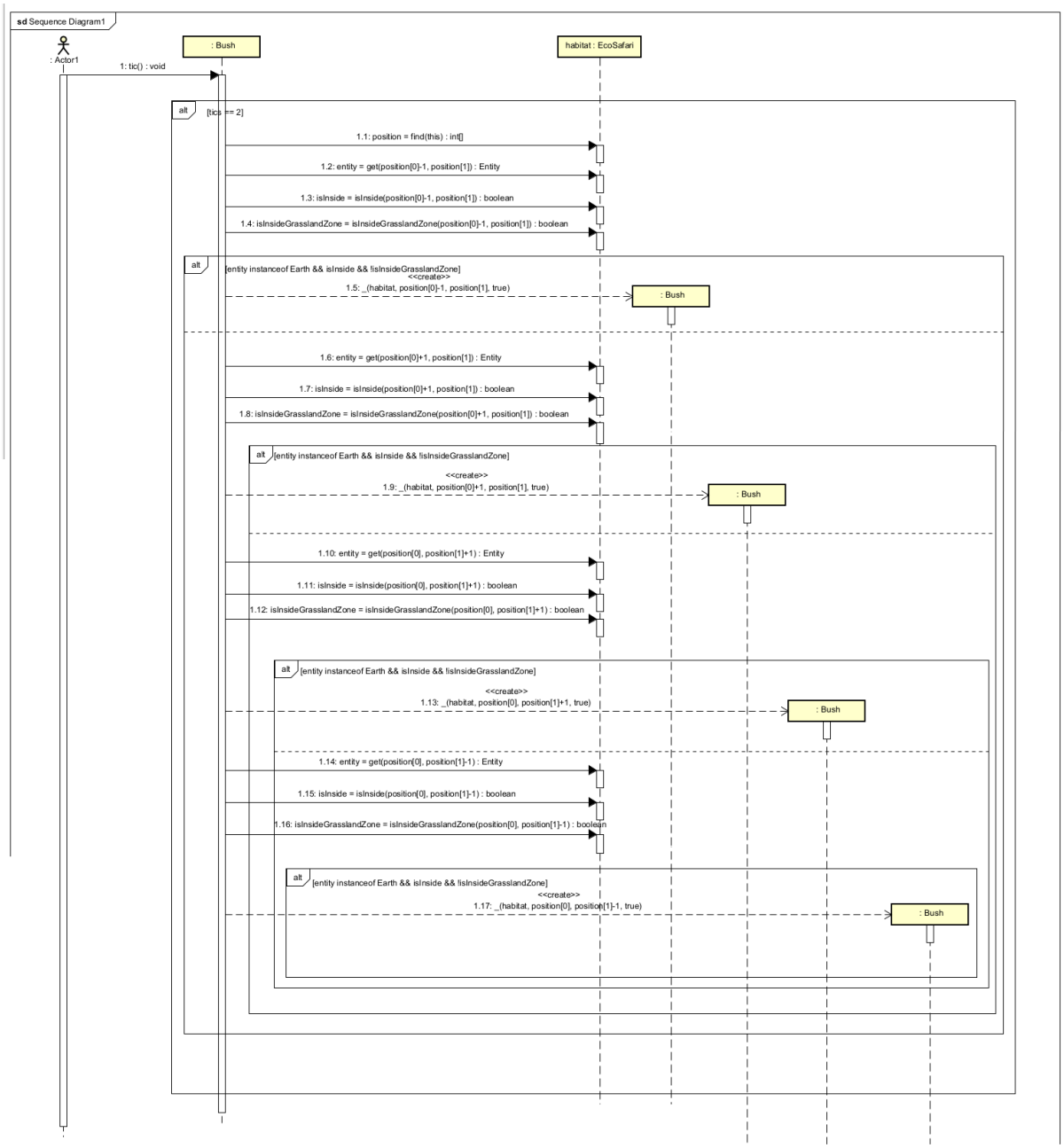
En este caso los de Entity son:


sd Sequence Diagram0
: Actor0
: Entity
habitat : EcoSafari
1: move(deltaRows:int, deltaColumns:int) : boolean
1.1: habitat = getHabitat() : EcoSafari
1.2: position = find(this) : int[]
alt
[position != null]
1.3: isInside = isInside(position[0] +deltaRows, position[1] + deltaColumns) : boolean
alt
[isInside]
1.4: set(null, position[0], position[1]) : void
<<create>>
1.5: _(habitat, position[0],position[1])
: Earth
1.6: set(this, position[0] +deltaRows, position[1] + deltaColumns) : void


sd Sequence Diagram1
: Actor0
: Entity
habitat : EcoSafari
1: disappear() : boolean
1.1: habitat = getHabitat() : EcoSafari
1.2: find(this) : int[]
<<create>>
1.3: [position!=null] _(habitat, position[0], position[1])
: Earth

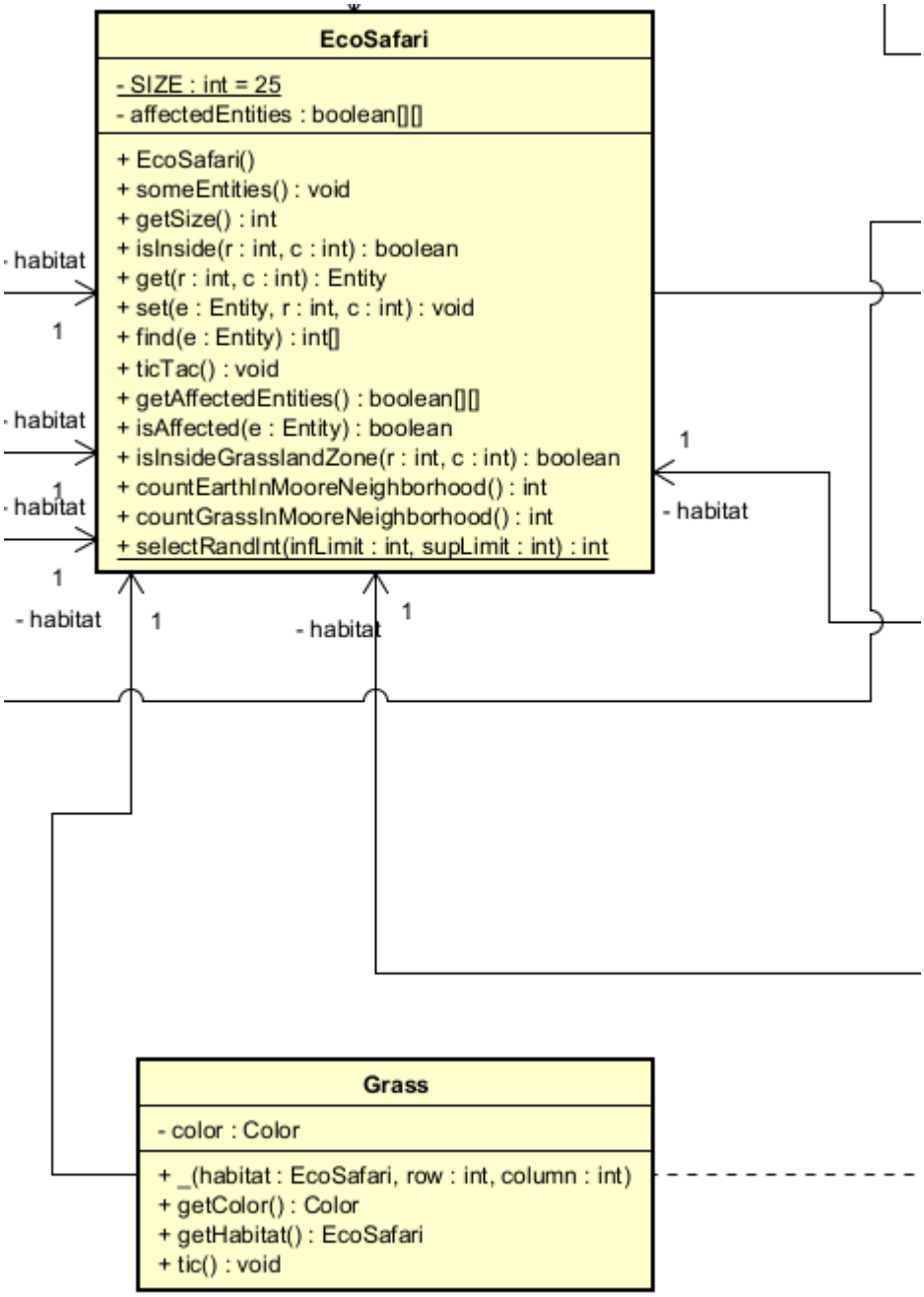

sd Sequence Diagram0_0
: Actor0
: Ovni
habitat : EcoSafari
1: move(newRow:int, newColumn:int) : boolean
1.1: habitat = getHabitat() : EcoSafari
1.2: position = find(this) : int[]
alt
[position != null]
1.3: e = get(newRow, newColumn) : Entity
1.4: isInsideGrasslandZone = isInsideGrasslandZone(newRow, newColumn) : boolean
1.5: isInside = isInside(newRow, newColumn) : boolean
alt
[e instanceof Earth && !isInsideGrasslandZone && isInside]
1.6: set(null, position[0], position[1]) : void
1.7: set(this, newRow, newColumn) : void

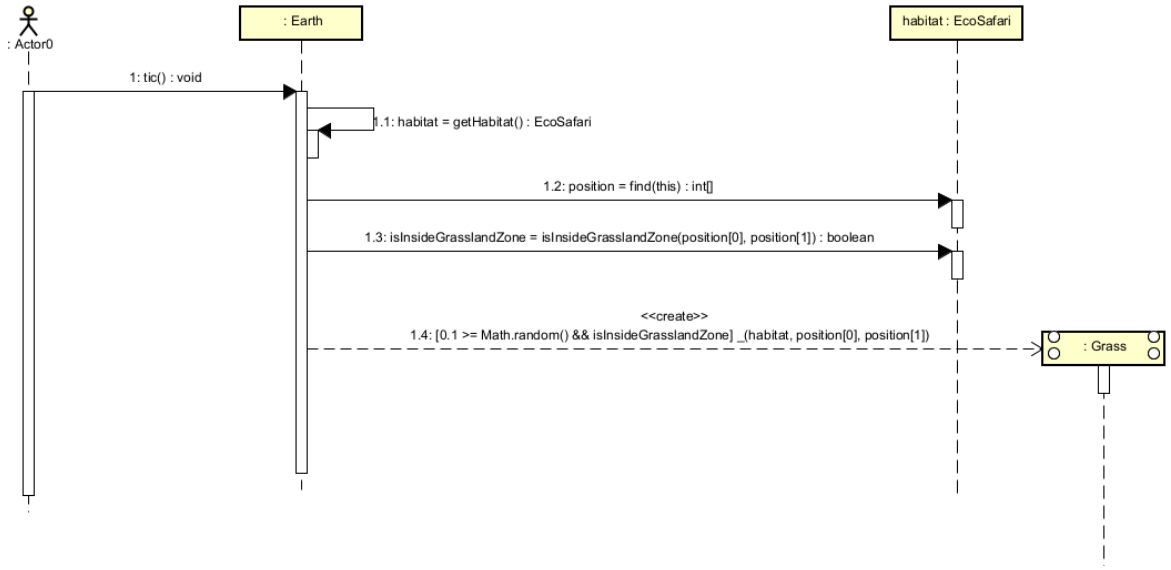

sd Sequence Diagram0
: Actor0
: AngryElephant
habitat : EcoSafari
1: move(deltaRows:int, deltaColumns:int) : boolean
1.1: habitat = getHabitat() : EcoSafari
1.2: position = find(this) : int[]
alt
[position != null]
1.3: e = get(position[0] + deltaRows, position[1] + deltaColumns) : Entity
1.4: energy = getEnergy() : int
1.5: isInside = isInside(position[0] + deltaRows, position[1] + deltaColumns) : boolean
1.6: isInsideGrasslandZone = isInsideGrasslandZone(position[0] + deltaRows, position[1] + deltaColumns) : boolean
alt
[e instanceof Earth && isInside && energy >= 40 && isInsideGrasslandZone]
<<create>>
1.7: _(habitat, position[0], position[1])
: Earth
1.8: set(this, position[0] + deltaRows, position[1] + deltaColumns) : void
[habitat.isInside(position[0]+1, position[1]+1)]
<<create>>
1.9: _(habitat, position[0], position[1])
: Earth
1.10: set(this, position[0] +1, position[1] +1) : void
1.11: killElephant() : void



2. Crear pasto y la tierra genera pasto con un 10% de probabilidad

El diagrama de clases para la clase de pasto será:


EcoSafari
- SIZE : int = 25
- affectedEntities : boolean[][]
+ EcoSafari()
+ someEntities() : void
+ getSize() : int
+ isInside(r : int, c : int) : boolean
+ get(r : int, c : int) : Entity
+ set(e : Entity, r : int, c : int) : void
+ find(e : Entity) : int[]
+ ticTac() : void
+ getAffectedEntities() : boolean[][]
+ isAffected(e : Entity) : boolean
+ isInsideGrasslandZone(r : int, c : int) : boolean
+ countEarthInMooreNeighborhood() : int
+ countGrassInMooreNeighborhood() : int
+ selectRandInt(infLimit : int, supLimit : int) : int
- habitat
1
- habitat
1
- habitat
1
- habitat
1
- habitat
1
1
- habitat
Grass
- color : Color
+ _(habitat : EcoSafari, row : int, column : int)
+ getColor() : Color
+ getHabitat() : EcoSafari
+ tic() : void



En este último se enfoca en que cuando haga tic() exista la posibilidad de generar pasto.

Grass × Earth ×

Compilar | Deshacer | Cortar | Copiar | Pegar | Buscar... | Cerrar | Código Fuente

```java
package domain;
import java.awt.Color;


/**
 * Represents Grass that implements Entity<br>
 * <b>(color, habitat)</b>
 */
public class Grass implements Entity
{
    private Color color;
    private EcoSafari habitat;

    /**
     * Constructs a new Grass.
     * @param habitat is Grass' habitat.
     * @param row is the row position.
     * @param column is the column position.
     */
    public Grass(EcoSafari habitat,int row, int column){
        this.habitat=habitat;
        habitat.set((Entity)this, row, column);
        color = Color.GREEN;
    }

    @Override
    /**
     * Gives the Grass' habitat.
     * @return Grass' habitat.
     */
    public EcoSafari getHabitat(){
        return habitat;
    }
```

```java
@Override
/**
 * Gives Grass' color depending its energy.
 * @return Grass' color.
 */
public final Color getColor(){
    return color;
}

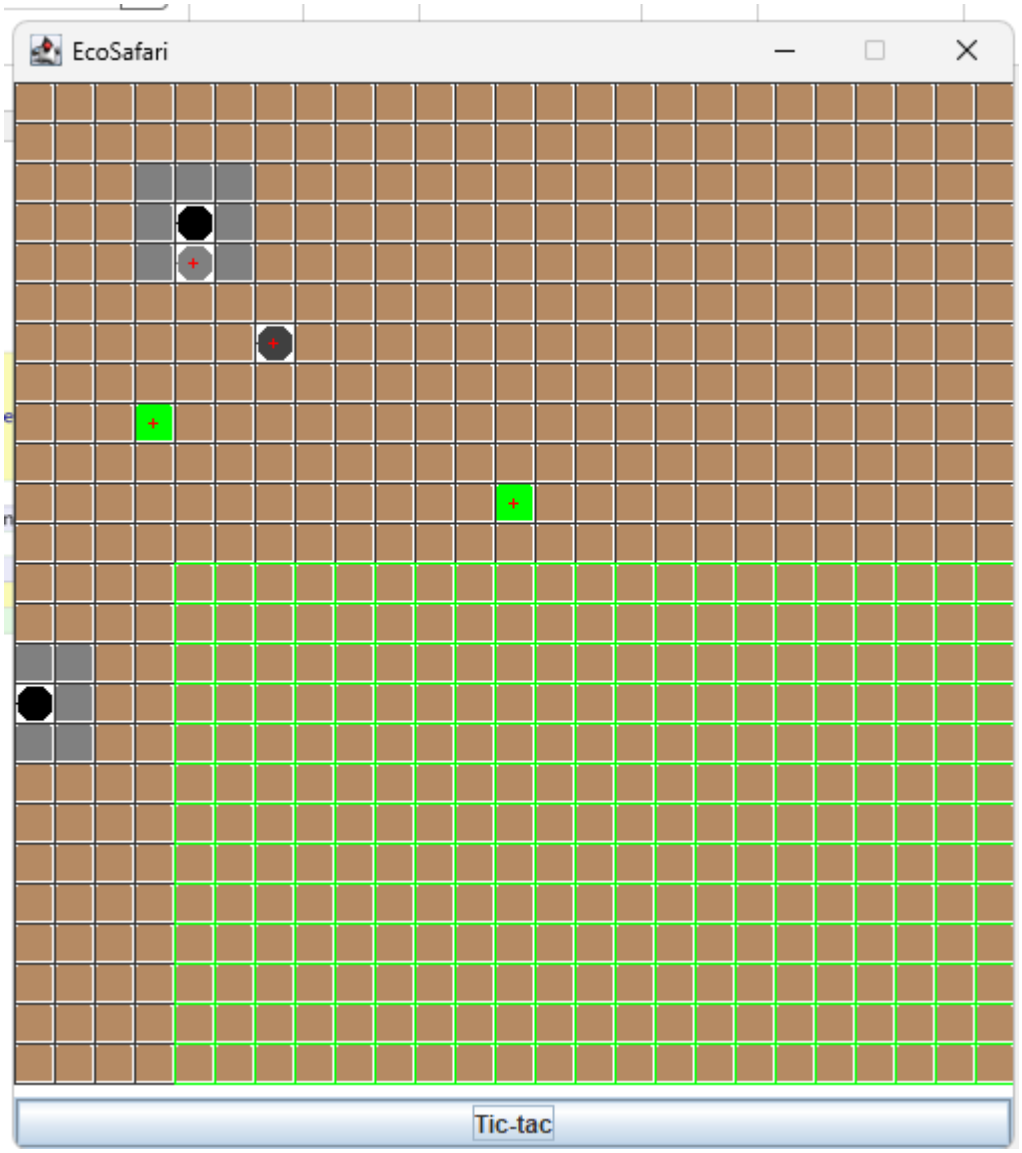
@Override
/**
 * Reacts to tic
 */
public void tic(){
}
```

Ahora es necesario implementar el método faltante de Earth:

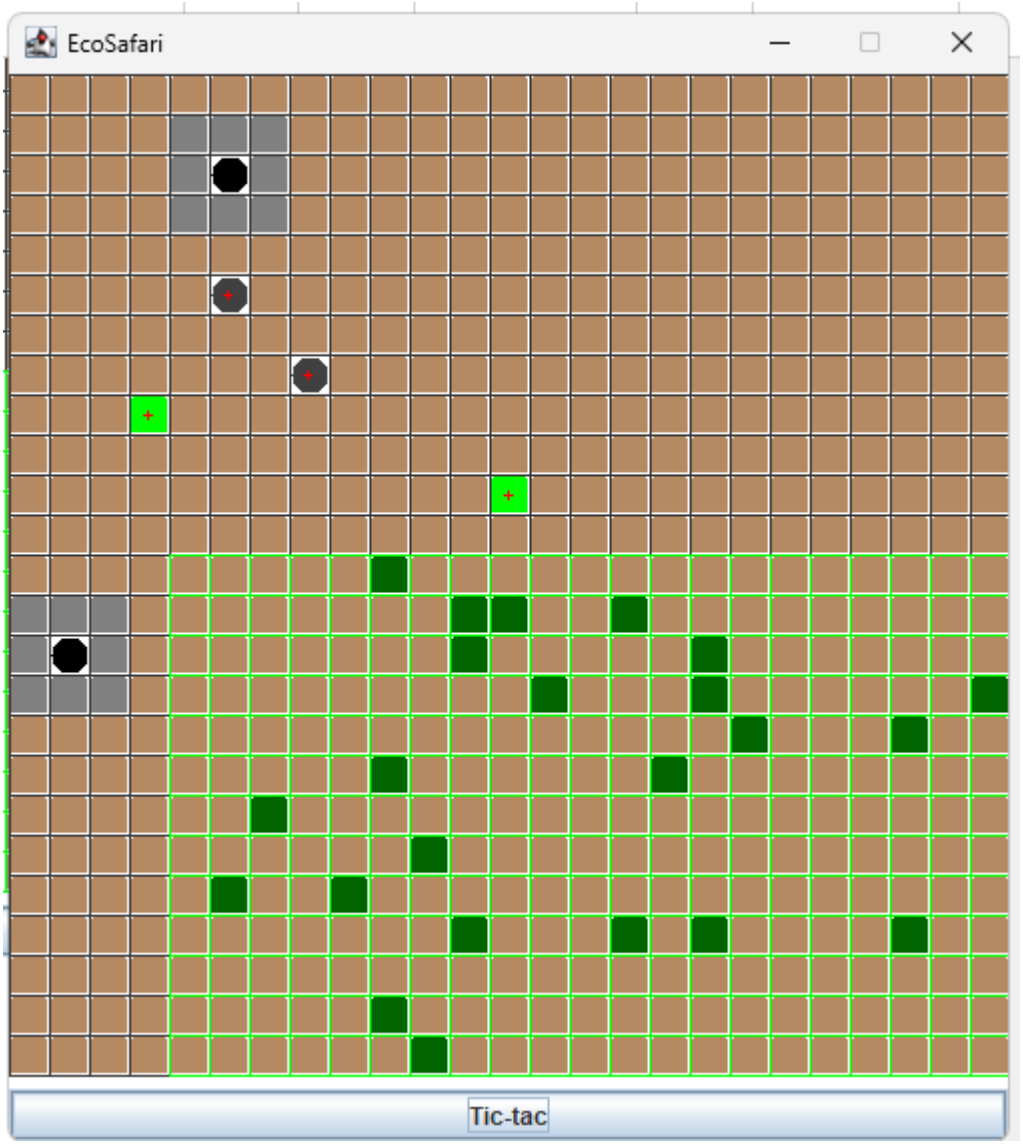
El método de tic(), donde la tierra tiene un 10% de probabilidades de generar pasto:

```java
@Override
/**
 * If earth is inside the grasslandZone there is a 10% of probability it will become grass.
 */
public void tic(){
    int [] position=getHabitat().find(this);
    if(0.1>= Math.random() && habitat.isInsideGrasslandZone(position[0],position[1])){
        new Grass(getHabitat(), position[0], position[1]);
    }
}
```

Esto se puede ver de la siguiente manera:



Al darle Tic-tac:



Se definió la siguiente prueba:

```java
/**
 * At least one of the earth will become grass in the grassland zone.
 */
@Test
public void shouldAtLeastOneOfEarthWillBecomeGrassInGrasslandZone(){
    for(int i = 0; i < 25; i++){
        for(int j = 0; j <25; j++){
            new Grass(es,i,j);
        }
    }
    es.ticTac();
    boolean theresGrass = false;
    for(int i = 14; i < 25; i++){
        for(int j = 4; j <25; j++){
            theresGrass = true;
            break;
        }
    }

    assertTrue(theresGrass);
}
```



3. Implementación del vecindario de Moore

Vemos necesario implementar ahora el vecindario de Moore para futuras acciones, por tanto funcionará del siguiente modo:

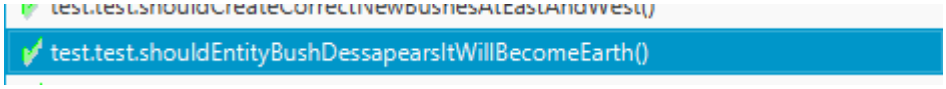
Se darán unas coordenadas de filas y columnas, y se dará la cantidad de determinada clase.

Por el momento como solo se tiene la tierra, solo se hará para la clase Earth:

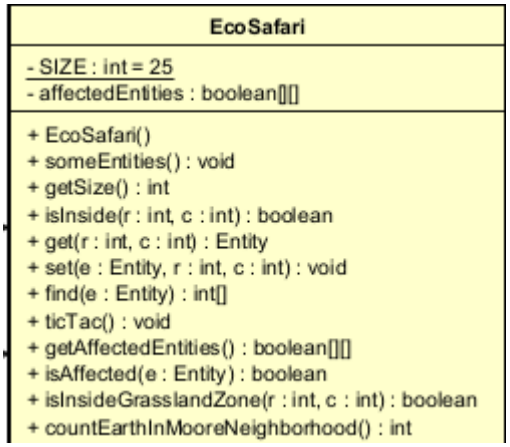
```java
/**
 * Looks how much of Earth there are in the spaces around a specific position
 * @param r is the row
 * @param c is the column
 * @return the number of times that earth appears in the Moree neighborhood
 */
public int countEarthInMooreNeighborhood(int r, int c){
    int cant = 0;
    for(int i =-1; i<=1; i++){
        for(int j= -1; j<=1; j++){
            if(i!=0 || j!=0 && isInside(i+r,j+c)){
                if(get(i+r,j+c) instanceof Earth){
                    cant++;
                }
            }
        }
    }
    return cant;
}
```

Para esto se implementa la siguiente prueba:

```java
/**
 * Should give 8 Earth if we know the Moore neighborhood is just Earth.
 */
@Test
public void shouldGiveEightEarthInMooreNeighborhood(){
    for(int i = 0; i <=2; i++){
        for(int j = 0; j<= 2; j++){
            new Earth(es,i,j);
        }
    }
    assertEquals(8, es.countEarthInMooreNeighborhood(1,1));
}
```

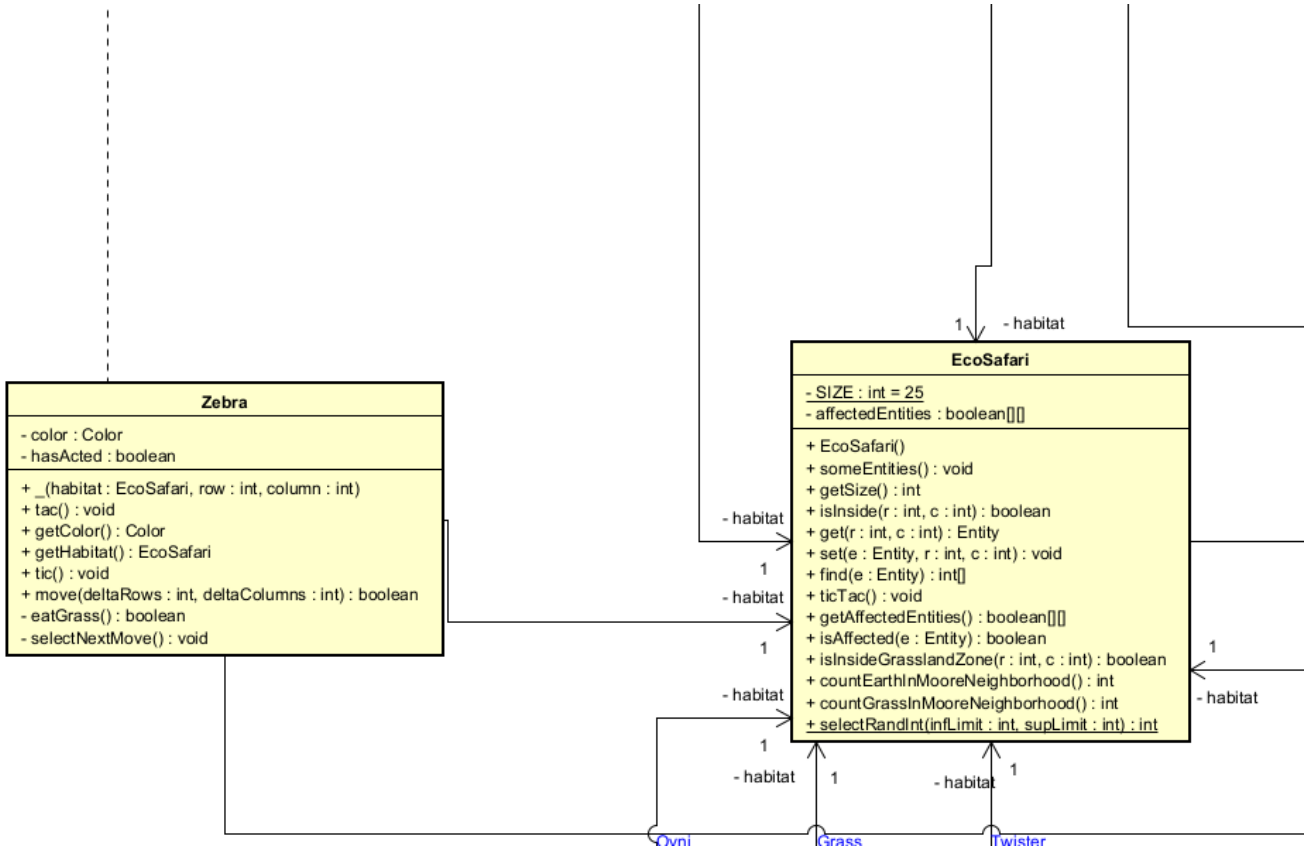


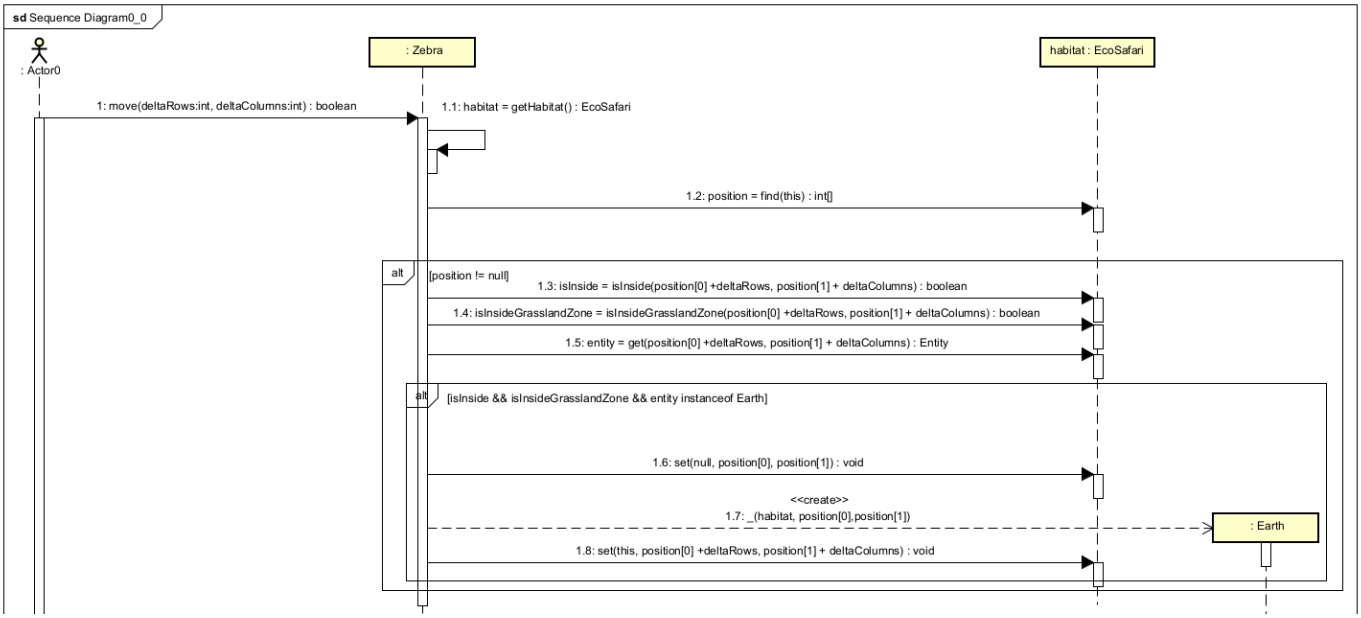
Con el siguiente cambio al diagrama de clases:



4. Crear zebra. Se mueve sobre la tierra. Avanzan dos celdas por paso. Pierde 10% de su energía, e intercambian su posición con la celda de tierra ocupada. Muere si se queda sin energía.

El diagrama de clases se hace de la siguiente manera:





Además se define un test para comprobar que el comportamiento:

```java
/**
 * Should zebra move 2 cells.
 */
@Test
public void shouldZebraMoveTwoCells(){
    Zebra zebra1 = new Zebra(es,24,24);

    Grass grass1 = new Grass(es,24,22);
    Grass grass2 = new Grass(es,23,22);
    Grass grass3 = new Grass(es, 22,22);
    Grass grass4 = new Grass(es,22,23);

    assertEquals(zebra1, es.get(22,24));
}
```

Ya para el código de la zebra:

```java
package domain;
import java.awt.Color;
import java.util.ArrayList;

/**
 * Represents a Zebra that is an organism and implements Entity<br>
 * <b>(habitat, hasActed)</b>
 */
public class Zebra extends Organism implements Entity{
    private final EcoSafari habitat;
    private boolean hasActed;
    private Color color;

    /**
     * Constructs a new Zebra.
     * @param habitat is Zebra's habitat.
     * @param row is the row position.
     * @param column is the column position.
     */
    public Zebra(EcoSafari habitat,int row, int column){
        this.habitat=habitat;
        habitat.set((Entity)this, row, column);
        hasActed=false;
        color = Color.GRAY;
    }

    @Override
    /**
     * Gives the Zebra's habitat.
     * @return Zebra's habitat.
     */
    public EcoSafari getHabitat(){
        return habitat;
    }

```

```java
@Override
/**
 * Gives Zebra's color depending its energy.
 * @return Zebra's color.
 */
public final Color getColor(){
    return(getEnergy()>=40? color: Color.LIGHT_GRAY);
}

@Override
/**
 * It moves the Zebra if it hasn't acted and it loses energy. If energy is 0 it disappears.
 */
public void tic(){
    if(!hasActed){
        int[] nextMove = selectNextMove();
        if(nextMove[0] != 0 || nextMove[1] != 0){
            if(move(nextMove[0], nextMove[1])){
                changeEnergy(-10);
                if (getEnergy()==0){
                    disappear();
                }
            }
        }
    }

    hasActed=true;
}
```

```java
/**
 * Selects the next Zebra's move, if it can't move it will be in the same position
 * @return the x and y position of the selected next move.
 */
private int[] selectNextMove(){
    int[] nextMove = new int[]{0,0};
    int cont =0;
    ArrayList<int[]> opciones = new ArrayList<>();
    for(int i =-2; i <=2; i=i+2){
        for(int j = -2; j<= 2; j= j+2){
            if(i!=0 || j!=0){
                int[] añadir = new int[]{i,j};
                opciones.add(añadir);
            }
        }
    }
    int start = (int)(Math.random()*8);
    while(cont < 8){
        int[] actualOpc = opciones.get((cont+start)%(8));
        int r= actualOpc[0] + habitat.find(this)[0];
        int c = actualOpc[1] + habitat.find(this)[1];
        if(habitat.isInside(r,c) && habitat.isInsideGrasslandZone(r,c) && habitat.get(r,c) instanceof Earth){
            return actualOpc;
        }
        else{
            cont++;
        }
    }

    return nextMove;
}
```

```java
/**
 * It change hasActed to false.
 */
public void tac(){
    hasActed=false;
}

@Override
/**
 * Moves the Zebra to a specific position, but it must be inside the grassland zone.
 * @param deltaRows are rows quantity it will move
 * @param deltaColumns are columns quantity it will move
 * @return if it could move.
 */
public boolean move(int deltaRows, int deltaColumns){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.isInside(r+deltaRows,c+deltaColumns) && habitat.isInsideGrasslandZone(r+deltaRows,c+deltaColumns) && habitat.get(r+deltaRows,c+deltaColumns)instanceof Earth){
            habitat.set(new Earth(getHabitat(), r, c),r,c);
            habitat.set(this,r+deltaRows,c+deltaColumns);
            ok=true;
        }
    }
    return ok;
}
```

Fue necesario poblar con al menos un especimen:

```java
/**
 * Populates the EcoSafari with some entities
 */
public void someEntities(){
    //We create dumbo and babar:
    //Dumbo will be in position (5,5), but in a matrix it represents [4][4]
    Elephant dumbo = new Elephant(this, 4,4);

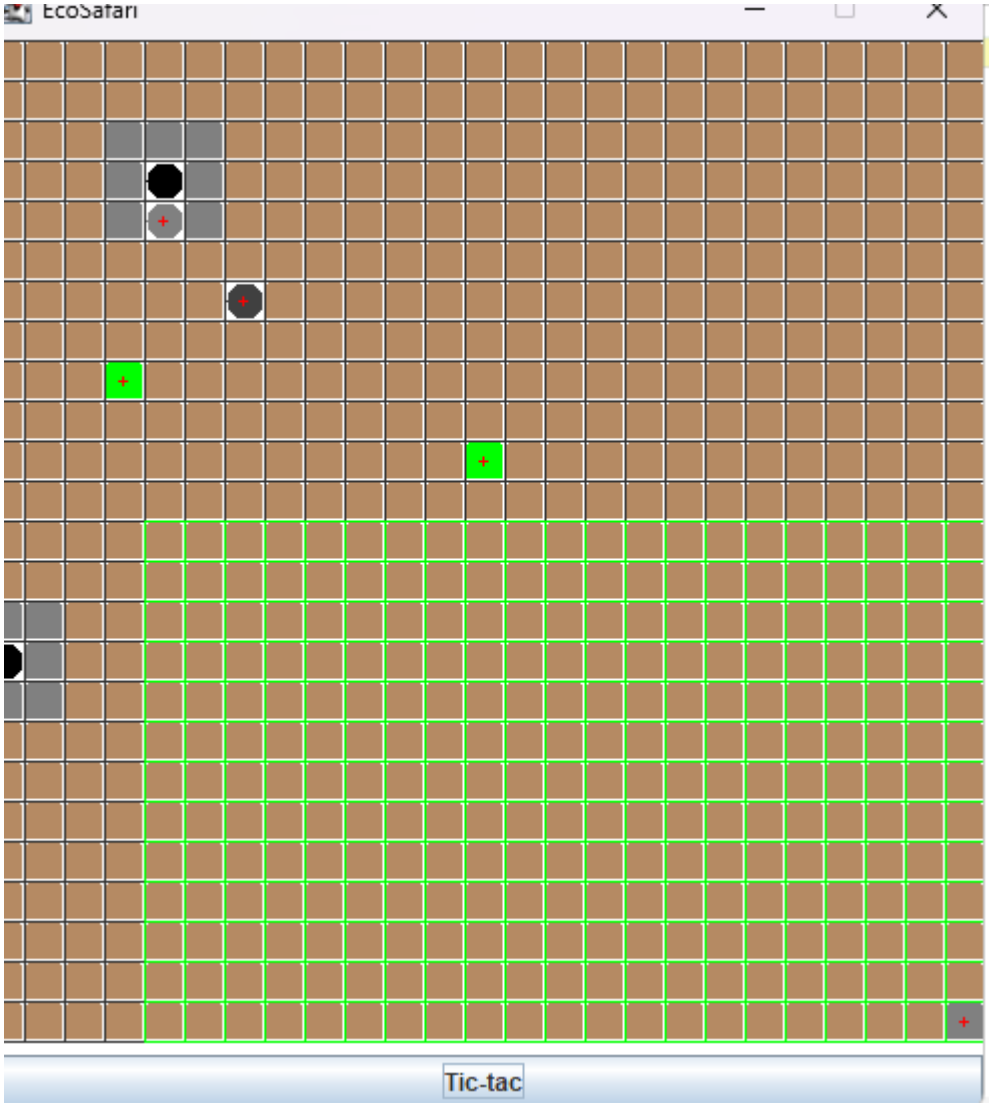
    //Babar will be in position (6,6) but in a matrix it represents [5][5]
    Elephant babar = new Elephant(this, 6,6);

    //We create mopane and acacia.
    Bush mopane = new Bush(this, 10,12,false);
    Bush acacia = new Bush(this, 8,3,false);


    //We put two twisters Thor y Tempest
    Twister thor = new Twister(this, 3,4);
    thor.tac();
    Twister tempest = new Twister(this, 15,0);
    tempest.tac();

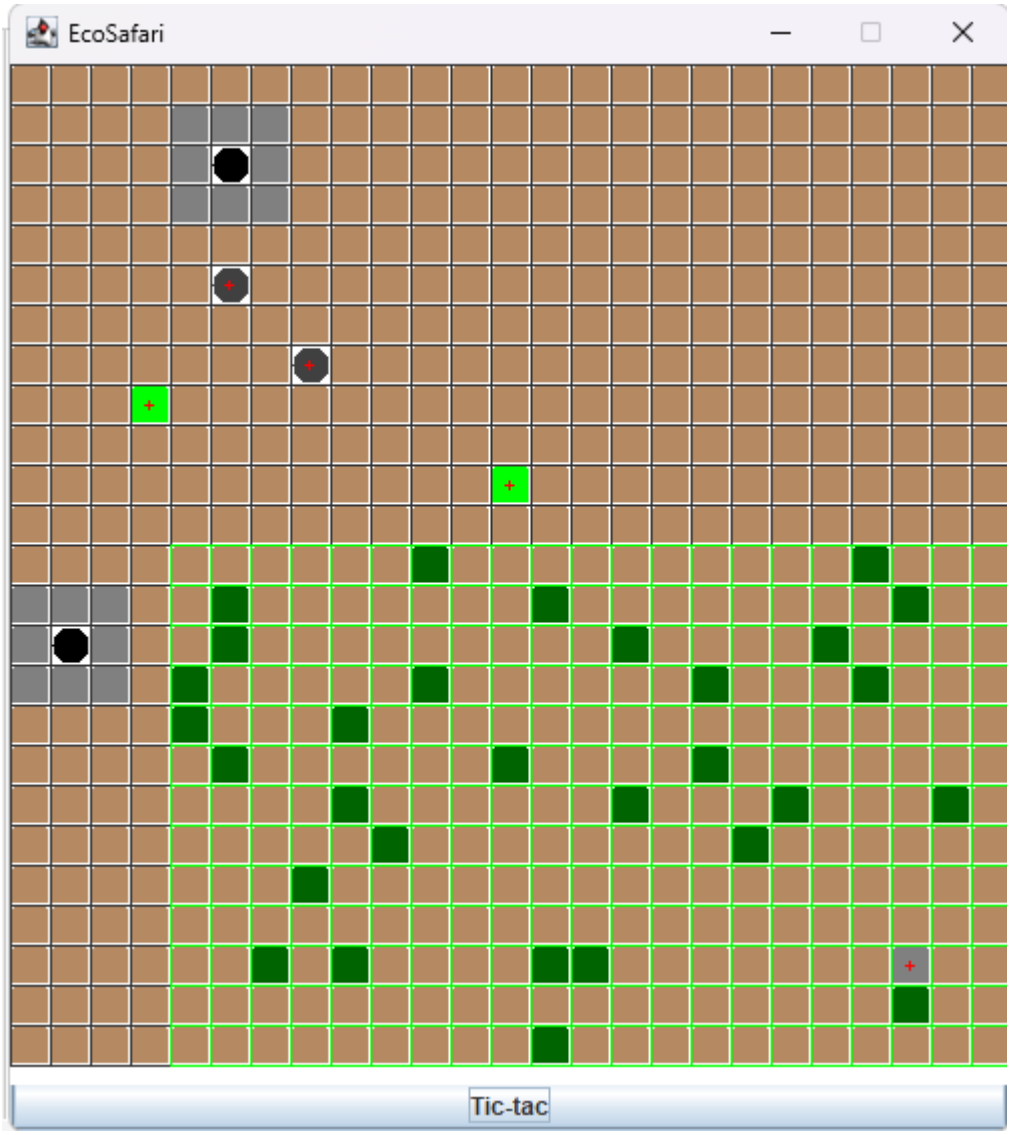
    //We put a zebra
    Zebra zebra = new Zebra(this,24,24);
}
```

Más visualmente se ve como:

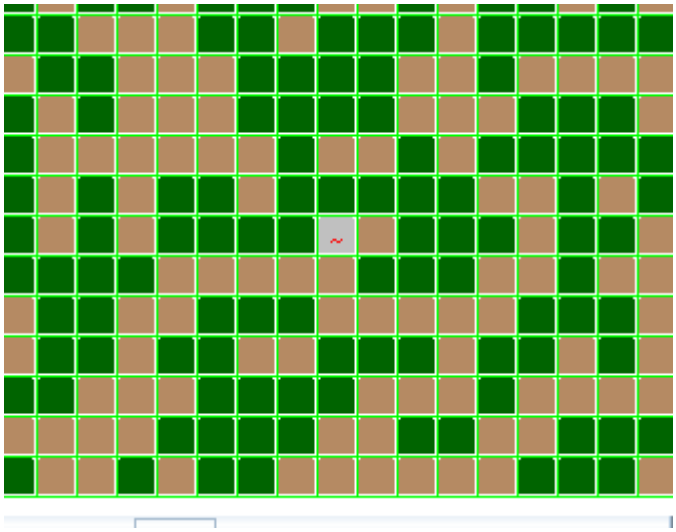


Siendo la zebra ese que está en la esquina inferior derecha.

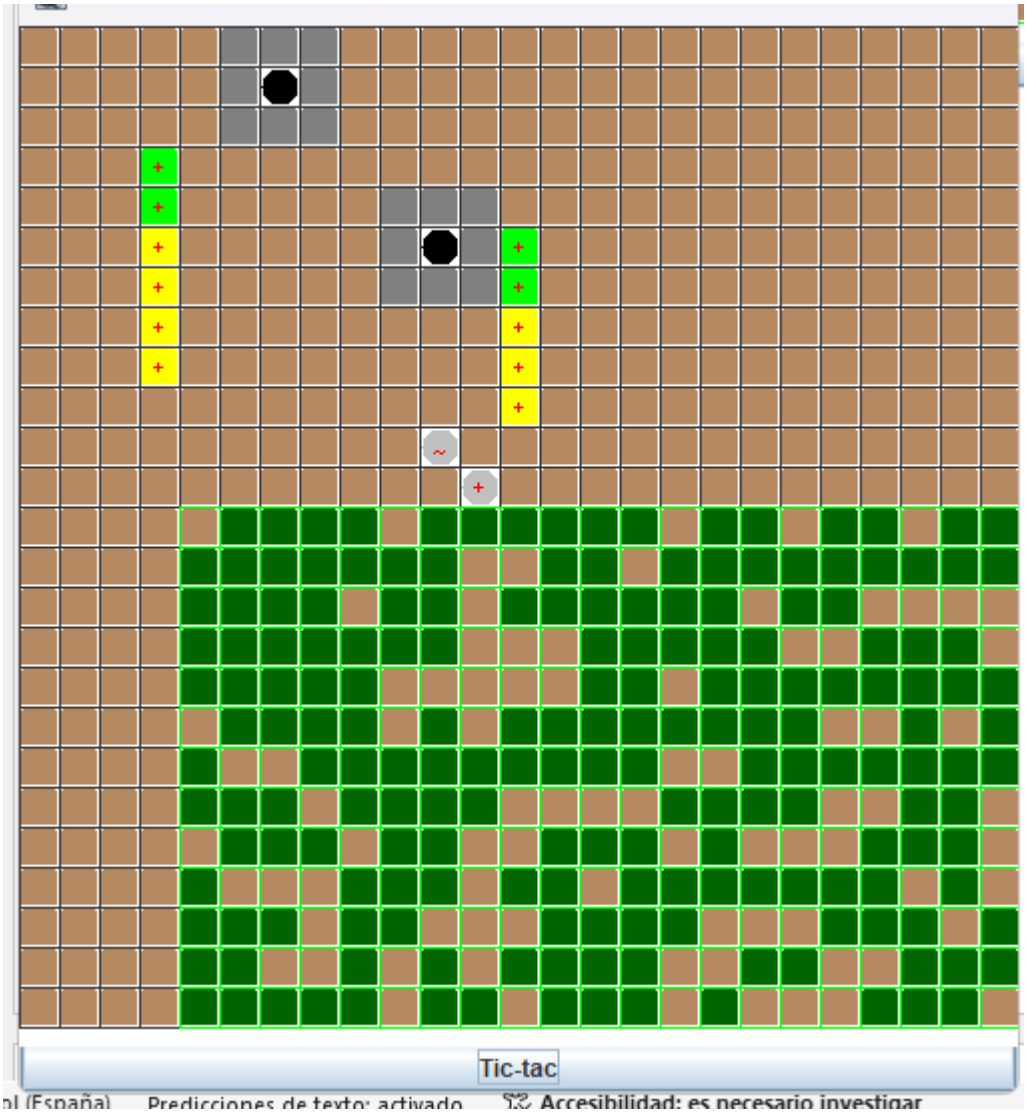
Al hacer Tic-tac se mueve:



Luego de hacer varios Tic-tac y cuando tiene menos vida se ve así:



Luego de varios clics:



Se ve como ya no está, o sea, se muere:

Eso sí, este diseño es temporal, se tiene planeado más adelante darle una mejora a la parte visual.

Además de que pasa la única prueba que se tenía para este momento:



5. Si la zebra queda cerca al pasto se lo come y aumenta su energía en un 25%.

Para esta parte es necesario crear otra versión del vecindario de Moore pero, solo será para identificar si hay pasto cerca.

```java
/**
 * Looks how much of Grass there are in the spaces around a specific position
 * @param r is the row
 * @param c is the column
 * @return the number of times that grass appears in the Moree neighborhood
 */
public int countGrassInMooreNeighborhood(int r, int c){
    int cant = 0;
    for(int i =-1; i<=1; i++){
        for(int j= -1; j<=1; j++){
            if(i!=0 || j!=0 && isInside(i+r,j+c)){
                if(get(i+r,j+c) instanceof Grass){
                    cant++;
                }
            }
        }
    }
    return cant;
}
```

Para comprobar este método se usa la siguiente prueba:

```java
}

/**
 * Should give 8 Grass if we know the Moore neighborhood is just Grass.
 */
@Test
public void shouldGiveEightGrassInMooreNeighborhood(){
    for(int i = 0; i <=2; i++){
        for(int j = 0; j<= 2; j++){
            new Grass(es,i,j);
        }
    }
    assertEquals(8, es.countGrassInMooreNeighborhood(1,1));
}


/**
```

Al ejecutar la prueba:



Por otro lado, es necesario implementar que la zebra se coma el pasto y aumente su cantidad de energía un 25%.

Para esto se definieron las siguientes pruebas:

```java
/**
 * Should zebra eat grass and recover all his energy
 */
@Test
public void shouldZebraRecoverEnergyEatingAGrass(){
    Zebra zebra1 = new Zebra(es,24,24);

    Grass grass1 = new Grass(es,24,22);
    Grass grass2 = new Grass(es,23,22);
    Grass grass3 = new Grass(es, 22,22);
    Grass grass4 = new Grass(es,22,23);

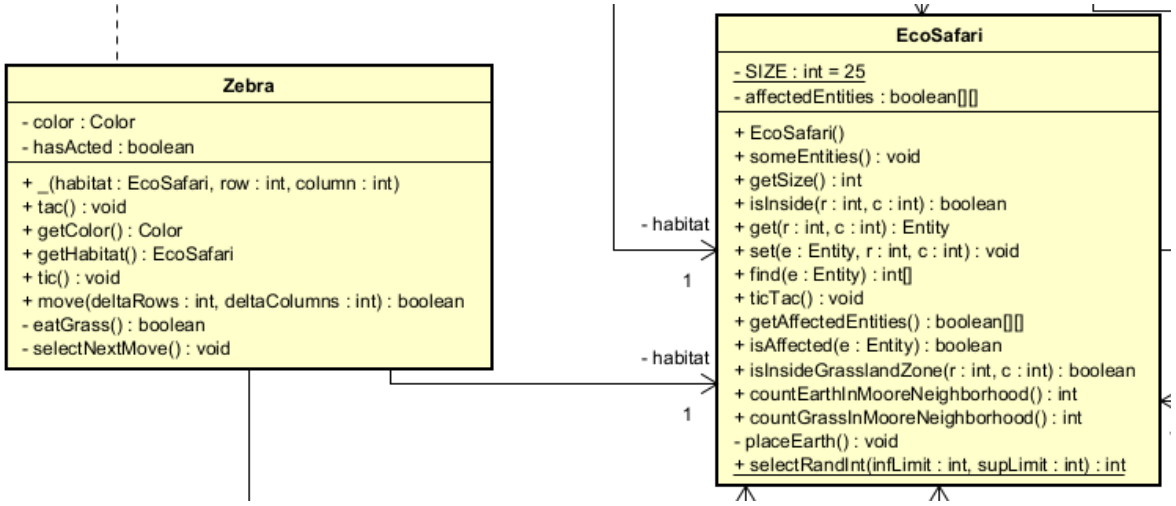
    es.ticTac();
    assertEquals(zebra1.getEnergy(), 100);
}

/**
 * Zebra should eat just one grass per Tic-tac.
 */
@Test
public void shouldZebraEatJustOneGrassPerTicTac(){
    Zebra zebra1 = new Zebra(es,24,24);

    Grass grass1 = new Grass(es,24,22);
    Grass grass2 = new Grass(es,23,22);
    Grass grass3 = new Grass(es, 22,22);
    Grass grass4 = new Grass(es,22,23);
    Grass grass5 = new Grass(es,21,23);
    Grass grass6 = new Grass(es,21,24);


    es.ticTac();
    assertEquals(2, es.countGrassInMooreNeighborhood(22,24));
}
```

Como se va a añadir una nuevo método en zebra y en EcoSafari fue necesario modificar el diseño del astah:



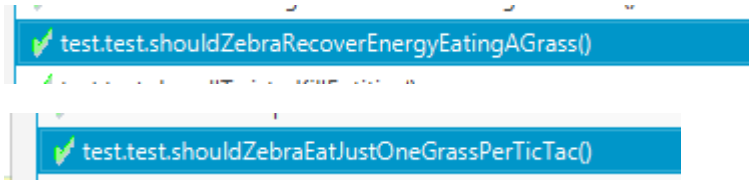
Donde se añade eatGrass() en zebra y countGrassInMooreNeighborhood para EcoSafari

Para la implementación del código quedó como:

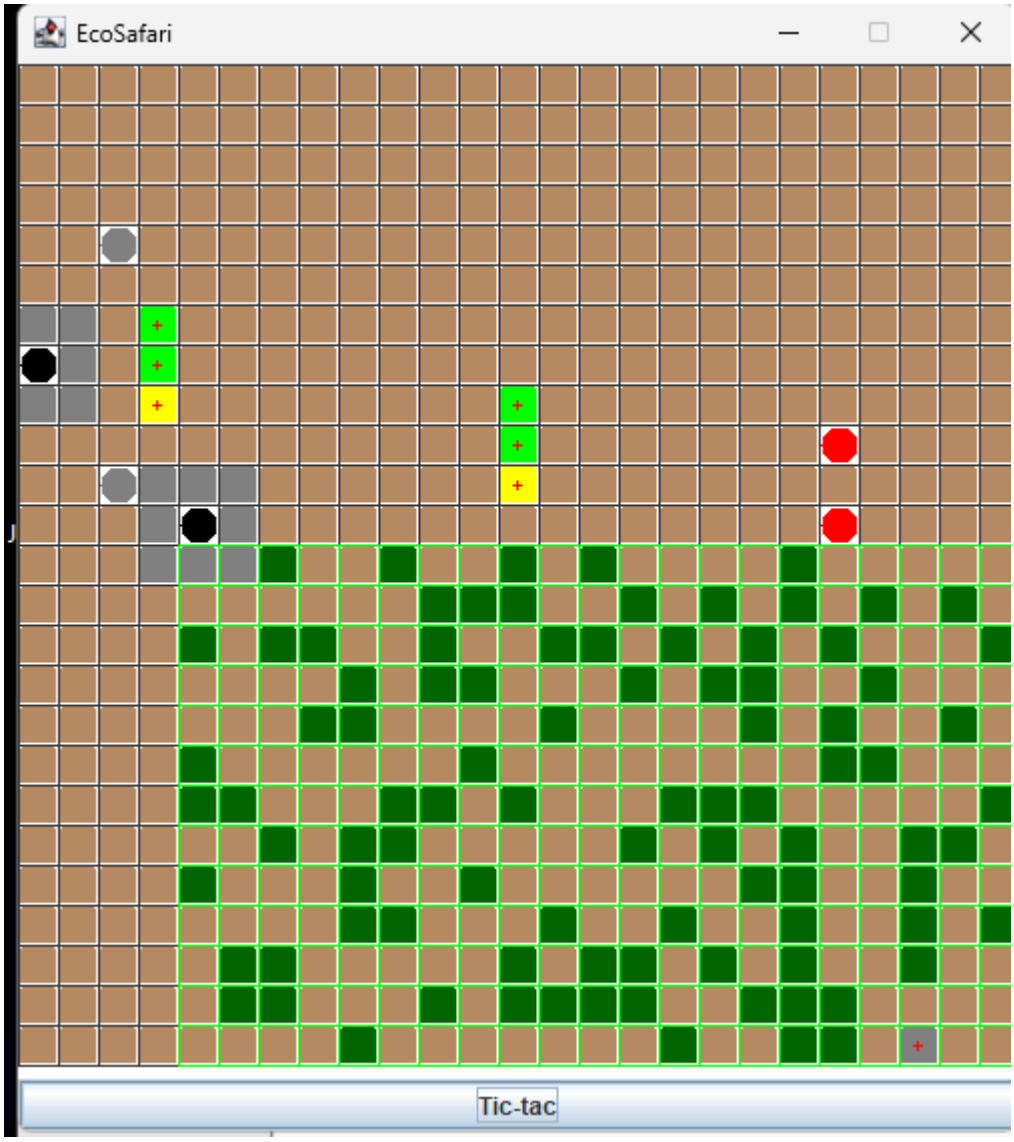
```java
/**
 * It change hasActed to false and eats grass.
 */
public void tac(){
    if(eatGrass()){
        changeEnergy((float) 0.25);
    }
    hasActed=false;
}

/**
 * Zebra eats one grass if it is near to it.
 * @return if zebra could eat grass
 */
private boolean eatGrass(){
    int[]pos = habitat.find(this);
    if(habitat.countGrassInMooreNeighborhood(pos[0], pos[1]) > 0){
        for(int i = -1; i <=1; i++){
            for(int j= -1; j <=1; j++){
                if(i != 0 || j != 0){
                    if(habitat.get(pos[0]+i, pos[1]+j) instanceof Grass){
                        habitat.get(pos[0]+i, pos[1]+j).disappear();
                        return true;
                    }
                }
            }
        }
    }
    return false;
}
```

Por último, tiene que haber pasado las pruebas necesarias:

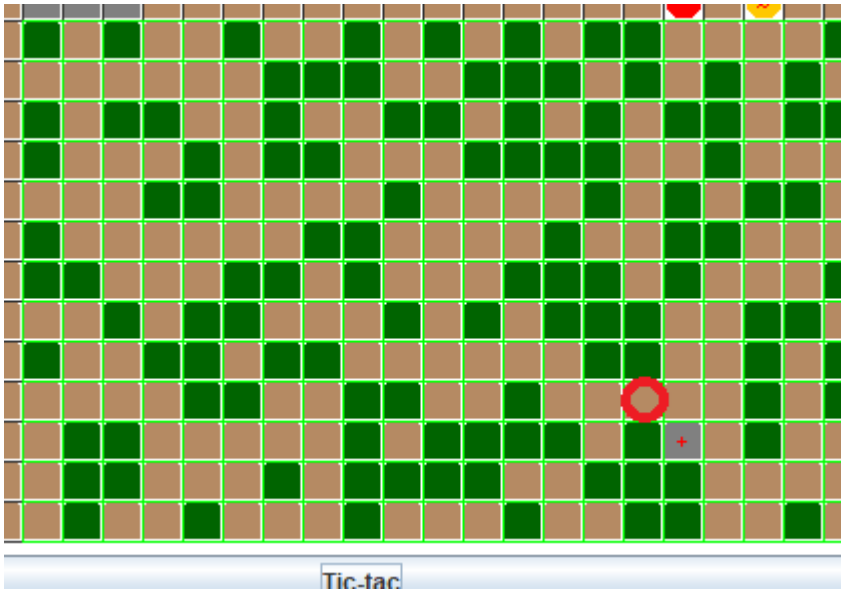


Para verlo un poco mejor:

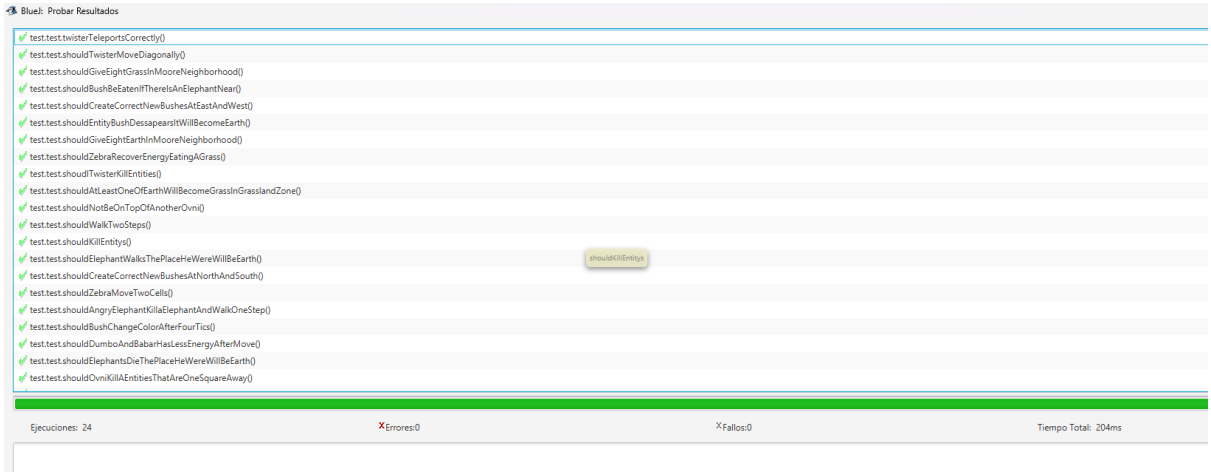


Se tiene ese estado de partida, la cebra tiene varias opciones para moverse, cuando hago Tic-tac de nuevo:

Se ve como desapareció el pedazo de pasto que estaba ahí antes.



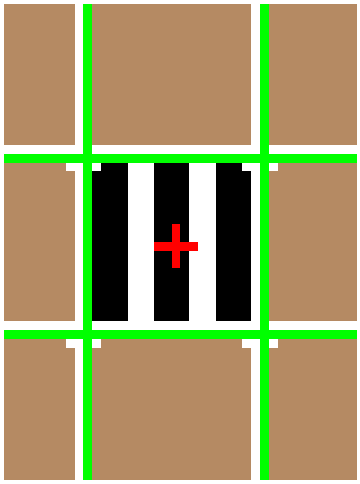
Por tanto, hasta el momento el resumen de pruebas es:



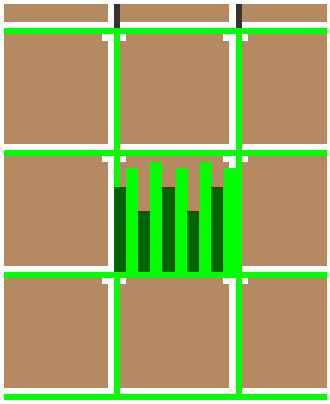
5.5 Mejora de la interfaz gráfica:

Se hizo una mejora de la interfaz gráfica para algunos de las clases que estaban ahí, más en específico: Las clases: Zebra, Grass, Ovni:

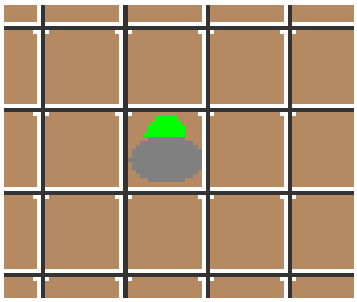
Zebra:



Grass:



Ovni:



Para esto fue necesario cambiar la forma en la que se pintaban los objetos en EcoSafariGUI:

```
}
for (int f=0;f<theEcoSafari.getSize();f++){
    for(int c=0;c<theEcoSafari.getSize();c++){
        g.setColor(new Color(181, 138, 99));
        g.fillRoundRect(gui.SIDE*c+1,gui.SIDE*f+1,gui.SIDE-2,gui.SIDE-2,2,2);

        if(theEcoSafari.get(f,c) instanceof Ovni){
            paintOvni(g, f, c);
        }
        else if(theEcoSafari.get(f,c) instanceof Zebra){
            paintZebra(g, f, c);
            g.setColor(Color.red);
            if (((Organism)theEcoSafari.get(f,c)).getEnergy()>=50){
                g.drawString("+",gui.SIDE*c+7,gui.SIDE*f+15);
            } else {
                g.drawString("~",gui.SIDE*c+7,gui.SIDE*f+17);
            }
        }
        else if(theEcoSafari.get(f,c) instanceof Grass){
            paintGrass(g, f, c);
        }
        else if (theEcoSafari.get(f,c)!=null){
            if(theEcoSafari.getAffectedEntities()[f][c]){
                g.setColor(Color.GRAY);
            }
            else{
                g.setColor(theEcoSafari.get(f,c).getColor());
            }
            if (theEcoSafari.get(f,c).shape()==Entity.SQUARE){
                g.fillRoundRect(gui.SIDE*c+1,gui.SIDE*f+1,gui.SIDE-2,gui.SIDE-2,2,2);
            }else {
                g.fillOval(gui.SIDE*c+1,gui.SIDE*f+1,gui.SIDE-2,gui.SIDE-2);
            }

            if (theEcoSafari.get(f,c).isOrganism()){
                if(theEcoSafari.get(f,c) instanceof AngryElephant){
                    g.setColor(Color.BLACK);
                }
                else{
                    g.setColor(Color.red);
                }
                if (((Organism)theEcoSafari.get(f,c)).getEnergy()>=50){
                    g.drawString("+",gui.SIDE*c+6,gui.SIDE*f+15);
                } else {
                    g.drawString("~",gui.SIDE*c+6,gui.SIDE*f+17);
```

Además de crear varios métodos privados, que sean los que se encarguen de dibujar las figuras de esas clases tan especiales.

```
private void paintOvni(Graphics g, int f, int c){
    if(theEcoSafari.getAffectedEntities()[f][c]){
        g.setColor(Color.GRAY);
    }
    else{
        g.setColor(Color.GREEN);
    }
    g.fillOval(gui.SIDE*c+5,gui.SIDE*f+1,gui.SIDE-10,gui.SIDE-10);

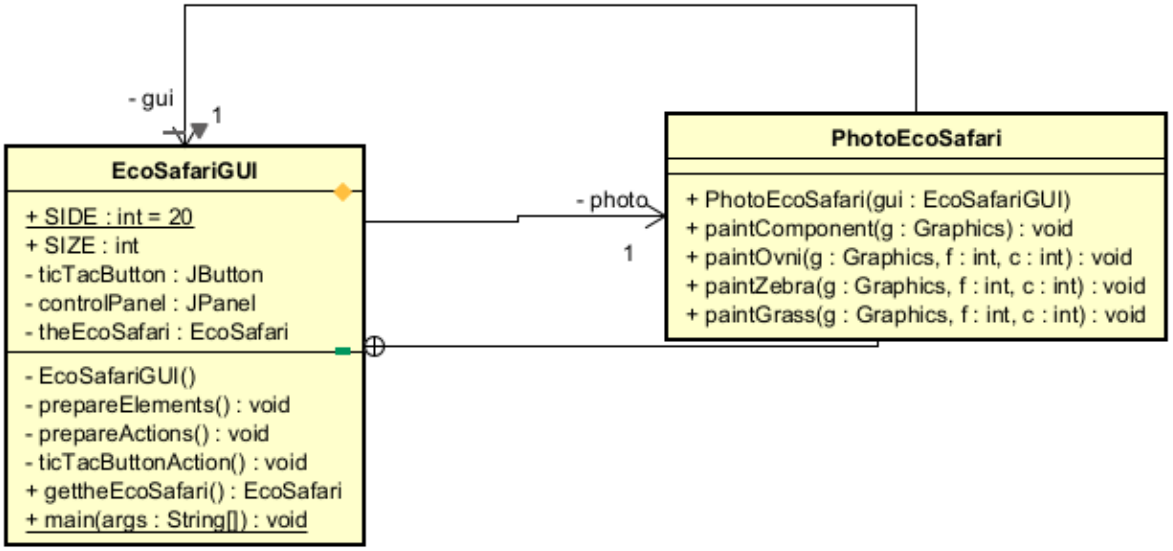
    g.setColor(Color.GRAY);
    g.fillOval(gui.SIDE*c+1,gui.SIDE*f+6,gui.SIDE-2,gui.SIDE-8);
}

private void paintZebra(Graphics g, int f, int c){
    if(theEcoSafari.getAffectedEntities()[f][c]){
        g.setColor(Color.GRAY);
        g.fillRoundRect(gui.SIDE*c+1,gui.SIDE*f+1,gui.SIDE-2,gui.SIDE-2,2,2);
    }
    else{
        g.setColor(Color.BLACK);
        g.fillRoundRect(gui.SIDE*c+1,gui.SIDE*f+1,gui.SIDE-2,gui.SIDE-2,2,2);
        g.setColor(Color.WHITE);
        g.fillRoundRect(gui.SIDE*c+5,gui.SIDE*f+1,gui.SIDE-17,gui.SIDE-2,5,0);
        g.fillRoundRect(gui.SIDE*c+12,gui.SIDE*f+1,gui.SIDE-17,gui.SIDE-2,5,0);
    }
}
```

```
private void paintGrass(Graphics g, int f, int c){
    boolean affected = theEcoSafari.getAffectedEntities()[f][c];
    if(affected){
        g.setColor(Color.GRAY);
        g.fillRoundRect(gui.SIDE*c,gui.SIDE*f+6,gui.SIDE-18,gui.SIDE-6,5,0);
        g.fillRoundRect(gui.SIDE*c+2,gui.SIDE*f+3,gui.SIDE-18,gui.SIDE-2,5,0);
        g.fillRoundRect(gui.SIDE*c+4,gui.SIDE*f+10,gui.SIDE-18,gui.SIDE-10,5,0);
        g.fillRoundRect(gui.SIDE*c+6,gui.SIDE*f+2,gui.SIDE-18,gui.SIDE-2,5,0);
        g.fillRoundRect(gui.SIDE*c+8,gui.SIDE*f+6,gui.SIDE-18,gui.SIDE-6,5,0);
        g.fillRoundRect(gui.SIDE*c+10,gui.SIDE*f+3,gui.SIDE-18,gui.SIDE-2,5,0);
        g.fillRoundRect(gui.SIDE*c+12,gui.SIDE*f+10,gui.SIDE-18,gui.SIDE-10,5,0);
        g.fillRoundRect(gui.SIDE*c+14,gui.SIDE*f+2,gui.SIDE-18,gui.SIDE-2,5,0);
        g.fillRoundRect(gui.SIDE*c+16,gui.SIDE*f+6,gui.SIDE-18,gui.SIDE-6,5,0);
        g.fillRoundRect(gui.SIDE*c+18,gui.SIDE*f+3,gui.SIDE-18,gui.SIDE-2,5,0);

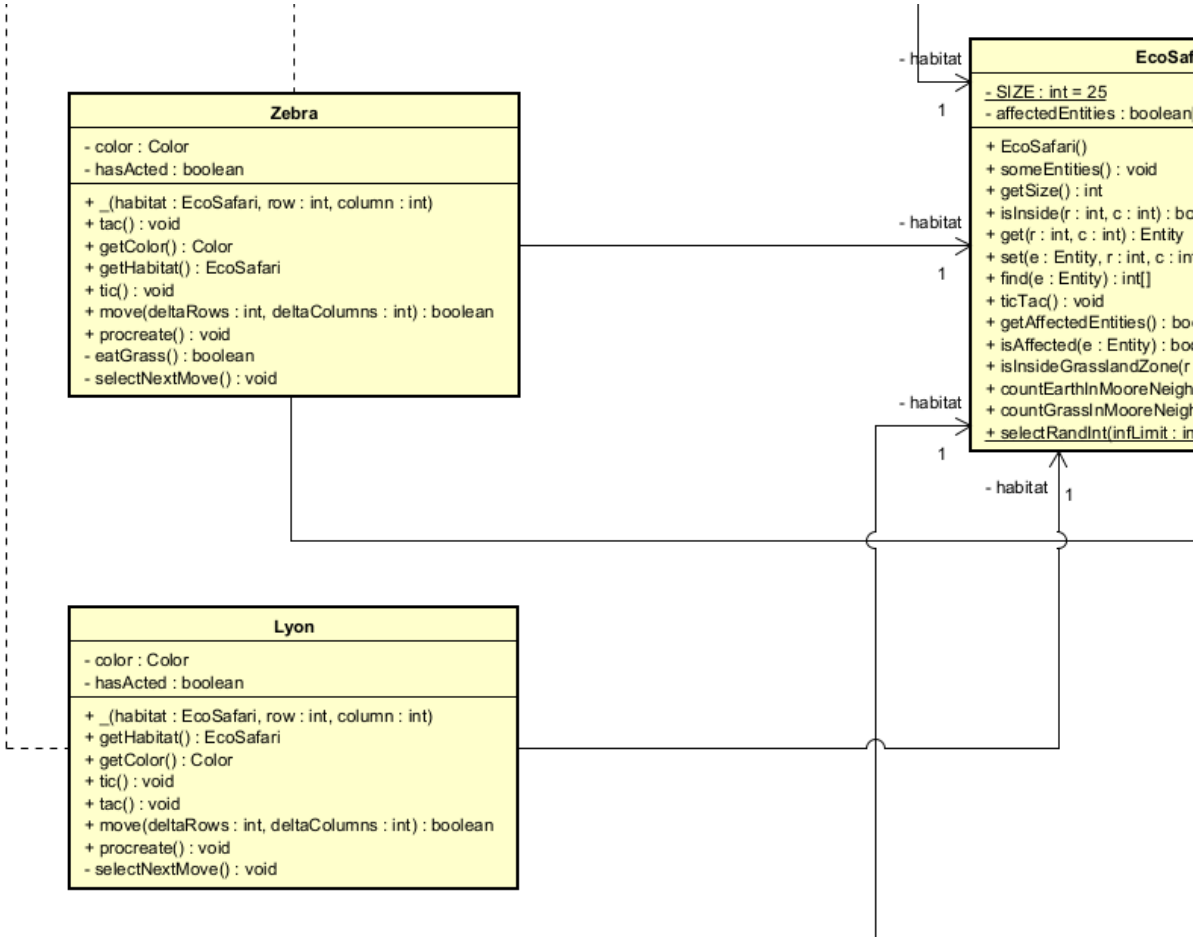
    }
    else{
        g.setColor(new Color(0,100,0));
        g.fillRoundRect(gui.SIDE*c,gui.SIDE*f+6,gui.SIDE-18,gui.SIDE-6,5,0);
        g.setColor(Color.GREEN);
        g.fillRoundRect(gui.SIDE*c+2,gui.SIDE*f+3,gui.SIDE-18,gui.SIDE-2,5,0);
        g.setColor(new Color(0,100,0));
        g.fillRoundRect(gui.SIDE*c+4,gui.SIDE*f+10,gui.SIDE-18,gui.SIDE-10,5,0);
        g.setColor(Color.GREEN);
        g.fillRoundRect(gui.SIDE*c+6,gui.SIDE*f+2,gui.SIDE-18,gui.SIDE-2,5,0);
        g.setColor(new Color(0,100,0));
        g.fillRoundRect(gui.SIDE*c+8,gui.SIDE*f+6,gui.SIDE-18,gui.SIDE-6,5,0);
        g.setColor(Color.GREEN);
        g.fillRoundRect(gui.SIDE*c+10,gui.SIDE*f+3,gui.SIDE-18,gui.SIDE-2,5,0);
        g.setColor(new Color(0,100,0));
        g.fillRoundRect(gui.SIDE*c+12,gui.SIDE*f+10,gui.SIDE-18,gui.SIDE-10,5,0);
        g.setColor(Color.GREEN);
        g.fillRoundRect(gui.SIDE*c+14,gui.SIDE*f+2,gui.SIDE-18,gui.SIDE-2,5,0);
        g.setColor(new Color(0,100,0));
        g.fillRoundRect(gui.SIDE*c+16,gui.SIDE*f+6,gui.SIDE-18,gui.SIDE-6,5,0);
        g.setColor(Color.GREEN);
        g.fillRoundRect(gui.SIDE*c+18,gui.SIDE*f+3,gui.SIDE-18,gui.SIDE-2,5,0);
    }
```

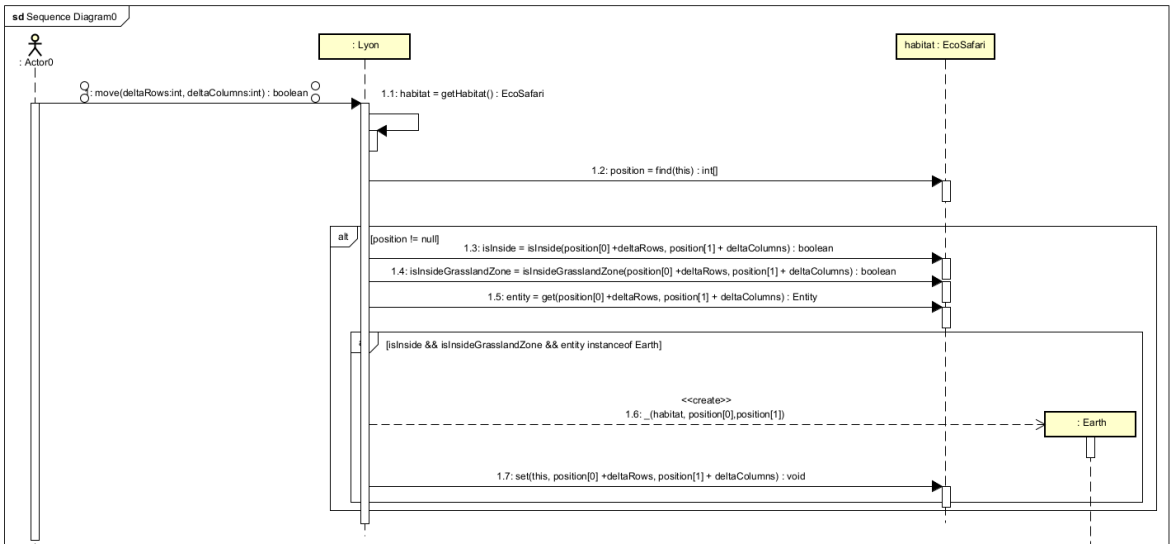
Viendose en astah como:



6. Crear león. Se mueve sobre la tierra. Avanza una celda por paso. Pierde 10% de su energía, e intercambian su posición con la celda de tierra ocupada. Muere si se queda sin energía.

Con respecto al astah:





Además se define un test para probar el comportamiento:

```java
/**
 * Lyon not must eat a grass
 */
@Test
public void shouldNotLyonEatGrass(){
    Lyon lyon1 = new Lyon(es,23,18);

    Grass grass1 = new Grass(es,22,18);
    Grass grass2 = new Grass(es,24,19);
    Grass grass3 = new Grass(es,24,17);
    Grass grass4 = new Grass(es,23,19);
    Grass grass5 = new Grass(es,23,17);
    Grass grass6 = new Grass(es,22,17);
    Grass grass7 = new Grass(es,22,19);

    lyon1.tic();
    lyon1.tac();


    assertEquals(7, es.countGrassInMooreNeighborhood(23,18));
}
```

Ya para el codigo de lyon:

```java
package domain;
import java.awt.Color;
import java.util.ArrayList;

/**
 * Represents a Lyon that is an organism and implements Entity<br>
 * <b>(habitat, hasActed)</b>
 */
public class Lyon extends Organism implements Entity{
    private EcoSafari habitat;
    private boolean hasActed;
    private Color color;

    /**
     * Constructor for objects of class Lyon
     */
    public Lyon(EcoSafari habitat, int row, int column){
        this.habitat = habitat;
        habitat.set((Entity)this, row, column);
        color = Color.YELLOW;
        hasActed=false;
    }

    @Override
    /**
     * Gives the Lyons habitat.
     * @return Lyons habitat.
     */
    public EcoSafari getHabitat(){
        return habitat;
    }
```

```java
@Override
/**
 * Gives the Lyons color.
 * @return Lyons color.
 */
public Color getColor(){
    return (this.getEnergy()>50? color: Color.GRAY);
}
```

```java
@Override
/**
 * It moves the Lyon if it hasn't acted and it loses energy. If energy is 0 it disappears.
 */
public void tic(){
    if(!hasActed){
        int[] nextMove = selectNextMove();
        if(nextMove[0] != 0 || nextMove[1] != 0){
            if(move(nextMove[0], nextMove[1])){
                changeEnergy(-10);
                if (getEnergy()==0){
                    disappear();
                }
            }
        }
    }
    hasActed=true;
}
```

```java
    /**
     * It change hasActed to false and eats zebra.
     */
    public void tac(){

        hasActed=false;
    }
```

```java
@Override
/**
 * Moves the lyon to a specific position, but it must be inside the grassland zone.
 * @param deltaRows are rows quantity it will move
 * @param deltaColumns are columns quantity it will move
 * @return if it could move.
 */
public boolean move(int deltaRows, int deltaColumns){
    int [] position=getHabitat().find(this);
    EcoSafari habitat=getHabitat();
    boolean ok=false;
    if (position!=null){
        int r = position[0];
        int c = position[1];
        if (habitat.isInside(r+deltaRows,c+deltaColumns) && habitat.isInsideGrasslandZone(r+deltaRows,c+deltaColumns) && habitat.get(r+deltaRows,c+deltaColumns)instanceof Earth){
            habitat.set(new Earth(getHabitat(), r, c),r,c);
            habitat.set(this,r+deltaRows,c+deltaColumns);
            ok=true;
        }
    }
    return ok;
}
```

```java
/**
 * Selects the next Lyons move, if it can't move it will be in the same position
 * @return the x and y position of the selected next move.
 */
private int[] selectNextMove(){
    int[] nextMove = new int[]{0,0};
    int cont =0;
    ArrayList<int[]> opciones = new ArrayList<>();
    for(int i =-1; i <=1; i++){
        for(int j = -1; j<= 1; j++){
            if(i!=0 || j!=0){
                int[] añadir = new int[]{i,j};
                opciones.add(añadir);
            }
        }
    }
    int start = (int)(Math.random()*8);


    while(cont < 8){
        int[] actualOpc = opciones.get((cont+start)%(8));
        int r= actualOpc[0] + habitat.find(this)[0];
        int c = actualOpc[1] + habitat.find(this)[1];
        if(habitat.isInside(r,c) && habitat.isInsideGrasslandZone(r,c) && habitat.get(r,c) instanceof Earth){
            return actualOpc;
        }
        else{
            cont++;
        }
    }

    return nextMove;
}
```

Fue necesario poblar con almenos un especimen:

```java
/**
 * Populates the EcoSafari with some entities
 */
public void someEntities(){
    //We create dumbo and babar:
    //Dumbo will be in position (5,5), but in a matrix it represents [4][4]
    Elephant dumbo = new Elephant(this, 4,4);

    //Babar will be in position (6,6) but in a matrix it represents [5][5]
    Elephant babar = new Elephant(this, 6,6);

    //We create mopane and acacia.
    Bush mopane = new Bush(this, 10,12,false);
    Bush acacia = new Bush(this, 8,3,false);


    //We put two twisters Thor y Tempest
    Twister thor = new Twister(this, 3,4);
    thor.tac();
    Twister tempest = new Twister(this, 15,0);
    tempest.tac();

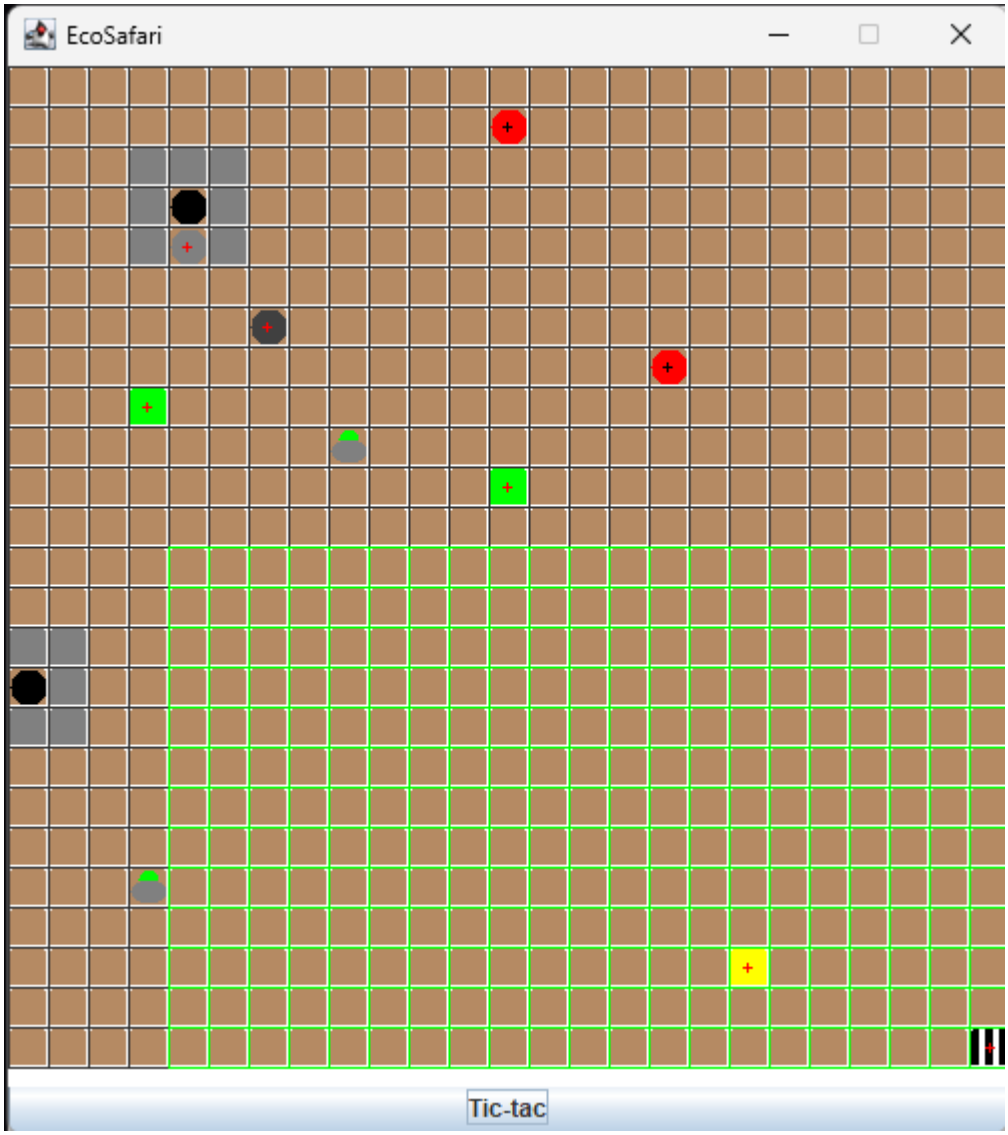
    //We create Garzon and Mendivelso in positions (1, 12) and (7, 16)
    AngryElephant Garzon = new AngryElephant(this, 1, 12);
    AngryElephant Mendivelso = new AngryElephant(this, 7, 16);


    //We put a zebra
    Zebra zebra = new Zebra(this,24,24);

    //We put a lyon
    Lyon lyon = new Lyon(this, 22, 18);

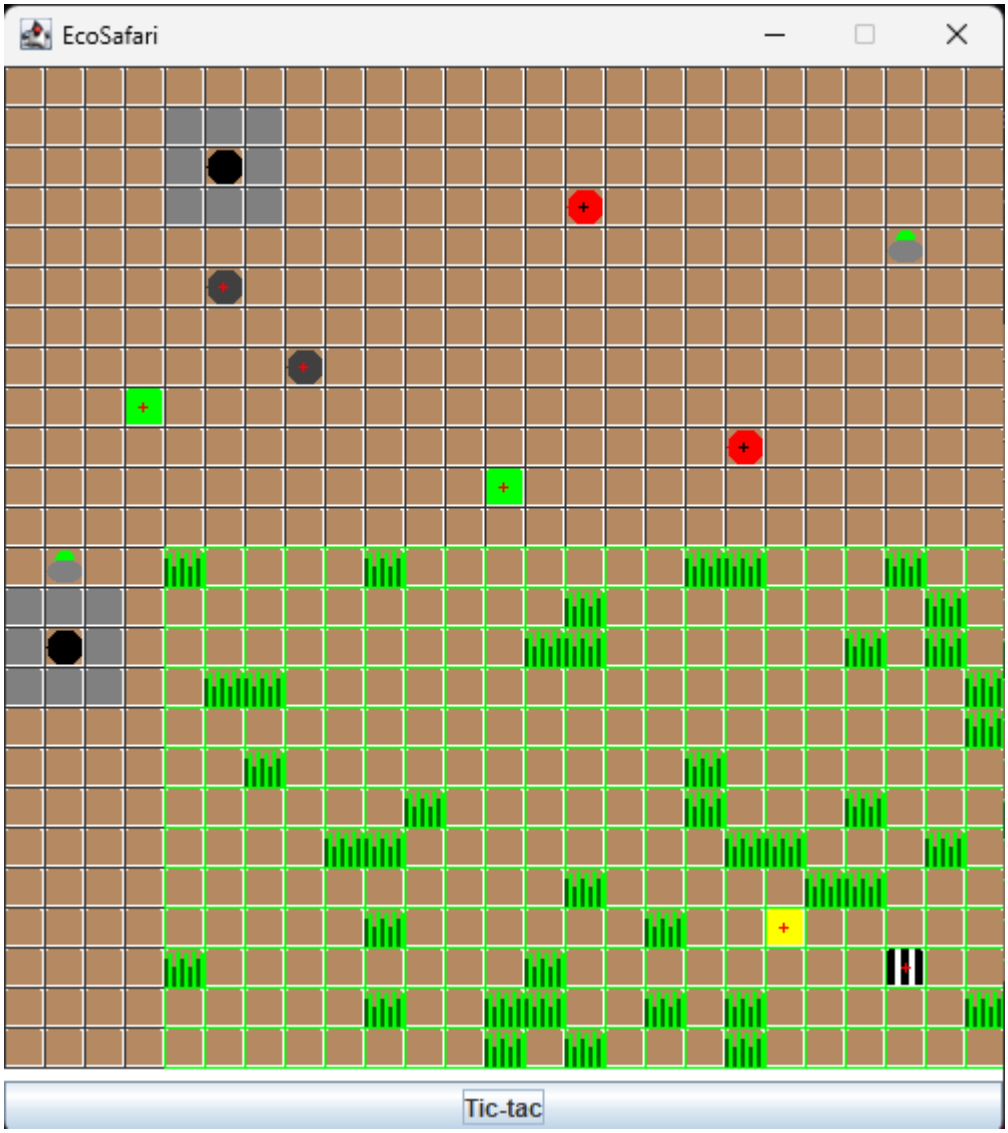
    //We create david and wilson
    Ovni david= new Ovni(this, 20, 3);
    Ovni wilson = new Ovni(this,9,8);
}
```

Mas visualmente se ve como:

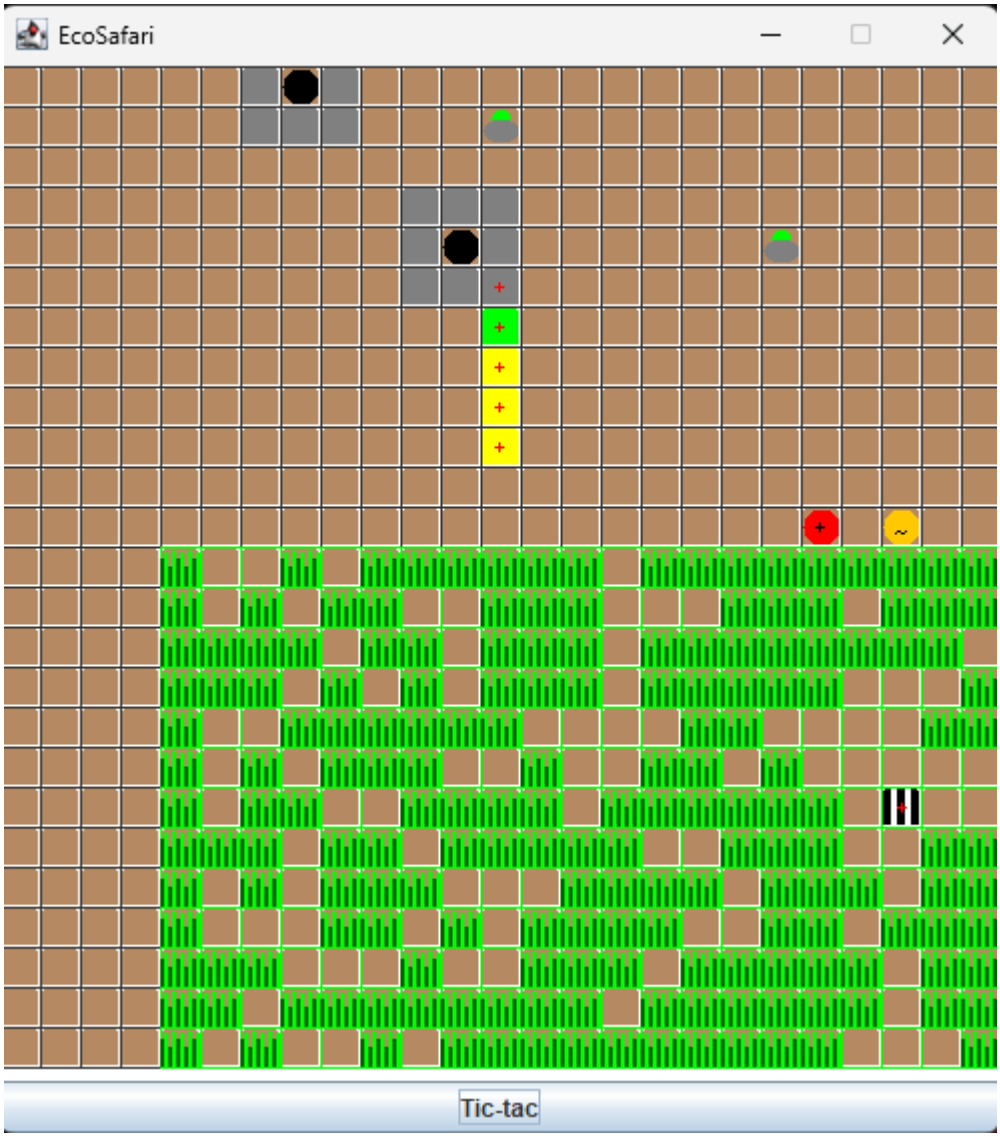


Siendo lyon el cuadro amarillo que esta en el simulador del reto.

Al hacer tic tac se mueve:



Luego de varios clics, lyon muere:



Además de que pasa la única prueba que se tenía para este momento:



7. Si el león queda cerca de la zebra se la come y aumenta su energía en un 50%.

Primero, definimos una prueba:

```java
/**
* Zebra should eat just one grass per Tic-tac.
*/
@Test
public void shouldLyonEatJustOneZebraPerTicTac(){
    Lyon lyon1 = new Lyon(es,22,18);

    Grass grass1 = new Grass(es,21,18);
    Grass grass2 = new Grass(es,21,19);
    Grass grass3 = new Grass(es,21,17);
    Grass grass4 = new Grass(es,22,19);
    Grass grass5 = new Grass(es,22,17);
    Grass grass6 = new Grass(es,23,17);
    Grass grass7 = new Grass(es,23,19);

    Zebra zebra1 = new Zebra(es, 23, 18);

    lyon1.tic();
    lyon1.tac();
    assertEquals(0, es.countZebraInMooreNeighborhood(22,18));
    assertEquals(100, lyon1.getEnergy());

}
```

Para esta parte es necesario crear otra versión del vecindario de Moore pero, solo sera para identificar si hay cebras cerca (una casilla de distancia).

```java
/**
 * Looks how much of Grass there are in the spaces around a specific position
 * @param r is the row
 * @param c is the column
 * @return the number of times that grass appears in the Moree neighborhood
 */
public int countZebraInMooreNeighborhood(int r, int c){
    int cant = 0;
    for(int i =-1; i<=1; i++){
        for(int j= -1; j<=1; j++){
            if(i!=0 || j!=0 && isInside(i+r,j+c)){
                if(get(i+r,j+c) instanceof Zebra){
                    cant++;
                }
            }
        }
    }
    return cant;
}
```

Este método nos será útil para crear el siguiente método en Lyon:

```java
/**
 * Lyon eats one zebra if it is near to it.
 * @return if lyon could eat zebra
 */
public boolean eatZebra(){
    int[]pos = habitat.find(this);
    if(habitat.countGrassInMooreNeighborhood(pos[0], pos[1]) > 0){
        for(int i = -1; i <=1; i++){
            for(int j= -1; j <=1; j++){
                if(i != 0 || j != 0){
                    if(habitat.get(pos[0]+i, pos[1]+j) instanceof Zebra){
                        habitat.get(pos[0]+i, pos[1]+j).disappear();
                        return true;
                    }
                }
            }
        }
    }
    return false;
}
```

Y para lograr el objetivo de esta parte, se implementa en el método tac y se aumenta su energia:

```java
/**
 * It change hasActed to false and eats zebra.
 */
public void tac(){
    if(eatZebra()){
        changeEnergy((float) 0.5);
    }
    hasActed=false;
}
```

Luego, tiene que haber pasado la prueba creada:



8. Si un animal queda a una casilla de tierra de otro de su misma especie, en la celda de tierra nace un nuevo animal de esa especie.

Primero, definimos unas pruebas:

```java
/**
 * Lyon must procreate
 */
@Test
public void shouldProcreateLyon(){
    Lyon lyon1 = new Lyon(es,14, 6);
    Lyon lyon2 = new Lyon(es,16, 6);
    lyon1.tac();

    assertTrue(es.get(15, 6) instanceof Lyon);
}
```

```java
/**
 * Zebra must procreate
 */
@Test
public void shouldProcreateZebra(){
    Zebra zebra1 = new Zebra(es,14, 8);
    Zebra zebra2 = new Zebra(es,16, 8);
    zebra1.tac();

    assertTrue(es.get(15, 8) instanceof Zebra);
}
```

Ahora, para implementar esta parte, definimos el metodo procreate para las clases Zebra y Lyon:

```java
/**
 * Zebra reproduce if they are within a distance of two blocks of each other and there is earth between them.
 * @return if animals can reproduce
 */
public boolean procreate(){
    int[] pos = habitat.find(this);
    int f = pos[0];
    int c = pos[1];
    for(int i = -1; i<=1; i++){
        for(int j = -1; j<=1; j++){
            if(habitat.get(2*i+f , 2*j+c) instanceof Zebra && habitat.get(f+i, c+j) instanceof Earth  && !(i==0 && j==0)){
                new Zebra(habitat, f+i, c+j);
                return true;
            }
        }
    }
    return false;
}
```

```java
/**
 * Lyon reproduce if they are within a distance of two blocks of each other and there is earth between them.
 * @return if animals can reproduce
 */
public boolean procreate(){
    int[] pos = habitat.find(this);
    int f = pos[0];
    int c = pos[1];
    for(int i = -1; i<=1; i++){
        for(int j = -1; j<=1; j++){
            if(habitat.get(2*i+f , 2*j+c) instanceof Lyon && habitat.get(f+i, c+j) instanceof Earth && !(i==0 && j==0)){
                new Lyon(habitat, f+i, c+j);
                changeEnergy(-50);
                return true;
            }
        }
    }
    return false;
}
```

Lo que implica que refactoriza el metodo tac en ambas clases:

```java
/**
 * It change hasActed to false and eats grass.
 */
public void tac(){
    if(this.getEnergy() < 50){
        if(eatGrass()){
            changeEnergy((float) 0.25);
        }

    }else if(procreate()){

    }else if(eatGrass()){
        changeEnergy((float) 0.25);
    }

    hasActed=false;
}
```

```java
/**
 * It change hasActed to false and eats zebra.
 */
public void tac(){
    if(this.getEnergy() < 50){
        if(eatZebra()){
            changeEnergy((float) 0.5);
        }

    }else if(procreate()){

    }else if(eatZebra()){
        changeEnergy((float) 0.5);
    }

    hasActed=false;

}
```

Ahora se verifica la prueba:



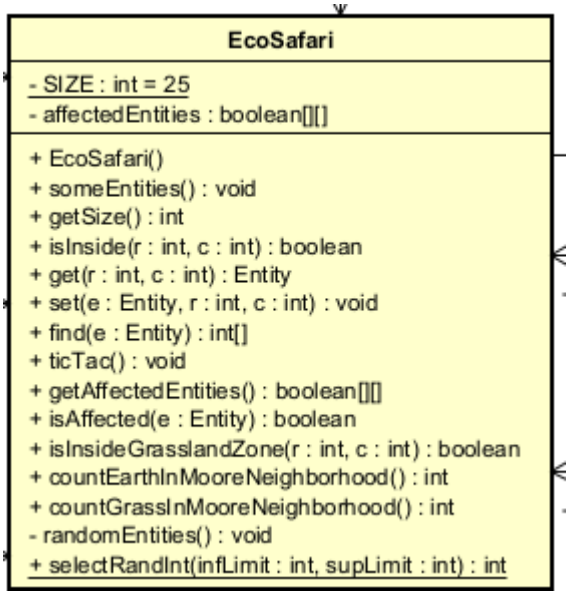


Ciclo 9. Creación del mapa al azar:

Para esta parte vamos a generar criaturas al azar dentro del mapa, dejando las ya establecidas en someEntities

```
/**
 * Puts random entities in ecoSafari.
 */
private void randomEntities(){
    for(int i =0; i < SIZE; i++){
        for(int j =0; j < SIZE; j++){
            if(get(i,j) instanceof Earth){
                int randNum = selectRandInt(0,100);
                if(!isInsideGrasslandZone(i,j)){
                    if(1 > randNum){
                        new Ovni(this, i, j);
                    }
                    else if(10 > randNum){
                        new Elephant(this, i,j);
                    }
                    else if(15 > randNum){
                        new AngryElephant(this, i, j);
                    }
                    else if(30 > randNum){
                        new Bush(this,i,j, false);
                    }
                }
                else{
                    if(5> randNum){
                        new Zebra(this,i,j);
                    }
                    else if(10 > randNum){
                        new Lyon(this, i, j);
                    }
                    else if(11> randNum){
                        new Grass(this,i,j);
                    }
                }

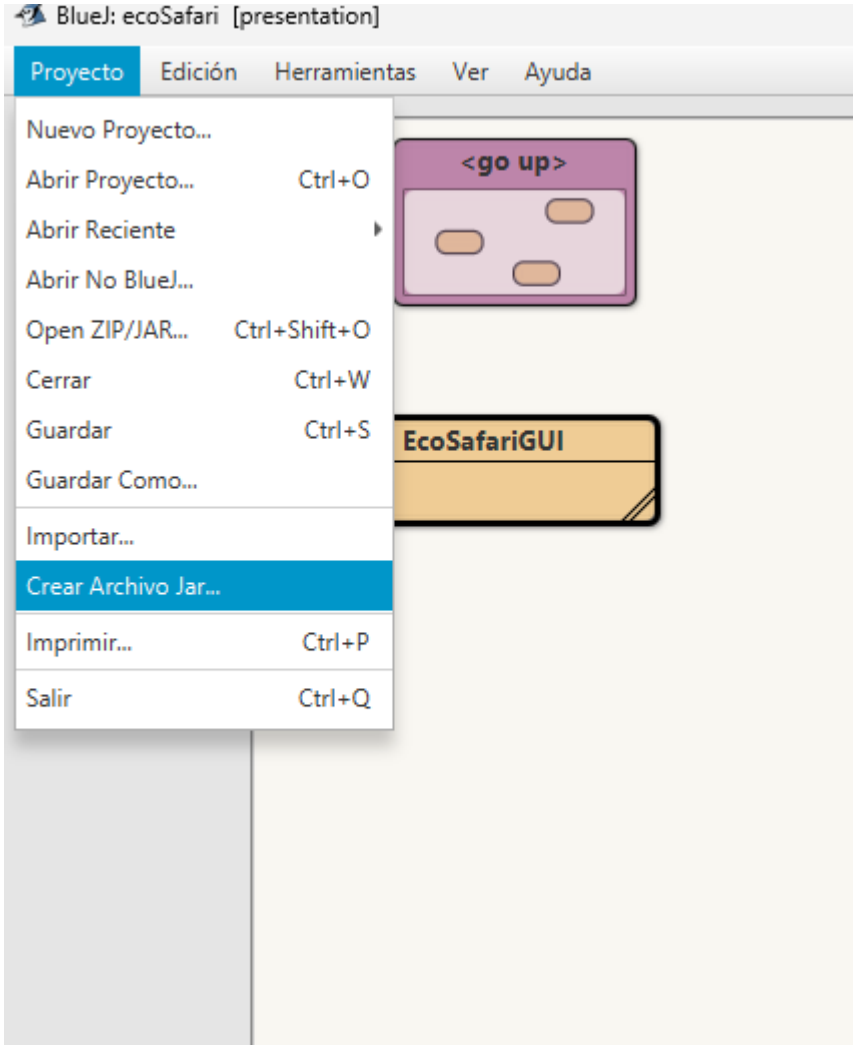
            }
        }
    }
```

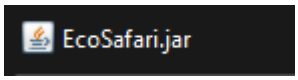


PARTE IV. Empaquetando el proyecto

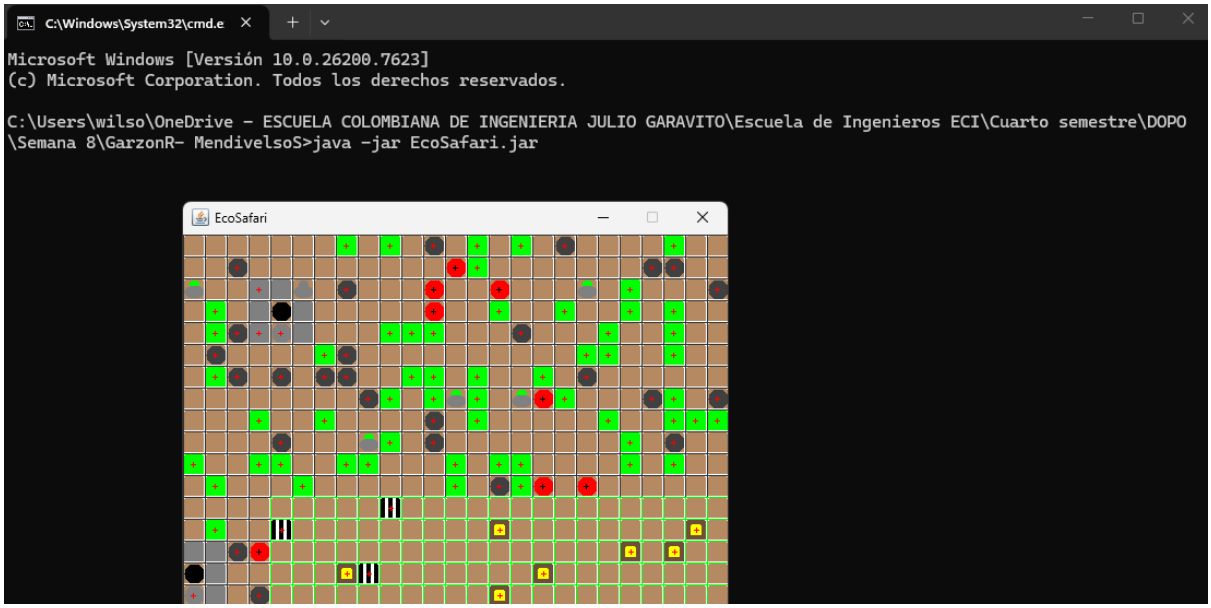
En un JAR (Java ARchive) se pueden empaquetar múltiples clases y archivos para distribuir y ejecutar artefactos software en un solo archivo comprimido.

1. Revise las opciones de BlueJ para empaquetar su programa en un archivo .jar. Genere el archivo correspondiente. (a) Incluya una pantalla con evidencia de la existencia del archivo.





2. Consulte el comando java para ejecutar un archivo jar. ejecútenlo (a) ¿qué pasa?



Al ejecutarlo se ejecuta el archivo .jar

3. (a) ¿Qué ventajas tiene esta forma de entregar los proyectos? Explique claramente.

Es más fácil hacer entregas de archivos, porque se encapsula todo el código y no es necesario tener cosas como BlueJ para acceder y poder usar el método main en determinado lado del código. Además, hace el envio mucho más limpio y permite que cualquier usuario con requerimientos mínimos sea capaz de probarlo, sin tener mayor idea de que como funciona java en su conjunto, simplemente ejecuta el archivo y listo.

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)

    -El tiempo invertido por cada uno fue de 14 horas

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

    - El estado actual es terminado, porque pudimos gestionar y desarrollar cada una de las partes dadas.

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

    - Volvemos a considerar que la mejor practica fue Pair Programing, Porque es la práctica que mejor se nos da, mientras que uno desarrolla, el otro va corrigiendo.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

-El mayor logro es simplemente que el simulador funcione, ya que aparte de todo lo que se debió de hacer, se tuvo que pensar bastante en que tan probable era que ciertos animales salieran para que todo estuviera equilibrado

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

- El problema técnico fue a la hora de entender EcoSafariGUI para poder dejar los animales de una mejor forma visual y también dejar el simulador restringido (la zona verde de los leones y cebras).

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

-Hicimos bien como equipo hacer la práctica de XP (Pair Programming). Nos comprometemos a implementar y explorar más practicas XP para futuros laboratorios.

7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

-Las únicas referencias que usamos son las notas de clase de la profesora María Irma Diaz, la más útil fueron las notas de clase de la semana 8